

A Framework for Compiling Custom Languages as Efficiently Verifiable Virtual Machines

Manuscript Draft v0.1.6

Assimakis A. Kattis
akis@kattis.io

Brian Klatt
Zorp Corp
brian@zorp.io

Philip Quirk
Zorp Corp
phil@zorp.io

Logan Allen
Zorp Corp
logan@zorp.io

June 11, 2025

Abstract

In this work, we develop a framework for compiling languages into efficient Interactive Oracle Proofs (IOPs, or ‘circuits’), motivated by applications in verifiable Virtual Machine (zkVM) design. We provide a set of sufficient conditions on a language under which it can be compiled into an efficient IOP, alongside corresponding performance costs. We identify a subclass of languages, which we denote as *traversable*, and demonstrate how traversable languages can be efficiently compiled as circuits using established techniques. To demonstrate the efficacy of our compilation framework, we develop a zkVM for the Nock programming language by (1) formalizing the existing Nock specification, and (2) applying our techniques to design an efficient IOP representation for the Nock VM. The resulting circuit is small, on par with existing state-of-the-art zkVM designs and can be generated for any traversable language in a generic way.

1 Introduction

Interactive Protocols (IPs) [BFL91], and more specifically Interactive Oracle Proofs (IOPs) [BSCS16], are cryptographic primitives that have been developed in order to efficiently verify the integrity of arbitrary computation. In this model, a potentially malicious prover needs to demonstrate knowledge of a witness with specific properties (or, alternatively, a witness satisfying a publicly-known circuit) to an honest verifier, doing so without revealing the witness itself. This setting is particularly useful in applications where cheap verifiability is desirable and has shown promise in the context of distributed payment systems and related applications. Although IOPs can be designed with a variety of different efficiency trade-offs in mind, a large body of work has focused on compiling them into Succinct Non-interactive Arguments of Knowledge (SNARKs) [BSCTV17], which are non-interactive protocols that are *succinctly verifiable*, i.e. the underlying proof object provided to the verifier is significantly smaller than the computation itself. These form a specific subset of Zero-Knowledge Proof (ZKP) systems, which form the general family that encompasses a variety of different verifiable compute gadgets (which may or may not be succinct).

Many contributions have focused on designing efficient SNARK gadgets with desirable properties. Given a circuit and potential witness candidate, the development of SNARKs with specific efficiency and trust-model guarantees has yielded many constructions that have been readily deployed in a variety of applications [Gro16, BSBHR18, GWC19, CHM⁺20]. However, as SNARKs become more effective at providing cryptographic attestations of computational integrity, it becomes important to ensure that *any* type of computable process can be efficiently encoded in a SNARK. To this end, the problem of efficiently encoding *generic computation* in a circuit that compiles into a SNARK starts to become more relevant. Zero-knowledge Virtual Machines (zkVMs) have shown particular promise in this regard, as they are the most general type of system that can perform this task. Indeed, a zkVM consists of a set of opcodes which can be sequenced in any order and which, if the opcodes are expressive enough, suffices to encode generic computation. The main objective of such systems is to

allow for the cryptographic attestation of any type of computation. In contrast to approaches focusing on the design of circuits for specific families of (fixed) programs, zkVMs are capable of transforming any type of computation to a version that can be efficiently verified by a fixed circuit. This property makes them particularly promising candidates for standardizing ZKP compute, usable across applications while incurring minimal overhead.

Thus, a fundamental question that needs to be answered is whether there exists a generic way to design zkVMs, *regardless* of the opcodes that they consist of or the language that they are written in. Until now, zkVM design has relied on either (1) the usage (or construction) of specific Instruction Set Architectures (ISAs) that are particularly amenable to being encoded as SNARK circuits, or (2) specific circuit design techniques that encode existing (yet fixed) ISAs in an efficient way. Although both of these approaches have yielded promising results, the questions of (1) what specific properties does an ISA need to satisfy in order to be efficiently arithmetizable as a circuit, and (2) whether there exists a generic mapping from an ISA specification to a zkVM architecture, remain open.

1.1 Related Work

Domain Specific Languages One approach tackles this efficiency problem by designing a custom Domain Specific Language (DSL) that allows efficient and modular circuit constructions. Depending on the underlying arithmetization of the proof system, the objective is to construct a language through which in-circuit constraints can be easily created while concurrently being expressive enough to be generically usable. The main benefit behind this approach lies in the concrete efficiency gains obtained due to the DSL’s customization, as the full freedom to interact with the in-circuit encoding in mind have been shown to yield order-of-magnitude improvements in performance [BMIMT⁺22], both with respect to concrete efficiency and circuit complexity. This becomes relevant in the context of easily auditing the system and ensuring that any potential implementation bugs are easily identified.

The Leo programming language [CWC⁺21] is a custom DSL designed to be efficient in the context of blockchain applications. It supports important ZKP-specific features, of which the most notable is the formal verification of the programs with respect to their higher-level specification. In addition, it provides tooling that allows for privacy preserving computation. In a similar vein, Noir [Azt25] is a DSL that has also been built with privacy-preserving computation in mind. These contributions demonstrate an additional benefit of custom DSLs: given the freedom to design all aspects of the DSL, novel features useful for specific applications can be effectively incorporated in the design.

Although the above benefits have uniquely positioned DSLs in developing verifiable solutions to specific problems, their main drawbacks come from the inherent customization of their standards. Given that a custom DSL designs specific circuits for each use case, as opposed to a ‘generic’ circuit for *all* computation, it does not allow for seamless transcribing and transformation of programs and tooling that has been created for more widely used systems, such as such as RISC-V and WebAssembly (WASM). Indeed, apart from the need to provide separate tooling infrastructure and independently harden the associated systems, something that comes with inherent complexity and additive costs, there exists a fundamental ‘switching’ cost in requiring end-users, who in their majority work with more standard programming languages and toolchains, to become familiar with specific DSLs.

General-purpose ISAs A second approach is to implement a widely used general-purpose ISA and its respective circuit, avoiding the method of providing different circuits for each type of computation that DSLs are used for. The fundamental motivation behind this is to resolve the concerns with the switching and fragmentation costs inherent to custom DSL designs, facilitating the compilation of widely used ISAs and their respective codebases into one type of circuit that can be used by all end-users. This is an extremely beneficial feature, significantly reducing the complexity associated with designing unique circuits for each use-case. The main drawback here is one of efficiency: indeed, given that the main ISA standards that are widely used currently were not designed with verifiable computation in mind, arriving at efficient ways to implement them within ZKP-compatible circuits is a non-trivial task and can be substantially less efficient without careful optimizations. Over the last couple of years, there have been a variety of high-impact contributions, most of which have focused on the RISC-V ISA.

One of the first efficient instantiations of a general-purpose zkVM was the TinyRAM construction in [BSCG⁺13], providing a proof-of-concept design for a zkVM tailored to efficiently verifying C programs within SNARKs. More recently, CairoVM [GPR21] provided an efficient CPU-based archi-

ture leveraging the AIR/RAP [Azt24] circuit-encoding arithmetization to construct an ISA with extremely efficient memory and proving overhead. The authors demonstrate that their design can prove statements using 128 trace elements per opcode alongside 46 transition gates to check for consistency of their operations, making the design of the CairoVM PL extremely efficient in minimizing the overhead incurred per opcode. The simplicity of their design also allows for formal security results for their protocol, which are also provided in the respective contribution.

The SP1 and Risc0 zkVMs [Suc25, Tea25] developed by Succinct and RiscZero are efficient systems for verifying RISC-V opcodes. They allow for compilation of RISC-V directly into state-of-the-art provers, with multiple optimizations to decrease proving costs and establish feasibility. Although efficient, there exists an inherent separation of in circuit complexity when compared to custom DSL solutions. At a high level, the fundamental RISC-V opcodes require at minimum thousands of columns and constraints due to the complexity of the RISC-V specification. This is indicative of the larger computational requirements inherent to such zkVMs, which can be seen as the drawback of supporting established compilation standards.

Generic Frameworks Finally, a newer approach to designing zkVMs involves making generic design choices at the circuit level that can be leveraged to support multiple different standards. This approach shows promise in that it could potentially incorporate benefits from both of the above families. Perhaps most similar to our work are the contributions in Jolt [AST24], in which a generic ISA frontend is constructed using lookup arguments. The Valida ISA [Lit25] provides a custom ISA specification and corresponding optimized ZKP circuits. Although not a DSL, Valida conjoins the design of opcodes to arrive at an extremely efficient zkVM while still supporting compilation from standard ISAs such as RISC-V and WASM. The architecture of the system is thus hybrid: there exist fundamental opcodes that have been specifically created for efficiency when encoded as circuits, while also supporting extensions for standard ISA support. Similarly, OpenVM [Axi25] also provides a generic framework to support multiple compilation targets without a fundamental alteration of the underlying zkVM architecture. These approaches all resemble a ‘generic’ framework in their modular approach and customized base ISA design.

1.2 Contributions

In this work, we are fundamentally interested in whether there exists a generic way to compile *any* programming language to a standard zkVM circuit architecture. Although similar to the approaches of [AST24, Lit25, Axi25], we are not just interested in whether compilation targets can be supported as extensions of some base custom ISA, but rather whether there exists a (hopefully generic) process by which any ISA can be compiled down to a cheap and efficient circuit architecture. The motivation for this is to be able to take any ISA, whether RISV-V, LLVM or specific to a given use-case, and compile it down to a simple yet efficient and easily auditable zkVM architecture with formal guarantees. In resolving this question, we are also interested in what *properties* a specific ISA needs to satisfy in order to be efficient and generically compilable. Although related, this is a distinct question about efficient polynomial representations of ISA primitives: does there exist an *equivalent representation* of the computational trace of some ISA that is both (1) efficient, and (2) generically compilable into a zkVM?

Sufficient Conditions for Generic Compilation Our first contribution is the identification of sufficient formal conditions that an ISA needs to satisfy in order to be efficiently compilable as a zkVM, alongside the requisite formal proofs of security. We specify two main requirements for generic compilation, although we leave open whether they are necessary preconditions. The first is that such ISAs need to be *memory-execution separable*, in that they admit representations of their computation for which the execution and memory traces can be verified independently of each other, i.e. the ISA can be decomposed into two different verification routines that only interact by sharing memory lookup operations. This is equivalent to using the Harvard architecture, splitting the verification of the ISA into checking opcode and memory sequences independently.

Perhaps most importantly, the second condition is based on the fundamental observation that, for a specific family of languages, both the memory and execution traces can be thought of as *verifiable traversals*. Indeed, much like how an Abstract Syntax Tree (AST) representation of program execution can be traversed in order to verify it, we can consider verification of the execution trace of some program as a stateful traversal over some n -ary tree. This is particularly useful in that it can be done extremely

efficiently: by using a recursive construction, the overhead to do this is in the order of tens of columns and gates when encoded as a RAP! This is independent of the size of the computation and whether there are multiple such traces being verified at a time, with the additional costs scaling only in the number of rows. Following this intuition, we design a specific circuit for verifying stateful traversals over a specific formal object we call a *recursively verifiable tree* as a RAP, and demonstrate that this object is powerful enough to act as a generic compilation engine.

Finally, we formally specify sufficient requirements for memory-execution separable ISAs to be compiled using only verifiable traversals over recursively verifiable trees. We denote these as *traversable* languages and provide a full formal framework for arriving at an efficient yet generic zkVM representation verifying both their execution and memory traces in a modular fashion. This is particularly interesting because the assumptions required for an ISA to be traversable are actually not particularly strong. We leave it as an open question whether established ISAs in general use today can be shown to satisfy these properties, which would immediately imply the existence of efficient zkVMs for any traversable ISA, based on the same underlying architecture.

NockVM: An Traversable zkVM In order to demonstrate the applicability of the above process, we provide formal results demonstrating the compilation of the Nock [YMDP16] ISA into a minimal zkVM. To do this, we first introduce a formal specification for Nock, in the process proposing a strengthening of the existing specification that allows us to demonstrate that Nock is indeed traversable and can thus be compiled down using our generic compilation techniques. This is necessary as the existing literature allowed for ambiguity, specifically in how the interpreter is designed. We propose an amendment to this, demonstrate that it is rigorously sound, and then provide a full specification of a zkVM that verifies Nock.

The zkVM design is efficient, with trace and gate counts similar to circuits of existing custom ISAs and about an order of magnitude more efficient than RISC-V. Although these comparisons are not direct due to the differences in opcode counts and features specified, we believe that they provide strong indication that our techniques can be leveraged to provide strong efficiency gains. As part of the generic compilation process, we provide formal results attesting to the security of NockVMROM. Over the Goldilocks prime field, it achieves a security level of 121 bits when instantiated with 512-bit length variables and a maximal number of 2^{31} unique data fields. It can verify computation using two RAP tables (or ‘chips’), for a total trace size of 395 field elements and 175 degree 4 transition gates, while the number of trace rows required is linear in the maximum of the total number of using Nock as a candidate zkVM, on par with state-of-the-art systems.

Indeed, even though Nock was designed and implemented without any ZKP applications in mind, the representation of the ISA in the form of traversals over the execution and memory traces allow for the generic compilation of its specification into a zkVM with efficiency comparable to that of ISAs purposefully built to be cheaply verifiable in-circuit. This is very encouraging, as the architecture has not been optimized yet remains efficient. Although to a large degree this is a result of finding the ‘correct’ representation of the execution trace as a set of traversable trees, which is facilitated by the sufficiency conditions of the compilation framework, this is also due to the minimal nature of the underlying ISA’s complexity. With only 10 opcodes in which all variables are represented as binary trees, Nock is a well-suited candidate for a minimal ISA that can yield efficient zkVM performance. We also note that, although not used extensively, Nock is a general-purpose ISA and thus demonstrates the wide applicability of our framework.

2 Preliminaries

2.1 Interactive Oracle Proofs

Definition 2.1. A pair of interactive PPT algorithms $(\mathcal{P}, \mathcal{V})$ is an *Interactive Oracle Proof system (IOP)* for some language $\mathcal{L}$ with $\epsilon : \{0,1\}^* \rightarrow [0,1]$ soundness and $k : \{0,1\}^* \rightarrow \mathbb{N}_+$ rounds of interaction if it satisfies the following properties:

1. *Completeness:*

$$\Pr [\langle P(x, w), V(x) \rangle = 1 | (x, w) \in \mathcal{L}] = 1,$$

2. *State-Restoration Soundness: There exists a PT extractor $\mathcal{E}_{\mathcal{L}}$, such that for all computationally unbounded malicious state-restoration provers $\mathcal{P}^*$ and statements x :*

$$\Pr \left[\begin{array}{c} \text{tr accepts} \\ (x, w) \notin \mathcal{L} \end{array} \middle| \begin{array}{c} \text{tr} \leftarrow \text{View}(\mathcal{P}^*(x), \mathcal{V}(x)) \\ w \leftarrow \mathcal{E}_{\mathcal{L}}(x, \text{tr}) \end{array} \right] \leq \epsilon(|x|),$$

with probabilities over the coins of $\mathcal{P}, \mathcal{V}$, engaging in at most $k(|x|)$ rounds of adaptive interaction.

The IOP protocol used is arithmetized using a specific set of semantics to encode the argument that will be proven. A common technique for doing this is Algebraic Intermediate Representation (AIR), and its more general form, a Randomized AIR with Preprocessing (RAP). This is specified below, and can be found in the literature.

Definition 2.2 (RAP). *A Randomized Algebraic Intermediate Representation with Preprocessing $\text{RAP} = (\mathbb{F}, (\mathbb{K}, \mathbf{e}), (k, \mathbf{w}_{\text{RAP}}), \mathbf{h}, \mathbf{d}, \mathbf{s}, (\omega, \beta), \mathbf{I}, \mathbf{Cset})$ is a tuple for which:*

1. $\mathbb{F}$ is the field over which the trace and polynomials are defined.
2. $(\mathbb{K}, \mathbf{e})$ correspond to the extension field $\mathbb{K}$ over $\mathbb{F}$ of degree $\mathbf{e}$.
3. $\mathbf{w}_{\text{RAP}}$ is a k -dimensional vector which denotes the number of columns corresponding to each round of interaction, during which the verifier responds with randomness $\rho_i, i \in [k]$ adaptively. Note that the first element of $\mathbf{w}_{\text{RAP}}$ corresponds to the pre-processing (selector) polynomials. We denote its size by $w_{\text{RAP}} := \sum_{i=1}^k \mathbf{w}_{\text{RAP}}(i)$.
4. $\mathbf{h}$ is the exponent value of the total length of the columns, which equals $2^{\mathbf{h}}$.
5. $\mathbf{d}$ is the maximal degree of the constraint gates.
6. $\omega \in \mathbb{F}$ has multiplicative order $\mathbf{e} \cdot 2^{\mathbf{h}}$ and $\beta \in \mathbb{F} \setminus \{0\}$. The commitment domain coset is $\mathbf{D} := \beta \langle \omega \rangle$ and the trace domain coset is $\mathbf{H} := \langle \omega^{1/\rho} \rangle$, where $\rho = 2^{\mathbf{h}}/|\mathbf{D}|$.
7. $\mathbf{l} \subseteq \{1, \dots, w_{\text{RAP}}\} \times \{0, \dots, 2^{\mathbf{h}} - 1\}$ corresponds to a subset of trace elements. The symbolic variables $\mathbf{Y} = \{\mathbf{Y}_{i,j} | (i, j) \in \mathbf{l}\}$ are indexed by $\mathbf{l}$.
8. $\mathbf{Cset} = (C_1, \dots, C_{\mathbf{s}})$ corresponds to the set of $\mathbf{s}$ constraint polynomials, where $C_i \in \mathbb{F}^{\leq d}[\mathbf{Y}]$ is an at-most degree d polynomial over the symbolic variables indexed by positions in the trace.

An assignment of a sequence of polynomials $\mathbf{P} = (P_1, \dots, P_{w_{\text{RAP}}})$ (known as the trace), with $P_i \in \mathbb{F}[X]$, to constraint polynomial $Q \in \mathbb{F}[\mathbf{Y}]$ is given by the univariate polynomial $Q \circ \mathbf{P} \in \mathbb{F}[X]$ obtained by replacing each symbolic variable $Y_{i,j}$ with $P_i(\omega^j \cdot X)$ in Q . The assignment $\mathbf{P}$ is said to satisfy RAP if and only if the following holds:

$$\forall i \in [\mathbf{s}] : x \in \mathbf{H} \Rightarrow (C_i \circ \mathbf{P})(x) = 0.$$

In the following sections, we will work with a RAP for which the following parameters are set:

1. $\mathbb{F} \equiv \mathbb{F}_p$ the cyclic group with modulus $p = 2^{64} - 2^{32} + 1$, known as the Goldilocks prime field,
2. $\mathbb{K}$ is taken as the degree $\mathbf{e} = 3$ extension over $\mathbb{F}_p$,
3. $\mathbf{l}$ is defined to include adjacent trace rows, giving $|\mathbf{Y}| = 2 \cdot w_{\text{RAP}}$ different symbolic variables.

We do not go into detail on the compilation of RAPs into SNARKs here, but state that there exist multiple frameworks that construct IOPs which efficiently verify that (a possibly malicious) prover has knowledge of a sequence of polynomials that satisfy any given RAP, with negligible state-restoration soundness error and perfect completeness. In this work, we focus on encoding techniques for establishing efficient RAP arithmetizations for a variety of different languages, which we will show suffice to generically compile a specific subset of programming language paradigms. As such, any techniques that generically compile an arbitrary RAP into a SNARK using IOPs can be leveraged, allowing for the usage of any such engine or primitive that may be applicable to the use-case at hand.

2.2 Offline Memory Verification

A fundamental concern when building a VM lies in the verification of program memory updates. In most cases, reading and writing from registers or the memory stack needs to be supported for feature-completeness, with various existing techniques providing for such functionality. Offline memory checking [BEG⁺94] is a framework developed for verifying the consistency of memory reads and writes, which suffices for the verification of program soundness. In this model, a memory access operation (either a read or write) is modeled as a triplet (a, v, t) , where a is the address encoding, v the value retrieved or written, and t the timestamp at which this happens. The verification process then entails checking that each read (a_r, v_r, t_r) is preceded by a write (a_w, v_w, t_w) to the same address, i.e. that $a_r = a_w$ of the same value $v_r = v_w$ so that $t_w < t_r$. Note that the last condition is necessary, as the state of each address necessarily changes over time.

Recent work [LZZ⁺24] has demonstrated that the above can be efficiently handled through the use of set-equality arguments. Moreover, a variety of different models of memory architectures have also been shown to reduce to set-equality, further demonstrating the power of this primitive. We provide the formal object we will be operating over below.

Definition 2.3. A memory encoding family $\text{MEM} = (\mathcal{A}, \mathcal{VD}, \mathcal{T})$ consists of an address space $\mathcal{A}$, a value domain $\mathcal{VD}$ and an ordered set $\mathcal{T}$ representing timestamps at which memory is read or written. A memory record (in the MEM encoding) is a tuple $(a, v, t) \in \mathcal{A} \times \mathcal{VD} \times \mathcal{T} := \text{Rec}^{\text{MEM}}$ and a memory record sequence of size k is an ordered list of k memory records, with universe $\text{Seq}_k^{\text{MEM}}$.

A closely related result is that of multiset equality, namely that two sets of distinct memory record sequences are equal up to permutation. This is useful when reasoning about Input-Output (IO) calls to memory, relating to the interaction between the computation of a VM and the underlying memory object, ensuring that all memory calls have been correctly resolved. We model this as two record sequences in a request-response model. The idea is that an execution trace will request memory values in a specific order, while a memory trace will respond in a (possibly different) order. This has the benefit of allowing for the decoupling of computation and memory, meaning that no ‘canonical trace’ object is needed. Indeed, by adding all memory requests to one multiset in the computation table and then verifying them all in another in the memory table, we can achieve extremely efficient IO linking between the two parts of the VM. This is an equivalent security notion to read-write memory consistency, where all reads have to correspond to a write in order for the memory requests to satisfy soundness. We provide some preliminary mathematical objects below, where Sym_n is defined as the permutation group over n elements.

2.2.1 Unique IO Record Matching

The first relevant language is that of matching each input record to a unique output. This is equivalent to read-once/write-once memory, since every request needs to be answered. Note that we require all requests to be unique, removing any additional complexity that multiplicities may add in this model.

$$\mathcal{L}_{\text{IO}}^{\text{MEM}, K} = \left\{ (\text{requests}, \text{responses}) \left| \begin{array}{l} \exists k \in [K] \text{ s.t. } \text{responses} \in \text{Seq}_k^{\text{MEM}} \\ \forall i, j \in [k] \text{ s.t. } i \neq j \Rightarrow \text{responses}(i) \neq \text{responses}(j) \\ \exists \sigma \in \text{Sym}_k \text{ s.t. } \sigma(\text{responses}) = \text{requests} \end{array} \right. \right\}$$

2.2.2 IO Record matching with Multiplicities

If the assumption that each request is unique is relaxed, we obtain a language that can be reduced to a lookup argument. In this case, the set of responses (which can come at any order) just needs to match each of the unique requests made, ignoring multiplicities. This is described below.

$$\mathcal{L}_{\text{mIO}}^{\text{MEM}, K} = \left\{ (\text{requests}, \text{responses}) \left| \begin{array}{l} \exists k \in [K] \text{ s.t. } \text{requests} \in \text{Seq}_k^{\text{MEM}} \\ \exists k_2 \leq k \text{ s.t. } \text{unique_requests} \in \text{Seq}_{k_2}^{\text{MEM}} \\ \forall i \in [k], \exists j \in [k_2] \text{ s.t. } \text{requests}(i) = \text{unique_requests}(j) \\ \forall i, j \in [k_2] \text{ s.t. } i \neq j \Rightarrow \text{unique_requests}(i) \neq \text{unique_requests}(j) \\ \exists \sigma \in \text{Sym}_{k_2} \text{ s.t. } \text{responses} = \sigma(\text{unique_requests}) \end{array} \right. \right\}$$

Lemma 2.1. For all MEM, K , it holds that $\mathcal{L}_{\text{IO}}^{\text{MEM}, K} \subsetneq \mathcal{L}_{\text{mIO}}^{\text{MEM}, K}$.

2.2.3 Multiset Arguments

We now provide the results that allows us to instantiate IOPs for the above languages. These are based on multiset equality, of which we provide two alternative constructions, multiplicative and log-derivative set equality. Moreover, for clarity of exposition we denote a random linear combination with a (random) variable Y of some array as $\text{RLC}^Y(\text{array}) = \sum_{i=1}^{|\text{array}|} Y^{i-1} \cdot \text{array}(i)$.

$$\mathcal{L}_{\text{MSE}}^{Y,K}(\mathbb{F}) = \left\{ (\text{requests}, \text{responses}) \left| \begin{array}{l} \exists k_1, k_2, m \in [K] \text{ s.t. } \text{requests} \in (\mathbb{F}^m)^{k_1}, \text{responses} \in (\mathbb{F}^m)^{k_2} \\ \prod_{i=1}^{k_1} (\text{RLC}^Y(\text{requests}(i)) + X) = \prod_{i=1}^{k_2} (\text{RLC}^Y(\text{responses}(i)) + X) \end{array} \right. \right\}$$

$$\mathcal{L}_{\text{LDSE}}^{Y,K}(\mathbb{F}) = \left\{ (\text{requests}, \text{responses}) \left| \begin{array}{l} \exists k_1, k_2, m \in [K] \text{ s.t. } \text{requests} \in (\mathbb{F}^m)^{k_1}, \text{responses} \in (\mathbb{F}^m)^{k_2} \\ \sum_{i=1}^{k_1} (\text{RLC}^Y(\text{requests}(i)) + X)^{-1} = \sum_{i=1}^{k_2} (\text{RLC}^Y(\text{responses}(i)) + X)^{-1} \end{array} \right. \right\}$$

Lemma 2.2. *Let $\{a_i(Y)\}_{i=1}^n$ and $\{b_i(Y)\}_{i=1}^n$ be multilinear polynomials over a field $\mathbb{F}$ with characteristic $p > n$, and $\mathbb{F}(X, Y)$ be the rational function field of $\mathbb{F}$ over X, Y . It follows that:*

$$\prod_{i=1}^n (X + a_i(Y)) = \prod_{i=1}^n (X + b_i(Y)) \text{ in } \mathbb{F}[X, Y] \Leftrightarrow \sum_{i=1}^n \frac{1}{X + a_i(Y)} = \sum_{i=1}^n \frac{1}{X + b_i(Y)} \text{ in } \mathbb{F}(X, Y).$$

Lemma 2.3. *If $\text{char}(\mathbb{F}) > K$, it follows that $\mathcal{L}_{\text{MSE}}^{Y,K}(\mathbb{F}) \equiv \mathcal{L}_{\text{LDSE}}^{Y,K}(\mathbb{F})$.*

We can thus denote both $\mathcal{L}_{\text{LDSE}}^{Y,K}(\mathbb{F}), \mathcal{L}_{\text{MSE}}^{Y,K}(\mathbb{F})$ as $\mathcal{L}_{\text{SE}}^{Y,K}(\mathbb{F})$ without any loss of generality if $\text{char}(\mathbb{F}) > K$, which will be considered to hold true in all our implementations of the relevant primitives. Given the above, we now show the equivalence and relationships between the IO languages and the underlying multiset arguments. We begin by demonstrating that set equivalence allows for strictly more than just matching unique inputs and output memory records, even without any ordering. We can arrive at a precise equivalence by noting that the multiset arguments exactly capture the IO language if they are restricted to sequences of unique elements, defined below.

$$\mathcal{L}_{\text{uSE}}^{Y,K}(\mathbb{F}) = \left\{ (\text{requests}, \text{responses}) \left| \begin{array}{l} (\text{requests}, \text{responses}) \in \mathcal{L}_{\text{SE}}^{Y,K}(\mathbb{F}) \\ \forall i, j \in [|\text{responses}|] \text{ s.t. } i \neq j \Rightarrow \text{responses}(i) \neq \text{responses}(j) \end{array} \right. \right\}.$$

Lemma 2.4. *For the memory encoding family $\text{MEM} := \mathbb{F}^m$, if $\text{char}(\mathbb{F}) > K \geq m$ then:*

1. $\mathcal{L}_{\text{IO}}^{\text{MEM},K} \subsetneq \mathcal{L}_{\text{SE}}^{Y,K}(\mathbb{F})$,
2. $\mathcal{L}_{\text{IO}}^{\text{MEM},K} \equiv \mathcal{L}_{\text{uSE}}^{Y,K}(\mathbb{F})$.

The final ingredient needed is a lemma based on the unique fractional decomposition of logarithmic derivatives, showing that we can prove the equivalence of the mIO language to a specific form of the log-derivative multiset argument. These are provided below

Lemma 2.5. *Let $\{a_i(Y)\}_{i=1}^{n_1}$ and $\{b_i(Y)\}_{i=1}^{n_2}$ be sequences of multilinear polynomials over a field $\mathbb{F}$ with characteristic $p > n_1, n_2$, represented as surjective functions $a : [n_1] \rightarrow S$ and $b : [n_2] \rightarrow T$, with image sets $S, T \subset \mathbb{F}(Y)$. Then $S \subseteq T$ if and only if there exists a sequence $\{m_i\}_{i=1}^{n_2}$ of field elements from $\mathbb{F}_p \subseteq \mathbb{F}$ such that:*

$$\sum_{i=1}^{n_1} \frac{1}{X + a_i(Y)} = \sum_{i=1}^{n_2} \frac{m_i}{X + b_i(Y)} \text{ in } \mathbb{F}(X, Y).$$

Moreover, if $S = T$ then there is a choice of m_i such that $m_i \neq 0$ for every $i \in [n_2]$; conversely, if $b_i \neq b_j$ when $i \neq j$ and $m_i \neq 0$ for every $i \in [n_2]$, then $S = T$.

$$\mathcal{L}_{\text{umLDSE}}^{Y,K}(\mathbb{F}) = \left\{ (\text{requests}, \text{responses}) \left| \begin{array}{l} \exists k_1, k_2, M \in [K] \text{ s.t. } \text{requests} \in (\mathbb{F}^M)^{k_1}, \text{responses} \in (\mathbb{F}^M)^{k_2} \\ \exists m \in (\mathbb{F}_{\text{char}(\mathbb{F})} \setminus \{0\})^{k_2} \\ \forall i, j \in [|\text{responses}|] \text{ s.t. } i \neq j \Rightarrow \text{responses}(i) \neq \text{responses}(j) \\ \sum_{i=1}^{k_1} (\text{RLC}^Y(\text{requests}(i)) + X)^{-1} = \sum_{i=1}^{k_2} m(i) \cdot (\text{RLC}^Y(\text{responses}(i)) + X)^{-1} \end{array} \right. \right\}$$

Lemma 2.6. *For $\text{MEM} := \mathbb{F}^M$, if $\text{char}(\mathbb{F}) > K \geq M$ then $\mathcal{L}_{\text{mIO}}^{\text{MEM},K} \equiv \mathcal{L}_{\text{umLDSE}}^{Y,K}(\mathbb{F})$.*

2.3 Recursive Tree Representations

In order to construct an efficiently verifiable zkVM, one of the most fundamental primitives that needs to be designed is its representation of the underlying memory and related computational objects. This should critically allow for both a succinct representation of large data arrays (regardless of the complexity of the overarching structure) as well as cheaply verifiable arbitrary computation. Although many techniques in the literature have been developed to such ends, including homomorphic commitments for arbitrary data or Merkle trees for ordering data arrays that allow for cheap inclusion proofs, none of these have been able to provide a general-purpose representation of VM computation that concurrently permits cheap verifiability of all underlying opcodes.

In order to achieve this, we opt for representing the VM computation and data as binary tree elements. This comes with the benefit that such elements are inherently structured and can thus be efficiently represented in a way that provides both homomorphic properties while retaining succinctness. Indeed, by limiting the VM to operate solely over binary trees, obtaining a succinct representation of such trees suffices for greatly simplifying the elements over which the verification process operates without requiring any modifications or extensions. This is not only important for simplicity, as it greatly reduces the potential for unintuitive errors or difficult auditability, but also allows for extremely efficient operations when designed with an overarching Programming Language paradigm that operates solely over binary trees as the only stack object.

2.3.1 Variables as S -Expressions

A natural starting point for reasoning about the inherent variable over which computation is performed is the S -expression. Defined recursively, this variable forms the building block for many programming language paradigms in use today, most notably by being the backbone of the List Processing (LISP) family. Apart from its simplicity, the S -expression is also desirable in this context due to its formality. Indeed, S -expressions are both simple to define but also general enough to be used in any context. We will show that a suitable restriction of their usage is especially versatile and efficient as a base semantic primitive for a minimally efficient VM architecture. S -expressions are of two types, ‘atoms’ or ‘cells’, depending on the number of elements that they contain. We denote by $\text{type} : \mathcal{S} \rightarrow \{\text{cell}, \text{atom}\}$ the functional over the set $\mathcal{S}$ of all S -expressions that identifies the two types. The former is a singular element, while the latter is a pair of elements that are also S -expressions.

Definition 2.4 (S -Expression). *An S -expression $s \in \mathcal{S}$ is an element of the following language:*

$$\mathcal{L}_{\mathcal{S}\text{-Exp}} = \{s \in \mathcal{S} \mid (\text{type}(s) = \text{atom}) \cup (\text{left}(s) \in \mathcal{L}_{\mathcal{S}\text{-Exp}} \cap \text{right}(s) \in \mathcal{L}_{\mathcal{S}\text{-Exp}})\}.$$

However, the above definition hides a subtle complexity that would make it challenging to use ‘as is’ in any generic VM design. It is possible for an S -expression to be circular - i.e. the elements in the pair can reference each other. This means that, although intuitively the above defines a binary tree structure, it can in principle result in a graph structure with cycles - as nothing prevents its elements from forming a circular topology. Indeed, this is a well known issue in LISP-based languages and has resulted in changes to the variable representation of S -expressions (that are usually binary tree lists) to accommodate this generality. In the context of defining a general VM, this is a concern and increases the complexity of the underlying variable representation. Since the efficiency of this primitive is critical and circularity is not a high-priority feature of most programming paradigms (while also not affecting the inherent expressivity of the language itself), we choose to *restrict* the underlying variable to a binary tree so as not to support this feature. Thus, although technically not an S -expression, the variable retains all benefits of the S -expression formalism while being extremely efficient to represent.

2.3.2 Proper Binary Trees

We define the set of all binary trees as $\mathcal{T}$ and restrict ourselves to *proper binary trees* as the fundamental data structure to operate over. This is defined as a binary tree whose nodes each have either zero or two children. A recursive definition, alongside some basic facts, is provided below, where the functional $\text{size} : \mathcal{T} \rightarrow \mathbb{N}$ returns the number of nodes in $t \in \mathcal{T}$ and $\text{left} : \mathcal{T} \rightarrow \mathcal{T}$, $\text{right} : \mathcal{T} \rightarrow \mathcal{T}$ return the left and right subtrees of the subject tree respectively. Note that the support for circularity (and hence generic graph objects) is removed by enforcing that the children of each proper binary tree are disjoint.

Definition 2.5 (Proper Binary Tree). *A proper (or full) binary tree $t \in \mathcal{T}$ is an element of the following language:*

$$\mathcal{L}_{\text{full_BT}} = \{t \in \mathcal{T} \mid (\text{size}(t) = 1) \cup (\text{left}(t) \in \mathcal{L}_{\text{full_BT}} \cap \text{right}(t) \in \mathcal{L}_{\text{full_BT}} \cap (\text{left}(t) \cap \text{right}(t) = \emptyset))\},$$

while the restriction to proper binary trees of size λ is given by:

$$\mathcal{L}_{\text{full_BT}}^\lambda = \{t \in \mathcal{L}_{\text{full_BT}} \mid \text{size}(t) = \lambda\}.$$

Definition 2.6. *Given $t \in \mathcal{L}_{\text{full_BT}}$, a node in t that has no children is called a leaf node, while a node in t that is not a leaf node is an internal node.*

Lemma 2.7. *A tree $t \in \mathcal{L}_{\text{full_BT}}^{2\lambda-1}$ has $\lambda - 1$ internal and λ leaf nodes.*

In order to represent proper binary trees optimally, we define a dyck word as an element of the following context-free language

$$\mathcal{L}_{dk} = \{w \in \{0, 1\}^* \mid w = \epsilon \text{ or } \exists w_1, w_2 \in \mathcal{L}_{dk} \text{ s.t. } w = 0w_11w_2\},$$

$$\mathcal{L}_{dk}^\lambda = \{w \in \mathcal{L}_{dk} \mid |w| = \lambda\},$$

and note that Lemma 2.8 below immediately implies that the dyck representation of proper binary trees is an optimal representation.

Lemma 2.8. $\mathcal{L}_{dk} \equiv \mathcal{L}_{\text{full_BT}}$ and $\forall \lambda \in \mathbb{N}, \mathcal{L}_{dk}^{2\lambda} \equiv \mathcal{L}_{\text{full_BT}}^{2\lambda+1}$ and $|\mathcal{L}_{dk}^{2\lambda}| = C_\lambda$, the λ -th Catalan number.

Given the above structure, the final ingredient required becomes the storage of information in the leaves of the binary tree. We denote by $\mathcal{DT}(\mathbb{F}) = \cup_{\lambda \in \mathbb{N}^+} \mathbb{F}^\lambda$ the universe of all orderings of binary tree leaves, where each leaf is an element from some field $\mathbb{F}$. By Lemma 2.7, to each proper binary tree t with $2 \cdot \lambda - 1$ nodes we can assign an ordered array of λ field elements $dt \in \mathcal{DT}(\mathbb{F})^\lambda$ so that the tuple (t, dt) corresponds to a unique proper binary tree whose leaves are labeled by the elements of dt , which we define as being a **noun**. Note that the label on each leaf corresponds to a unique field element and will be used to store data. In all subsequent sections, we refer to $\mathcal{L}_{dk}^\lambda, \mathcal{DT}(\mathbb{F})^\lambda$ as the restrictions of the given languages to their corresponding elements of size λ .

Definition 2.7. *The tuple $n := (t, dt) \in \mathcal{L}_{dk}^{2 \cdot (\lambda-1)} \times \mathcal{DT}(\mathbb{F})^\lambda := \mathcal{N}^\lambda(\mathbb{F})$ is a noun of size λ over field $\mathbb{F}$.*

2.3.3 Extension to Proper n -ary Trees

The above approach to providing an optimal representation for proper binary trees can be naturally extended to proper n -ary trees. Although we do not make use of this generalization here, the exact same techniques can be leveraged to arrive at the same optimal representations. These can then be leveraged using the primitives specified in the following sections to provide an additional parameter than can be used to optimize VM architecture representations. Thus, we provide the requisite definitions and theorems below for completeness. Note that the size functional (total number of nodes) and definitions for internal and leaf nodes remain unchanged from the above definition and are not re-specified. The only additional functional is given by $\text{child}_i : \mathcal{T}^n \rightarrow \mathcal{T}^n$, which maps some n -ary tree $t \in \mathcal{T}^n$ to its child at index i . Note this generalizes the **left**, **right** functionals above to an arbitrary number of children.

Definition 2.8 (Proper n -ary Tree). *A proper (or full) n -ary tree $t \in \mathcal{T}^n$ is an element of the following language:*

$$\mathcal{L}_{\text{full_T},n} = \left\{ t \in \mathcal{T}^n \mid (\text{size}(t) = 1) \cup \left(\left(\bigcap_{1 \leq i \leq n} \text{child}_i(t) \in \mathcal{L}_{\text{full_T}^n} \right) \cap \left(\emptyset = \bigcap_{1 \leq i \leq n} \text{child}_i(t) \right) \right) \right\},$$

while the restriction to proper n -ary trees of size λ is given by:

$$\mathcal{L}_{\text{full_T},n}^\lambda = \{t \in \mathcal{L}_{\text{full_T}^n} \mid \text{size}(t) = \lambda\}.$$

3 A Verifiable VM Compilation Framework

It is useful to consider the most general model of computation when defining the gadgets and architecture for a verifiable VM. In all programming paradigms, computation can be distilled into two separate processes: the execution of a sequence of opcodes, instantiated through the program’s *execution trace*, and the storage of its memory, whether that is Read-Only or Random-Access Memory (ROM/RAM), in its *memory trace*. When building a verifiable VM, both of these objects need to be formally checked to ensure that the computation has been performed correctly. In previous contexts, both of the above are instantiated inside a ‘global trace’ object, which performs both opcode resolution and memory checking in tandem. Although this is natural in that it accurately depicts how present-day processing operates *in silico*, we find that this intuition does not necessarily translate efficiently in the context of formal verification, where the elements of the execution and memory are all known beforehand and do not need to be computed in an online fashion.

To this end, we formalize the notion of *execution-memory separation*, wherein the execution trace, resolving opcode computations, and the memory trace, checking memory reads/writes, are formally verified independently of each other. In this model, two distinct processes need to be performed in order to provide soundness guarantees for the underlying computation:

1. Compute Table: This is an object that verifies the execution trace of the program. It takes as input a representation of the initial program state and has three distinct functions:
 - Serialization of the correct sequence of opcodes, depending on the initial state
 - Checking that these opcodes have been resolved correctly based on the memory, given an optimistically provided set of reads and writes to the memory stack
 - Outputting the final valid state of the program (which can be an error/crash flag)
2. Memory Table: This object verifies the full memory trace (or stack) of the program and takes as input the optimistically generated set of reads and writes from the compute table alongside a representation of the initial memory stack:
 - Checking that the representation of the initial memory stack is valid
 - For the optimistically provided set of reads and writes from the compute table:
 - (a) Ensuring that all updates (writes) to memory are performed correctly
 - (b) Ensuring that the read sequences provided are valid

Definition 3.1. A language of program-trace pairs $\mathcal{L}_{\text{Lang}} = \{x | \exists \mathbf{T} \text{ s.t. } \text{LangVM}(x, \mathbf{T}) = 0\}$, recognizing a verifier LangVM for a program x with some trace $\mathbf{T}$ as a witness, is uniquely memory-execution separable over memory encoding MEM if there exist (1) bijections $x \rightarrow x^* := (x_{\text{prog}}, x_{\text{mem}})$ and $\mathbf{T} \rightarrow \mathbf{T}^* := (\mathbf{T}_{\text{prog}}, \mathbf{T}_{\text{mem}})$ and (2) verification functions LangVMComputation , LangVMMemory such that:

$$x \in \text{LangVM} \Leftrightarrow \exists (x^*, \mathbf{T}^*, \text{IO} := (\text{IO}_{\text{prog}}, \text{IO}_{\text{mem}})) \text{ s.t. } \begin{aligned} & \text{LangVMComputation}(x_{\text{prog}}, \mathbf{T}_{\text{prog}}, \text{IO}_{\text{prog}}) = 0 \\ & \text{LangVMMemory}(x_{\text{mem}}, \mathbf{T}_{\text{mem}}, \text{IO}_{\text{mem}}) = 0 \\ & (\text{IO}_{\text{prog}}, \text{IO}_{\text{mem}}) \in \mathcal{L}_{\text{mIO}}^{\text{MEM}} \end{aligned}$$

Note that the IO variable encodes the shared object that communicates between the compute and memory tables - i.e. the memory read/writes that are ‘shared’ between the separate tables and which ensure that the two tables are operating consistently with the underlying computation. Moreover, it also requires that the IO variables are semantically equivalent to an array pair of memory records, of which the requests are all answered by the responses, which are further constrained to be unique.

The above formulation is particularly useful in formally verifying a VM, since it provides a very specific decomposition of which components need to be checked. Indeed, if we can demonstrate that some VM language Lang is uniquely memory-execution separable over some memory record representation MEM , we would only require the verification of the corresponding LangVMComputation , LangVMMemory methods, alongside a final check that the IO variables they use are elements of $\mathcal{L}_{\text{mIO}}^{\text{MEM}}$.

3.1 Input-Output Consistency

We apply the above paradigm in the context of a RAP arithmetization. Our objective here is to develop generic, yet efficient, transformations of an ISA into a memory-execution separable version. We restrict ourselves from the universe of memory-execution separable languages within the RAP arithmetization to ones that are encoded in a ‘canonical’ way, including two independently verifiable RAPs - the ‘compute’ and ‘memory’ tables.

Definition 3.2. *A uniquely memory-execution separable language recognized by LangVM over memory encoding MEM is considered to have a canonical RAP arithmetization if there exist:*

1. *The following common RAP parameters:*

- (a) *a finite field $\mathbb{F}$,*
- (b) *an extension field $\mathbb{K} := E(\mathbb{F})$ of degree e ,*
- (c) *elements $h, d, (\omega, \beta)$,*
- (d) *witness vector $\mathbf{w}_{\text{RAP}}$, and*

2. *the following constraint polynomials, subset index sets and column interaction variables:*

- (a) $\mathbf{V}_{\text{prog}} := (\mathbf{C}_{\text{prog}}, \mathbf{l}_{\text{prog}}, \mathbf{w}_{\text{prog}})$ *with $\mathbf{w}_{\text{prog}} \subseteq \mathbf{w}_{\text{RAP}}$, and*
- (b) $\mathbf{V}_{\text{mem}} := (\mathbf{C}_{\text{mem}}, \mathbf{l}_{\text{mem}}, \mathbf{w}_{\text{mem}})$ *with $\mathbf{w}_{\text{mem}} \subseteq \mathbf{w}_{\text{RAP}}$,*

such that:

$\text{LangVMComputation}(\mathbf{W}_{\text{prog}}, \mathbf{IO}_{\text{prog}}) = 0 \Leftrightarrow (\mathbf{W}_{\text{prog}}, \mathbf{IO}_{\text{prog}})$ *is a satisfying assignment for RAP_{prog} ,*

$\text{LangVMMemory}(\mathbf{W}_{\text{mem}}, \mathbf{IO}_{\text{mem}}) = 0 \Leftrightarrow (\mathbf{W}_{\text{mem}}, \mathbf{IO}_{\text{mem}})$ *is a satisfying assignment for RAP_{mem} ,*

where

$$\text{RAP}_{\text{prog}} := (\mathbb{F}, (\mathbb{K}, e), (|\mathbf{w}_{\text{prog}}|, \mathbf{w}_{\text{prog}}), h, d, |\mathbf{C}_{\text{prog}}|, (\omega, \beta), \mathbf{l}_{\text{prog}}, \mathbf{C}_{\text{prog}}),$$

$$\text{RAP}_{\text{mem}} := (\mathbb{F}, (\mathbb{K}, e), (|\mathbf{w}_{\text{mem}}|, \mathbf{w}_{\text{mem}}), h, d, |\mathbf{C}_{\text{mem}}|, (\omega, \beta), \mathbf{l}_{\text{mem}}, \mathbf{C}_{\text{mem}}).$$

Definition 3.3. *A uniquely memory-execution separable language recognized by LangVM over memory encoding MEM with a canonical RAP representation is considered opcode and memory one-hot of size N_{OP} and arity M_{OP} if there exists a set of N_{OP} different opcodes $\text{OP} = \{\text{op}_1, \dots, \text{op}_{N_{\text{OP}}}\}$ and a subset of opcodes $\text{OP}_{\text{IO}} \subseteq \text{OP}$ that take in at most M_{OP} memory records each. More specifically, the memory-execution separation bijection $(x, \mathbf{T}) \rightarrow (x^*, \mathbf{T}^*)$ satisfies the semantics below.*

1. *One opcode is resolved per row in RAP_{prog} :*

- (a) $x_{\text{prog}} \in \{0, 1\}^{h_1 \times N_{\text{OP}}} \subseteq \mathbb{F}^{h_1 \times N_{\text{OP}}}$ N_{OP} *columns with $h_1 \leq 2^h$.*
- (b) $x_{\text{prog}}(i, j)$ *equals 1 if the j -th opcode is active in row i and is 0 otherwise.*
- (c) *If $x \in \text{LangVM}$, $\sum_{j=1}^{N_{\text{OP}}} x_{\text{prog}}(i, j) = 1$ for all rows $i \in [h_1]$.*

2. *One memory record is verified per row in RAP_{mem} : $x_{\text{mem}} \in \text{MEM}^{h_2} \subseteq \mathbb{F}^{h_2 \times N_{\text{mem}}}$ denoting the verification of $h_2 \leq 2^h$ memory records where $\text{MEM} \subseteq \mathbb{F}^{N_{\text{mem}}}$, i.e. N_{mem} denotes the number of elements from $\mathbb{F}$ needed to represent an element in MEM.*

3. *Each opcode reads at most M_{OP} memory record segments: $\mathbf{IO}_{\text{comp}} \in \text{MEM}^{h_1 \times M_{\text{OP}}} \subseteq \mathbb{F}^{h_1 \times M_{\text{OP}} \times N_{\text{mem}}}.$*

4. *Only OP_{IO} opcodes request records:*

- (a) *There exists a mapping $\text{regs} : \text{OP}_{\text{IO}} \rightarrow [M_{\text{IO}}]$ providing the number of memory records requested per memory requesting opcode, which is the same for all $x \in \text{LangVM}$.*
- (b) *If $x \in \text{LangVM}$, then $\forall i \in [h_1]$:*

$$\sum_{j \in \text{OP}_{\text{IO}}} x_{\text{prog}}(i, j) = 0 \Rightarrow \mathbf{IO}_{\text{prog}}(i, \cdot) = \mathbf{0}^{M_{\text{OP}} \times N_{\text{mem}}}$$

$$j \in \text{OP}_{\text{IO}}, x_{\text{prog}}(i, j) = 1 \Rightarrow k \in [M_{\text{OP}}] \setminus [\text{regs}(j)], \mathbf{IO}_{\text{prog}}(i, k) = \mathbf{0}^{N_{\text{mem}}},$$

5. *IO records are resolved in-place:* $\text{IO}_{\text{mem}} \subseteq x_{\text{mem}}, \text{IO}_{\text{mem}} \in \text{MEM}^{\mathbf{h}_R} \subseteq \mathbb{F}^{\mathbf{h}_2 \times \mathbf{N}_{\text{mem}}}$, where $\mathbf{h}_R \leq \mathbf{h}_2$.

Definition 3.4. For a language satisfying Definition 3.3, define the total number of memory requesting opcodes req_ops and total memory record requests mem_ops for a given tuple $(x_{\text{prog}}, \text{IO}_{\text{prog}})$ as:

$$\text{req_ops}(x_{\text{prog}}) := \sum_{i=1}^{\mathbf{h}_1} \sum_{j \in \text{OP}_{\text{IO}}} x_{\text{prog}}(i, j), \text{mem_ops}(\text{IO}_{\text{prog}}) := \sum_{i=1}^{\mathbf{h}_1} \sum_{j=1}^{\mathbf{M}_{\text{OP}}} \mathbf{1}[\text{IO}_{\text{prog}}(i, j) \neq \mathbf{0}^{\mathbf{N}_{\text{mem}}}],$$

where $\mathbf{1}[\cdot]$ the indicator function.

Definition 3.2 can be interpreted as a refinement of the *modular verification* of the memory and execution of any program x using a RAP encoding. It specifies a compute and memory table, each restricting to circuits encoded by the polynomial constraints $\mathbf{C}_{\text{prog}}$ and $\mathbf{C}_{\text{mem}}$ for **LangVMComputation**, **LangVMMemory** respectively. Note that this is possible because of Definition 3.1, as for some valid program-trace representation $(x, \mathbf{T})$, we can transform these into independent statement-witness pairs for the compute and memory tables and thus (if their constraints can be represented using algebraic constraints), build them using two RAP languages.

We now briefly describe the semantics in Definition 3.3. To begin with, we encode opcode resolution in the ‘one-hot’ paradigm: each opcode has its own column in the compute table, with elements constrained to booleanity and such that only one element is active at any point. In general, opcode resolution requires the fetching of data from memory and thus a set of ‘requests’ to the memory stack of the VM. The opcodes with memory requests are a subset of all opcodes OP_{IO} , and when their relevant selectors are set to 1, they will need to pull a number of memory records from ROM in order to perform opcode resolution. The maximal such number per opcode is the *memory arity* $\mathbf{M}_{\text{OP}}$ of the language. These memory requests are encoded in IO_{prog} , which are equivalent to local memory registers. Note that the compute table does not immediately check their veracity. Below we summarize the design of the trace that will be built. We will show that, by only adding e more columns, we can extend the provided RAPs to check for the consistency of IO with minimal overhead on the tables and verifier. This will be achieved using a set of accumulating columns $\text{acc}_{\text{prog}}, \text{acc}_{\text{mem}}$, based on multiset equality.

1. A compute table **compute**, consisting of:

- (a) A column of the opcodes that need to be resolved (i.e. the x_{prog} object)
- (b) A set of columns that witness opcode resolutions at each row (i.e. the $\mathbf{T}_{\text{prog}}$ object)
- (c) The memory request registers (i.e. the IO_{prog} object)
- (d) A memory request accumulator $\text{acc}_{\text{prog}} \in \mathbb{K}^{\mathbf{h}_1} \subseteq \mathbb{F}^{\mathbf{h}_1 \times e}$, which will contain all the requests (reads) from memory needed by the opcode resolution columns
- (e) A terminal output variable **requests** $\in \mathbb{K} \subseteq \mathbb{F}^e$, which is the last element of acc_{prog}

2. A memory table **memory**, consisting of:

- (a) A set of columns that contain the memory records to be verified (i.e. the x_{mem} object)
- (b) A set of columns witnessing memory verification (i.e. the $\mathbf{T}_{\text{mem}}$ object)
- (c) A memory response accumulator $\text{acc}_{\text{mem}} \in \mathbb{K}^{\mathbf{h}_2} \subseteq \mathbb{F}^{\mathbf{h}_2 \times e}$, which will contain all the responses from memory needed by the compute table **compute**
- (d) An initial input variable **responses** $\in \mathbb{K} \subseteq \mathbb{F}^e$, which is the first element of acc_{mem}

3.1.1 Compute Table IO Semantics

The RAP constraint system techniques in this and the following section are not novel, having seen widespread use through the ‘LogUp’ argument [Hab22, EH24], although we do provide a formal setting for their application. We denote by $\text{Com}(\cdot)$ a binding commitment scheme, as per standard RAP interactive constructions. Below we define a new RAP $\text{RAP}_{\text{prog-IO}}$ with an additional final round of interaction.

$\text{RAP}_{\text{prog-IO}}$ Table Construction

1. Prover engages with the Verifier as per RAP_{prog} to build $(x_{\text{prog}}, T_{\text{prog}}, \text{IO}_{\text{prog}})$
2. Prover sends $\text{Com}(\text{IO}_{\text{prog}})$ to the Verifier
3. Verifier responds with random elements $\alpha, \beta \in \mathbb{K}$
4. Prover builds e columns encoding the accumulating multiset as an extension field element, denoted as $\text{acc}_{\text{prog}} \in \mathbb{K}^{\mathbf{h}_1} \subseteq \mathbb{F}^{\mathbf{h}_1 \times e}$, in a fixed basis representation of $\mathbb{K}$ over $\mathbb{F}$

We denote the union of the above as the *compute table IO trace*, $T_{\text{IO}}^{\text{prog}} := (\text{IO}_{\text{prog}}, \text{acc}_{\text{prog}}) \in \mathbb{F}^{\mathbf{h}_1 \times (\text{M}_{\text{OP}} + e)}$, whose function it will be to aggregate all read requests to the ROM into the accumulator and output it as a terminal variable. Note that we optimistically complete the elements of acc_{prog} , as they will be checked in the memory table. Thus, in order to commit to them securely, we add a layer of interaction between the prover and verifier in between building IO_{prog} and acc_{prog} that commits to IO_{prog} . This suffices to build the multiset and provides the soundness guarantees required.

Below we specify the constraints between elements in $T_{\text{IO}}^{\text{prog}}$ and x_{prog} , the table of opcode selectors for the compute table, using the log-derivative multiset argument. All constraints will be between adjacent rows, denoted by indices i and $i + 1$, which suffice to define the symbolic variables in $\text{I}_{\text{prog-IO}}$ of the RAP.

$\text{RAP}_{\text{prog-IO}}$ Constraints

1. Transition Constraints

- (a) Define the IO selector constraints on row i as $\text{io}(i) := \sum_{j \in \text{OP}_{\text{IO}}} x_{\text{prog}}(i, j)$
- (b) Log-Derivative Multiset Argument Aggregation Constraint:
 - i. *Specification:*

$$\begin{aligned}
 & \left(\sum_{l \in \text{OP}_{\text{IO}}} \left(x_{\text{prog}}(i, l) \cdot \prod_{k=1}^{\text{regs}(l)} (\text{RLC}^\alpha(\text{IO}_{\text{prog}}(i, k, \cdot)) - \beta) \right) \right) \cdot (\text{acc}_{\text{prog}}(i) - \text{acc}_{\text{prog}}(i-1)) = \\
 & = (\text{acc}_{\text{prog}}(i) - \text{acc}_{\text{prog}}(i-1)) \cdot (1 - \text{io}(i)) + \\
 & + \sum_{m \in \text{OP}_{\text{IO}}} x_{\text{prog}}(i, m) \cdot \left(\sum_{l=1}^{\text{regs}(m)} \prod_{k=1, k \neq l}^{\text{regs}(m)} (\text{RLC}^\alpha(\text{IO}_{\text{prog}}(i, k, \cdot)) - \beta) \right)
 \end{aligned}$$

- ii. *Description:* This ensures that, at each row i , if the active opcode is an element of OP_{IO} and thus requires memory input, then all the non-zero memory records in $\text{IO}_{\text{prog}}(i, \cdot, \cdot)$ are added to the accumulator. Note that exactly the first $\text{regs}(\text{op}) \leq \text{M}_{\text{IO}}$ memory records are non-zero per opcode $\text{op} \in \text{OP}_{\text{IO}}$, bounded above by memory arity.

2. Terminal Constraints

- (a) The final output $\text{requests} \in \mathbb{K}$ of the acc_{prog} columns, constrained as $\text{requests} = \text{acc}_{\text{prog}}(\mathbf{h}_1)$.

3.1.2 Memory Table IO Semantics

Given the generation of the accumulated memory records, the memory table will verify that they are indeed correct. Note that in the previous subsection, the memory records can be adversarially chosen, meaning that the prover can input malicious code to alter the results of the execution trace. It thus becomes the function of the IO_{mem} columns in the memory table to ensure that this does not happen. The approach is modular to the underlying verification process of the memory table. We begin by seeding the memory table with the terminal output of the compute table and proceeding similarly.

$\text{RAP}_{\text{mem-IO}}$ Initial Inputs & Constraints

1. The Verifier randomness $\alpha, \beta \in \mathbb{K}$ obtained from the computation table's $\mathbf{w}_{\text{prog-IO}}$ array

2. The initial input **responses** $\in \mathbb{K}$, constrained as:

$$\text{ver}(1) \cdot (\text{RLC}^\alpha(x_{\text{mem}}(1, \cdot)) - \beta) \cdot (\text{responses} - \text{acc}_{\text{mem}}(1)) = (\text{responses} - \text{acc}_{\text{mem}}(1)) \cdot (1 - \text{ver}(1)) + \text{ver}(1)$$

RAP_{mem-IO} Table Construction

1. Prover engages with the Verifier as per RAP_{mem} to build $(x_{\text{mem}}, T_{\text{mem}})$ where $\text{IO}_{\text{mem}} \subseteq x_{\text{mem}}$
2. Prover builds one column **ver** $\in \{0, 1\}^{\mathbf{h}_2} \subseteq \mathbb{F}^{\mathbf{h}_2}$, denoting whether a memory access is verified
3. Prover builds **e** columns encoding the accumulating multiset as an extension field element, denoted as $\text{acc}_{\text{mem}} \in \mathbb{K}^{\mathbf{h}_2} \subseteq \mathbb{F}^{\mathbf{h}_2 \times \mathbf{e}}$, in a fixed basis representation of $\mathbb{K}$ over $\mathbb{F}$

The union of the above is defined as the *memory table IO trace* $T_{\text{IO}}^{\text{mem}} := (\text{ver}, \text{acc}_{\text{mem}}) \in \mathbb{F}^{\mathbf{h}_2 \times (1+\mathbf{e})}$ and is parametrized by the random extension field elements α, β and initial input **responses**.

RAP_{mem-IO} Constraints

1. Transition Constraints

(a) Opcode Booleanity:

- i. *Specification*: $\text{ver}(i) \cdot (\text{ver}(i) - 1) = 0$
- ii. *Description*: For every row i , **ver**(i) checks that a memory access is verified

(b) Accumulation Step

- i. Log-Derivative Multiset Argument Aggregation:

A. *Specification*:

$$\begin{aligned} \text{ver}(i) \cdot (\text{RLC}^\alpha(x_{\text{mem}}(i, \cdot)) - \beta) \cdot (\text{acc}_{\text{mem}}(i - 1) - \text{acc}_{\text{mem}}(i)) = \\ (\text{acc}_{\text{mem}}(i - 1) - \text{acc}_{\text{mem}}(i)) \cdot (1 - \text{ver}(i)) + \text{ver}(i) \end{aligned}$$

B. *Description*: At each row i , if the memory record requires memory verification, it is removed from the accumulator as per the log-derivative multiset argument.

(c) Terminal Constraints

- i. The final output $\text{out}_{\text{mem}} \in \mathbb{K}$ of the acc_{mem} columns, constrained as $\text{out}_{\text{mem}} = \text{acc}_{\text{mem}}(\mathbf{h}_2)$.

3.1.3 IO Consistency Checks

Given the above constructions, we provide the sufficient consistency checks for $\text{IO} \in \mathcal{L}_{\text{mIO}}$ that ensure all memory records are accounted for. This involves a set of equality constraints between initial and terminal inputs of the two tables. Note, however, that none of the *internal* columns of the IO traces are actually required; it is solely a function of their output values. Given the terminal output out_{prog} and randomness α, β of the computation table alongside the terminal output out_{mem} of the memory table, the Verifier performs the following equality checks:

1. **requests** = **responses**
2. $\text{out}_{\text{mem}} = 0$

In practice the above reduce to only checking that the terminal output of the memory table is zero, as the **requests** element has been retrieved and can be used as initial inputs to the memory table ‘as-is’.

We can thus utilize the construction above as a generic extension for any uniquely memory-execution separable language that is opcode and memory one-hot. Indeed, these are the only assumptions needed on the RAP to ensure that the IO is consistent. There exists one final ingredient: since there are no multiplicities involved in the basic construction of the accumulator, we will have to assume that all memory record requests are *unique*. We note that although this is limiting, as in practice memory addresses are looked up multiple times, there exists a similarly efficient solution utilizing a multiplicity column for each table and which does not require any additional rounds of interaction. We provide the formal results below.

Definition 3.5. A uniquely memory-execution separable language recognized by LangVM over memory encoding MEM which is opcode and memory one-hot is a unique request language if $\forall x \in \text{LangVM}$:

$$(\text{io}(i_1) = 1) \cap (\text{io}(i_2) = 1) \Rightarrow \{(((i_1, j_1) \neq (i_2, j_2)) \cap (j_{1,2} \in [\text{regs}(i_{1,2})]) \Rightarrow \text{IO}_{\text{prog}}(i_1, j_1) \neq \text{IO}_{\text{prog}}(i_2, j_2))\};$$

moreover it is considered a unique response language if $\forall x \in \text{LangVM}$:

$$(i \neq j) \Rightarrow (\text{IO}_{\text{mem}}(i, \cdot) \neq \text{IO}_{\text{mem}}(j, \cdot)).$$

Corollary 3.1. If a language is unique request, then it's also unique response.

Theorem 3.2. For all unique request languages $\mathcal{L}_{\text{Lang}}$, there exist $\text{RAP}_{\text{prog-IO}}$, $\text{RAP}_{\text{mem-IO}}$ such that:

$$x \in \mathcal{L}_{\text{Lang}} \Leftrightarrow \begin{array}{l} (x_{\text{prog}}, T_{\text{prog}}, T_{\text{IO}}^{\text{prog}}) \text{ satisfies } \text{RAP}_{\text{prog-IO}} \\ (x_{\text{mem}}, T_{\text{mem}}, T_{\text{IO}}^{\text{mem}}) \text{ satisfies } \text{RAP}_{\text{mem-IO}} \end{array}$$

Moreover, the concrete costs of this protocol are as follows:

1. Table Building

- (a) The table building costs for RAP_{mem} , RAP_{prog}
- (b) $\text{RAP}_{\text{prog-IO}}$ Additional Costs
 - i. One round of interaction, providing two random elements in $\mathbb{K}$
 - ii. One commitment to $\text{mem_ops}(\text{IO}_{\text{prog}})$ non-zero elements in $\mathbb{F}$
 - iii. $\mathbf{e}$ columns
- (c) $\text{RAP}_{\text{mem-IO}}$ Additional Costs
 - i. $\mathbf{e} + 1$ columns

2. Transition Constraints

- (a) Transition constraint gates for RAP_{mem} , RAP_{prog}
- (b) $\text{RAP}_{\text{prog-IO}}$ Additional Costs
 - i. $\mathbf{e}$ gates of degree $1 + M_{\text{IO}}$
- (c) $\text{RAP}_{\text{mem-IO}}$ Additional Costs
 - i. One gate of degree 1
 - ii. $\mathbf{e}$ gates of degree 3

3. Initial & Terminal Constraints

- (a) Initial & terminal constraints for RAP_{mem} , RAP_{prog}
- (b) $\text{RAP}_{\text{prog-IO}}$ Additional Costs
 - i. One equality constraint
- (c) $\text{RAP}_{\text{mem-IO}}$ Additional Costs
 - i. $\mathbf{e}$ gates of degree 3
 - ii. Two equality constraints

3.2 Stateful Verifiable Traversals

In general, programming language execution traces tend to be representable as Abstract Syntax Trees (ASTs). In an AST, nodes are labeled with the opcode that is being resolved, while its children are the inputs to it that are either (1) data objects, which are denoted by leaves, or (2) other opcodes, which are subtrees that follow the same structure. Depending on the complexity and semantics of the language, an AST for its execution can have multiple children, of which the maximal number is called the *opcode arity*. This intuition about AST representations leads to a key insight when designing the compute table of a general-purpose VM: we can consider the computation table as performing a *stateful* traversal down some tree representation of the given program. Here, each node not only has a label,

denoting the program data, but also a *state*, denoting the opcode being resolved. This is useful, as it provides a specific framework in which to (1) verify transitions between opcodes, and (2) construct the internal stack of the execution in an efficient way. We thus opt to define a generic ‘stateful tree traversal’ primitive, which we show is capable of performing most of the verification checks for the execution traces of a large family of programming languages.

We begin by reinforcing the notion of a proper n -ary tree to a recursively verifiable stateful variant, with state universe $\mathcal{S}$ and data universe $\mathcal{DT}$, which together define the label of the node $\mathcal{LT} := \mathcal{S} \times \mathcal{DT}$. Similar to the construction of n -ary trees, we use a functional $\text{child_label} : [n] \times (\mathcal{L}_{\text{full_T},n} \times \mathcal{LT}) \rightarrow \mathcal{LT}$ that provides the label of the i -th child of some n -ary labeled tree node.

Definition 3.6. A stateful labeled proper n -ary tree with labeling space $\mathcal{LT} := \mathcal{S} \times \mathcal{DT}$ is an element of the following language:

$$\mathcal{L}_{\text{SLT}}^n = \left\{ \text{slt} = (t, l) \mid \begin{array}{l} t \in \mathcal{L}_{\text{full_T},n}, l \in \mathcal{LT} \\ (\cap_{i=1}^n (\text{child}_i(\text{slt}) \in \mathcal{L}_{\text{SLT}}^n)) \cup (\text{size}(t) = 1) \end{array} \right\}$$

A recursively verifiable stateful labeled proper n -ary tree over labeling language $\mathcal{L}(F, n) =: \{(x, x_{\text{out}}) \in (\mathcal{LT}, (\mathcal{LT} \cup \{\perp\})^n) \mid \exists w \text{ s.t. } \forall_F((x, x_{\text{out}}), w) = 0\} \cup \{(\perp)^{n+1}\}$ is an element of the following language:

$$\mathcal{L}_{\text{RVT}}^{F,n} = \left\{ \text{rvt} = (t, l) \in \mathcal{L}_{\text{SLT}}^n \mid \begin{array}{l} ((l, (\text{child_label}(i, \text{rvt}))_{i=1}^n) \in \mathcal{L}(F, n)) \cup (\text{size}(t) = 1) \\ (\cap_{i=1}^n (\text{child}_i(\text{rvt}) \in \mathcal{L}_{\text{RVT}}^{F,n}) \cup (\text{size}(t) = 1) \\ (\text{size}(t) = 1) \Leftrightarrow (l, \{\perp\}^n) \in \mathcal{L}(F, n) \end{array} \right\}.$$

Remark 3.3. The language $\mathcal{L}(F, n)$ is, in our context, more properly considered an algebraically constrainable language on statements in $\mathcal{LT} \times (\mathcal{LT} \cup \{\perp\})^n$, in which the first element can be understood as an ‘input’ and the second element n ‘outputs’, all from the universe $\mathcal{LT}$. The inclusion of $\perp$ in the outputs refers to leaf nodes, which will have a label and no children but still need to satisfy $(\text{leaf_label}, \{\perp\}^n) \in \mathcal{L}(F, n)$.

Example 3.4. (Fibonacci) Let $f : \mathbb{N} \rightarrow \mathbb{N}$ be the Fibonacci function defined by the recursive algorithm $f(1) = 1$, $f(2) = 1$, and $f(n) = f(n-1) + f(n-2)$ for $n \geq 3$, and define $\mathcal{S} = \{\text{decompose}, \text{halt}\}$, $\mathcal{G} = \{(n, f(n)) : n \in \mathbb{N}\}$, so $\mathcal{G}$ is the graph of f over $\mathbb{N}$. For $n \in \mathbb{N}$, define a stateful labeled proper binary tree $\text{slt}(n)$ with labeling space $\mathcal{LT}_f = \mathcal{S} \times \mathcal{G}$ by: if $n = 1, 2$ the node has label $(\text{halt}, (n, f(n)))$ and no child nodes; otherwise the node has label $(\text{decompose}, (n, f(n)))$ and left and right children $\text{slt}(n-1)$ and $\text{slt}(n-2)$. This defines a language $\mathcal{L}_{\text{Fib}} \subset \mathcal{L}_{\text{SLT}}^2$, which by inspection can be seen to iteratively decompose a Fibonacci sum in a binary tree structure until it halts when $n = 1, 2$.

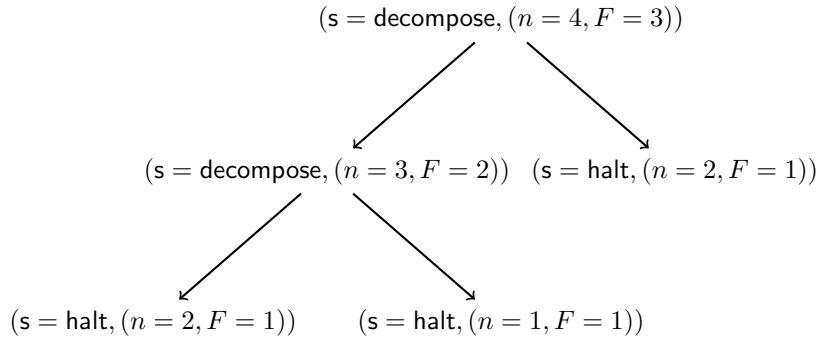


Figure 1: An example graph element $s \in \mathcal{L}_{\text{RVT}}^{G,2} \equiv \mathcal{L}_{\text{Fib}}$.

We can also define a recursively verifiable tree $\mathcal{L}_{\text{RVT}}^{G,2}$ with $\mathcal{LT} = \mathcal{S} \times \mathbb{N}^2$ which operates with $(x, x_{\text{out}}) = (\text{parent}, (\text{lc}, \text{rc}))$ where:

$$\text{parent} = (s, (n, F)), \quad \text{lc} = (s_L, (n_L, F_L)) \text{ or } \perp, \quad \text{rc} = (s_R, (n_R, F_R)) \text{ or } \perp,$$

according to a verifier V_G for labeling language $\mathcal{L}(G, 2)$ accepting $(\text{parent}, (\text{lc}, \text{rc}))$ iff:

1. $(\text{parent} = (\text{halt}, (1, 1)), \text{lc} = \perp, \text{rc} = \perp)$ or $(\text{parent} = (\text{halt}, (2, 1)), \text{lc} = \perp, \text{rc} = \perp)$,

2. (parent = (decompose, (3, 2)), lc = (halt, (2, 1)), rc = (halt, (1, 1))),
3. (parent = (decompose, (4, 3)), lc = (decompose, (3, 2)), rc = (halt, (2, 1))),
4. (s = s_L = s_R = decompose) ∩ (n ≥ 5) ∩ (n_L = n − 1) ∩ (n_R = n − 2) ∩ (F = F_L + F_R).

Despite the fact that $\mathcal{L}_{\text{SLT}}^2$ uses the Fibonacci function f to define every label as $(n, f(n))$, and $\mathcal{L}_{\text{RVT}}^{G,2}$ only enforces the a priori weaker constraint $F = F_L + F_R$ between a parent and its children, $\mathcal{L}_{\text{Fib}} \equiv \mathcal{L}_{\text{RVT}}^{G,2}$ over the language $\mathcal{L}(G, 2)$. We argue for this below, but hasten to point out that this is analogous to what happens in the **NockVM**, which is why we draw attention to this simpler example.

The reason for the above, briefly, is as follows: according to the definitions of $\mathcal{L}_{\text{RVT}}^{G,2}$ and $\mathcal{L}(G, 2)$, leaf nodes are exactly (halt, 2, 1) and (halt, 1, 1), which correspond to the terminal states $f(2) = 1$ and $f(1) = 1$ of recursively computing the Fibonacci function. Given this and the definition of $\mathcal{L}(G, 2)$, the only tree with two leaves in $\mathcal{L}_{\text{Fib}}$ has parent = (decompose, 3, 2), lc = (halt, 2, 1), rc = (halt, 1, 1), which corresponds to the first non-terminal case of the Fibonacci function $f(3) = f(2) + f(1) = 1 + 1 = 2$. An inductive argument from these base cases and the definition of $\mathcal{L}(G, 2)$ – specifically the constraints $n_L = n - 1, n_R = n - 2, F = F_L + F_R$ – shows that $\mathcal{L}_{\text{Fib}} \equiv \mathcal{L}_{\text{RVT}}^{G,2}$.

Definition 3.7. The unique memory-execution separable language $\mathcal{L}_{\text{Lang}} := \{x | \exists T \text{ s.t. } \text{LangVM}(x, T) = 0\}$ recognized by **LangVM** is execution traversable over F if and only if there exists a bijection $\phi_{\text{comp}} : \mathcal{L}_{\text{Lang}}^{\text{comp}} \rightarrow \prod_{i=1}^{k_{\text{comp}}} \mathcal{L}_{\text{RVT}}^{F_i, n}$, where:

$$\mathcal{L}_{\text{Lang}}^{\text{comp}} = \{x_{\text{prog}} | \exists T_{\text{prog}}, \text{IO}_{\text{prog}} \text{ s.t. } \text{LangVMComputation}(x_{\text{prog}}, T_{\text{prog}}, \text{IO}_{\text{prog}}) = 0\}.$$

Moreover, a language $\mathcal{L}_{\text{Lang}}$ is considered memory traversable over F^* if and only if there exists a bijection $\phi_{\text{mem}} : \mathcal{L}_{\text{Lang}}^{\text{mem}} \rightarrow \prod_{i=1}^{k_{\text{mem}}} \mathcal{L}_{\text{RVT}}^{F_i^*, n}$, where:

$$\mathcal{L}_{\text{Lang}}^{\text{mem}} = \{x_{\text{mem}} | \exists T_{\text{mem}}, \text{IO}_{\text{mem}} \text{ s.t. } \text{LangVMMemory}(x_{\text{mem}}, T_{\text{mem}}, \text{IO}_{\text{mem}}) = 0\}.$$

A language is (F, F^*) -traversable if it is execution traversable over F and memory traversable over F^* .

For traversable languages, it suffices to construct a verifiable RAP instance for language $\mathcal{L}_{\text{RVT}}^{F, n}$, as it would be isomorphic to proving that the underlying execution and memory traces are indeed valid.

3.2.1 Verifiable Walks over n -ary Trees

We specify an efficient RAP for verifying $\mathcal{L}_{\text{RVT}}^F$ when F is state-data separable and solely state traversable. The main idea is to traverse the tree structure in a prespecified ordering, checking that F is satisfied for all nodes, and terminating at the leaves. We assume that F is algebraically constrainable, in that there exist s_F constraints $C_F = \{C_{F,1}, \dots, C_{F,s_F}\}$ that suitably constrain the verification of an instance of F . For an n -ary tree over labels in a universe $\mathcal{LT}$, we also assume that every (sub)tree node can be (succinctly) represented with `node_size` elements from $\mathbb{F}$. We make a final restriction to the definition: that states can be decomposed into two types, ‘leaf’ and ‘non-leaf’ states.

Definition 3.8. Let $\mathcal{L}(F, n)$ be as in Definition 3.6. Suppose $\exists \mathcal{S}_{\text{leaf}} \subseteq \mathcal{S}$, s.t. $\forall \text{rvt} := (t, (s, d)) \in \mathcal{L}_{\text{RVT}}^{F, n}$,

$$s \in \mathcal{S}_{\text{leaf}} \Leftrightarrow \text{type}(\text{rvt}) = \text{leaf}.$$

Then $\mathcal{L}(F, n)$ is a (solely) state-traversable language, where states in $\mathcal{S}_{\text{leaf}}$ are terminal states.

Moreover, if for all $s \in \mathcal{S} \setminus \mathcal{S}_{\text{leaf}}$ there exists $\text{fanout}(s) \in [n]$ such that

$$(x, x_{\text{out}}) \in \mathcal{L}(F, n) \text{ where } x := (s, d) \Rightarrow x_{\text{out}} = \{(s_i, d_i)\}_{i=1}^{\text{fanout}(s)} \times \{(\perp, \perp)\}^{n - \text{fanout}(s)}$$

where $(s_i, d_i) \neq (\perp, \perp)$, then $\mathcal{L}(F, n)$ is a state-defined dynamic labeling language.

Remark 3.5. Note that in Example 3.4, we have that $\mathcal{S}_{\text{leaf}} = \{\text{halt}\}$ and $\forall s \in \mathcal{S} \setminus \mathcal{S}_{\text{leaf}} = \{\text{decompose}\} \Rightarrow \text{fanout}(s) = 2$. It thus satisfies the above definition and so $\mathcal{L}(G, 2)$ is a solely state-traversable and state-defined dynamic labeling language verifying the correct decomposition of any Fibonacci number.

It will be convenient to have the function `fanout` available to us as notation. As can be gleaned from the previous definition, $\text{fanout}(s)$ is the number of non-null nodes that are children of nodes with state s . If $s \in \mathcal{S}_{\text{leaf}}$, then we can define $\text{fanout}(s) = 0$. We also interpret `fanout` as a function on $[n]$ via pullback by the labeling function $[n] \rightarrow \mathcal{S}$ of the states. Said differently, for the state $s_j \in \mathcal{S}$ represented by the j -th $\text{st}(\cdot, j)$ column, we write $\text{fanout}(j)$ as synonymous with $\text{fanout}(s_j)$. This is a technical yet important point: we can model dynamic branching in this way in a stateful manner: this is captured by the definition of a state-defined dynamic labeling function where the state is enough to decide how many non-null children exist and thus need to be traversed during the walk. Moreover, we can assign to each state the set of indices of the child states that are non-null. These will be required as they need to be added to the traversal itself. More specifically, we have the mapping $\text{indices} : j \mapsto [\text{fanout}(j)] \subseteq [n]$ of which children are non-null states.

The reason we restrict to working with an F that is solely state traversable and state-defined dynamic is in part due to the efficiency gains from encoding state values as selector columns. Indeed, if we know the opcode that is being resolved at some level of an AST, which is encoded as a state, with the right AST encoding we can be certain whether and how many deeper branches for the AST exist at that level as a function of the opcode. These are exactly the ideas of solely state traversable and state-defined dynamic labeling functions. We provide a high-level overview of RAP_{walk} for checking inclusion in $\mathcal{L}_{\text{RVT}}^F$ if F satisfies Definition 3.8.

1. $\mathbf{h}$ different rows, each corresponding to a unique node in the tree
2. One column representing the current node being verified $\text{current_node} \in \mathcal{LT}^{\mathbf{h}} \subseteq \mathbb{F}^{\mathbf{h} \times \text{node_size}}$
3. n columns `children` representing the n children of `current\_node` in left-to-right order, with $\text{children} \in \mathcal{LT}^{\mathbf{h} \times n} \subseteq \mathbb{F}^{\mathbf{h} \times n \times \text{node_size}}$
4. $\mathbf{T}_F$ columns, to verify the evaluation of F
5. A line number column $\text{ln} \in \mathbb{K}^{\mathbf{h}} \subseteq \mathbb{F}^{\mathbf{h} \times \mathbf{e}}$, with the number of nodes resolved until now
6. A count column $\text{count} \in \mathbb{K}^{\mathbf{h}} \subseteq \mathbb{F}^{\mathbf{h} \times \mathbf{e}}$, with the total number of non-leaf nodes observed until now
7. A set of $|\mathcal{S}|$ boolean columns $\text{st} \in \{0, 1\}^{\mathbf{h} \times |\mathcal{S}|} \subseteq \mathbb{F}^{\mathbf{h} \times |\mathcal{S}|}$, designating the state (as an element of $\mathcal{S}$) of each node using a one-hot encoding
8. An accumulator column $\text{acc}_{\text{walk}} \in \mathbb{K} \subseteq \mathbb{F}^{\mathbf{h} \times \mathbf{e}}$

RAP_{walk} Table Construction

1. Prover performs Breadth-First Search traversal on `rvt.root` to compute the ordered sequence of nodes $\{\text{walk}_i\}_{i=1}^{\mathbf{h}}$ to be verified, where $\mathbf{h}$ corresponds to the total nodes including the root
2. Prover builds the columns `current\_node`, `st` in the exact order of $\{\text{walk}_i\}_{i=1}^{\mathbf{h}}$ and sends a commitment $\text{Com}(\{(i_j, \text{walk}_{i_j}) | \text{state}(\text{walk}_{i_j}) \in \mathcal{S}_{\text{leaf}}\}_{j=1}^{\mathbf{h}_{\text{leaf}}})$ to the Verifier, where $i_j \in [\mathbf{h}]$ the row number of the j -th walk_{i_j} and $\mathbf{h}_{\text{leaf}}$ total nodes with terminal states
3. Verifier responds with random elements $\alpha, \gamma \in \mathbb{K}$
4. Prover builds columns `ln`, `count`
5. Prover builds $\mathbf{e}$ columns encoding the accumulating multiset as an extension field element, denoted as $\text{acc}_{\text{walk}} \in \mathbb{K}^{\mathbf{h}} \subseteq \mathbb{F}^{\mathbf{h} \times \mathbf{e}}$, in a fixed basis representation of $\mathbb{K}$ over $\mathbb{F}$

RAP_{walk} Initial Inputs & Constraints

1. The initial inputs `seed`, `initial\_count` $\in \mathbb{K}$, constrained as:

$$\begin{aligned} & \text{ln}(1) \cdot \text{RLC}^\gamma(\text{current_node}(1, \cdot)) + \text{acc}_{\text{walk}}(1) = \\ & \sum_{s_j \in \mathcal{S} \setminus \mathcal{S}_{\text{leaf}}} \left(\text{st}(1, j) \cdot \left(\sum_{k=1}^{\text{fanout}(j)} \alpha^k \cdot \text{initial_count} \cdot \text{RLC}^\gamma(\text{children}(1, k, \cdot)) \right) \right) + \text{seed} \end{aligned}$$

2. $\text{count}(1) = \text{initial_count} \cdot \left(\sum_{j=1}^{|\mathcal{S}|} \text{st}(1, j) \cdot \alpha^{\text{fanout}(j)} \right)$
3. The initial line constraint $\ln(1) = \alpha$

RAP_{walk} Constraints

1. Transition Constraints

(a) State Booleanity

- i. *Specification:* $\sum_{j=1}^{|\mathcal{S}|} \text{st}(i, j) = 1, \sum_{j=1}^{|\mathcal{S}|} \text{st}(i, j) \cdot (1 - \text{st}(i, j)) = 0$
- ii. *Description:* For every row i , only one state is active

(b) F Constraints

- i. *Specification:* $\mathbf{C}_F = \{\mathbf{C}_{F,1}, \dots, \mathbf{C}_{F,s_F}\}$
- ii. *Description:* For every row i , s_F constraints checking that children and F are correctly related. Note these will use the selector st and F witness columns $\mathbf{T}_F$.

(c) Line Number Increment:

- i. *Specification:* $\ln(i) = \alpha \cdot \ln(i - 1)$
- ii. *Description:* For every row i , we iteratively constrain using supplied randomness α to get $\ln(i) = \alpha^i$ as an index with which to store walk elements in the aggregator

(d) Non-Terminal Node Count Increment:

- i. *Specification:* $\text{count}(i) = \text{count}(i - 1) \cdot \left(\sum_{j=1}^{|\mathcal{S}|} \text{st}(i, j) \cdot \alpha^{\text{fanout}(j)} \right)$
- ii. *Description:* For every row i , multiply count by $\alpha^{\text{fanout}(j)}$, where $\text{fanout}(j)$ is the total number of nodes that need to be discovered if the j -th column of st is active (i.e. state $s_j \in \mathcal{S}$). This is equivalent to keeping track of how many new nodes need to be traversed due to resolving current_node . count increases only for non-terminal states.

(e) Accumulation Step

- i. Line-Indexed Multiset Argument Aggregation:

A. *Specification:*

$$\begin{aligned} & \ln(i) \cdot \text{RLC}^\gamma(\text{current_node}(i, \cdot)) + \text{acc}_{\text{walk}}(i) - \text{acc}_{\text{walk}}(i - 1) = \\ & \sum_{s_j \in \mathcal{S} \setminus \mathcal{S}_{\text{leaf}}} \left(\text{st}(i, j) \cdot \left(\sum_{k=1}^{\text{fanout}(j)} \alpha^k \cdot \text{count}(i - 1) \cdot \text{RLC}^\gamma(\text{children}(i, k, \cdot)) \right) \right) \end{aligned}$$

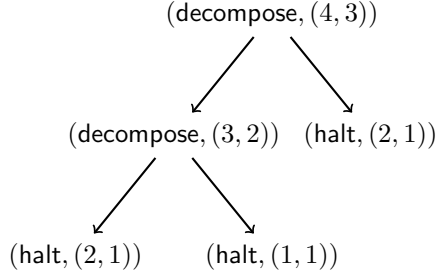
- B. *Description:* At each row i , the accumulator element indexed by the line number is removed and traversed as current_node . Simultaneously, all non-terminal children of current_node are added to the accumulator, indexed by increasing products over the current count. This gives them a unique line number at which they will be resolved in the future. For terminal states, the accumulator does not add any elements.

(f) Terminal Constraints

- i. The final output $\text{out}_{\text{walk}} \in \mathbb{K}$ of the acc_{walk} columns, constrained as $\text{out}_{\text{walk}} = \text{acc}_{\text{walk}}(\mathbf{h})$.

3.2.2 Consistency Checks on Verifiable n -ary Walks

The above construction of RAP_{walk} can be made to recognize whether some n -ary tree and label pair $\text{rvt}_{\text{root}} = (t_{\text{root}}, l_{\text{root}})$ is an element of $\mathcal{L}_{\text{RVT}}^{F,n}$ by using it as a seed in the initial input. If rvt_{root} can be represented in $\mathbb{F}^{\text{node_size}}$, by setting $\text{seed} = \alpha \cdot \text{RLC}^\gamma(\text{rvt}_{\text{root}})$ and $\text{initial_count} = \alpha$, it suffices to check that the final accumulator output $\text{out}_{\text{walk}} = 0$. The fundamental parameters defining the efficiency of building RAP_{walk} are the number of internal, leaf and total nodes of the input element rvt , which we denote by $\text{internal_nodes}(\text{rvt})$, $\text{leaf_nodes}(\text{rvt})$, and $\text{total_nodes}(\text{rvt})$ respectively.



State	Label	Left Child	Right Child	ln	count	acc _{walk}
Seed Input	$\perp$	$\perp$	$\perp$	1	α	$\alpha \cdot \text{RLC}((\text{decompose}, (4, 3)))$
decompose	(4, 3)	(decompose, (3, 2))	(halt, (2, 1))	α	α^3	$\alpha^2 \cdot \text{RLC}((\text{decompose}, (3, 2))) + \alpha^3 \cdot \text{RLC}((\text{halt}, (2, 1)))$
decompose	(3, 2)	(halt, (2, 1))	(halt, (1, 1))	α^2	α^5	$\alpha^3 \cdot \text{RLC}((\text{halt}, (2, 1))) + \alpha^4 \cdot \text{RLC}((\text{halt}, (2, 1))) + \alpha^5 \cdot \text{RLC}((\text{halt}, (1, 1)))$
halt	(2, 1)	$\perp$	$\perp$	α^3	α^5	$\alpha^4 \cdot \text{RLC}((\text{halt}, (2, 1))) + \alpha^5 \cdot \text{RLC}((\text{halt}, (1, 1)))$
halt	(2, 1)	$\perp$	$\perp$	α^4	α^5	$\alpha^5 \cdot \text{RLC}((\text{halt}, (1, 1)))$
halt	(1, 1)	$\perp$	$\perp$	α^5	α^5	0

Figure 2: (Above) An example graph for $s \in \mathcal{L}_{\text{RVT}}^{G,2}$. (Below) RAP_{walk} table verifying $s \in \mathcal{L}_{\text{RVT}}^{G,2}$.

Theorem 3.6. *If $\text{slt} \in \mathcal{L}_{\text{SLT}}^n$, F is such that $\mathcal{L}(F, n)$ is algebraically constrainable and satisfies Definition 3.8 and $\mathcal{LT} \subseteq \mathbb{F}^{\text{node_size}}$, then there exists RAP_{walk} with perfect completeness:*

$$\text{slt} \in \mathcal{L}_{\text{RVT}}^{F,n} \Rightarrow \exists \mathbf{T}_{\text{walk}} \text{ satisfying } \text{RAP}_{\text{walk}} \text{ with } (\text{seed}, \text{initial_count}) = (\text{RLC}^\gamma(\text{slt}) \cdot \alpha, \alpha), \text{out}_{\text{walk}} = 0,$$

where $(\alpha, \gamma) \in \mathbb{K}$ the Verifier responses. Moreover, the protocol's soundness error is bounded as:

$$\Pr \left[\text{slt} \notin \mathcal{L}_{\text{RVT}}^{F,n} \mid \exists \mathbf{T}_{\text{walk}} \text{ satisfying } \text{RAP}_{\text{walk}} \text{ with } (\text{seed}, \text{initial_count}) = (\text{RLC}^\gamma(\text{slt}) \cdot \alpha, \alpha), \text{out}_{\text{walk}} = 0 \right] \leq \frac{\mathbf{h}}{|\mathbb{K}|},$$

where $\mathbf{h}$ the total number of nodes in slt . Moreover, the concrete costs of this protocol are as follows:

1. Table Building

- (a) One BFS traversal of rvt
- (b) One commitment to at most $2 \cdot \text{internal_nodes}(\text{rvt})$ non-zero elements in $\mathbb{F}$
- (c) One round of interaction, providing three random elements in $\mathbb{K}$
- (d) $|\mathbf{T}_F| + |S| + (n+1) \cdot \text{node_size} + 3 \cdot \mathbf{e}$ columns

2. Transition Constraints

- (a) $2 + 2 \cdot \mathbf{e}$ gates of degree 1
- (b) s_F transition constraint gates for F
- (c) $\mathbf{e}$ gates of degree 3

3. Initial & Terminal Constraints

- (a) $2 \cdot \mathbf{e}$ gates of degree 1
- (b) $\mathbf{e}$ gates of degree 3
- (c) One equality check

4. The total number of rows in $\mathbf{T}_{\text{walk}}$ are equal to $\mathbf{h}$

3.3 Verifiable Walks over Multiple n -ary Trees

The previous design can be efficiently modified to provide for verifying a more general gadget, where traversal happens over an n -ary forest. The fundamental difference here is that, instead of dealing with only one root, there is a set of k different labeled n -ary trees. The language that is being verified here is just $\prod_{i=1}^k \mathcal{L}_{\text{RVT}}^{F,n}$, with the only difference being the initial inputs, encoded as initial constraints. For $\{\text{rvt}_i\}_{i=1}^k$, the gadget above verifies inclusion in $\prod_{i=1}^k \mathcal{L}_{\text{RVT}}^{F,n}$ with the exact same efficiency, apart from a multiplicative change in the total number of rows to equal the total number of (potentially equal) nodes in all the trees in aggregate.

Corollary 3.7. *If $(\text{slt})_{i=1}^k \in \prod_{i=1}^k \mathcal{L}_{\text{SLT}}^n$, F is such that $\mathcal{L}(F, n)$ is algebraically constrainable and satisfies Definition 3.8 and $\mathcal{LT} \subseteq \mathbb{F}^{\text{node_size}}$, then there exists RAP_{walk} such that:*

$$(\text{slt}_i)_{i=1}^k \in \prod_{i=1}^k \mathcal{L}_{\text{RVT}}^{F,n} \Rightarrow \exists T_{\text{walk}} \text{ that satisfies } \text{RAP}_{\text{walk}} \text{ with } (\text{seed}, \text{initial_count}) = \left(\sum_{i=1}^k \text{RLC}^\gamma(\text{slt}_i) \cdot \alpha^i, \alpha^k \right),$$

where $(\alpha, \gamma) \in \mathbb{K}$ are the Verifier responses. The soundness error of this protocol is bounded above by:

$$\Pr \left[(\text{slt}_i)_{i=1}^k \notin \prod_{i=1}^k \mathcal{L}_{\text{RVT}}^{F,n} \mid \exists T_{\text{walk}} \text{ satisfying } \text{RAP}_{\text{walk}} \text{ with } (\text{seed}, \text{initial_count}) = \left(\sum_{i=1}^k \text{RLC}^\gamma(\text{slt}_i) \cdot \alpha^i, \alpha^k \right) \right] \leq \frac{\mathbf{h}}{|\mathbb{K}|},$$

where $\mathbf{h}$ is the total number of nodes in $(\text{slt}_i)_{i=1}^k$. Moreover, the concrete costs of this protocol are as in Theorem 3.6, replacing all instances of $\text{internal_nodes}(\text{slt})$ with $\sum_{i=1}^k \text{internal_nodes}(\text{slt}_i)$.

The above is immediate from the observation that the design is fully local in the walk resolution. Indeed, the `seed` variable can be thought of as a ‘preloading’ of the first k elements that need to be walked, indexed by their line numbers, while the `initial_count` variable gives the line number at which elements are added to the stack. By preloading k different roots - that may or may not be of disjoint trees - we can walk them in parallel without requiring any changes to the constraint system itself. This is because every transition is locally updated and does not require any global inputs, guaranteeing that we can parallelize the walk to multiple trees. This also provides intuition as to why the only change in efficiency is over the total number of nodes that are traversed: we are walking k trees in a deterministic ordering, each of which requires the resolution of all its nodes.

The concern here comes in the inefficiency inherent to walking trees that have overlapping subtrees. This is of particular concern in our context, as will be further explained in the following sections. The above corollary will walk all subtrees, thus making it substantially less efficient in the case where there are multiple roots that ‘feed into’ the same subtree. We thus provide a modification that allows for walking the minimum common set of nodes in a forest, without multiplicities, below.

4 Formalizing the Nock Programming Language

The purpose of this section is to provide a formal treatment of the Nock programming language. In its current form, the Nock specification is too ambiguous to be used ‘as is’ in the setting of formal verification. To this end, we revise the definition of the Nock interpreter and demonstrate that, with a slight strengthening of its semantic properties, we can arrive at a specification of the Nock Virtual Machine (NockVM) that is amenable to formal verification guarantees.

4.1 Executable Nouns

We begin by strengthening the underlying data structure that we will be operating over. In the original specification, the fundamental object has been the noun of length λ , consisting of (1) a dyck word of length $2 \cdot (\lambda - 1)$, and (2) λ leaf elements. This representation is equivalent to a unique proper (or full) binary tree whose leaves are labeled with elements from some field $\mathbb{F}$. These are used to store data, which can also be parsed as executable code. Indeed, any Nock program is represented as:

$$\text{program} := *[\text{subject} \quad \text{formula}].$$

The problem with representing **program** purely as a noun $\text{noun} := [\text{subject}, \text{formula}]$ is that we lose some fundamental information about its computation: namely, we are not aware that the above program needs to be run by executing the Nock opcode $*$ at the given level of the tree, i.e. **noun** is ambiguous in that it does not contain any information about the depth at which the Nock opcode needs to be executed - both $[\text{subject}, \text{formula}]$ and $[\text{subject}, * \text{formula}]$ have **noun** as the underlying representation of its memory but are completely different *programs*.

This means that, for a fully executable program, we need to also store the Nock operators that have yet to be executed on the trace. This can be done by noting that, in dyck word notation, each 0 in the $2(\lambda - 1)$ -bit dyck word represents the opening of a new subtree and can thus be used to index the operator array. More specifically, for a complete binary tree with λ field elements as leaves and associated dyck word w of $2 \cdot (\lambda - 1)$ bits, we can label each of its $\lambda - 1$ internal nodes using a $\lambda - 1$ -bit array that maps the i -th bit to the subtree given by the i -th 0 value in the dyck word representation. Note that all complete binary trees have $\lambda - 1$ internal nodes, equivalent to $\lambda - 1$ different 0s in their dyck representation.

Definition 4.1. A verb **verb** of length λ over alphabet $\mathcal{X} \cup \{\perp\}$, arithmetization $\text{map} : [\lambda] \times \mathcal{N}^\lambda(\mathbb{F}) \rightarrow \mathcal{N}^\lambda(\mathbb{F})$ and field $\mathbb{F}$ is a pair consisting of a noun $\text{noun} \in \mathcal{N}^\lambda(\mathbb{F})$ with λ leaves and a list of updates actions, which is a $(\lambda - 1)$ -length array of elements from $\mathcal{X} \cup \{\perp\}$:

$$\text{verb} := (\text{actions}, \text{noun}) \in (\mathcal{X} \cup \{\perp\})^{\lambda-1} \times \mathcal{N}^\lambda(\mathbb{F}) := \mathcal{V}^\lambda(\mathbb{F}, \mathcal{X}).$$

Moreover, **verb** is considered (fully) executed or in terminal state if its actions array consists only of the null alphabet element $\perp$.

The alphabet $\mathcal{X}$ above represents the set of operators over nouns, with the operator op_i in the actions array at index i representing execution of op_i on $\text{map}(i, \text{noun})$ or the i -th subtree of **noun** with respect to the arithmetization **map**. We can thus completely represent the state of each execution trace by using its syntax tree, as recorded in the actions array, and the data it operates over, as recorded in the noun element. Together, these two variables constitute a **verb**, or an ordered list of actions over a noun. Implicit above is the mapping of the $\lambda - 1$ indices in the actions variable to their subtrees in **noun**: this can be defined in any way by specifying **map**, which will prove useful later on.

4.2 Operators & Opcodes

We define the notion of an operator formally as, given some (ordered list of) noun(s) **input** as input, the program runs deterministic logic and either outputs a noun **output** or crashes, which we denote with a $\perp$. We make use of a helper function $\text{type} : \mathcal{N}^\lambda(\mathbb{F}) \rightarrow \{\text{atom}, \text{cell}\}$, which checks if a given noun is an atom or cell. Note that this functional is also inherited by the verb type in the natural way: by applying it to its corresponding noun (or second element in the tuple). The fundamental elements of Nock are operations on nouns, which are all defined as deterministic mappings from a (set of) noun(s) to a noun or the crash state. We formally specify them in Appendix A, Algorithm 2. Note that there is no notion of execution here: these operators work only on the nouns and do not interact with the actions, which will be used only when keeping track of opcodes (which are defined by operators).

When strengthening the specification, it will be useful to also define an ordering operator $- : \mathcal{N}^\lambda(\mathbb{F}) \times \mathcal{N}^\lambda(\mathbb{F}) \rightarrow \mathcal{N}^\lambda(\mathbb{F})$, which will have the same domain $\mathcal{N}^\lambda(\mathbb{F}) \times \mathcal{N}^\lambda(\mathbb{F})$ as the $=, /$ operators. This can be thought of as a lookup operator that conditions on its first input (which is always an atom) and executes one of two operators on single nouns on its second input. Note that this can be generalized to arbitrary operators over individual nouns (endomorphisms) but we will restrict to $+, ?$ which are enough for a Turing-Complete instruction set. We also note that $*$ is not technically considered an operator and has a unique function in the update process of the Nock language. We clarify its functionality when defining the notion of opcodes below.

4.2.1 Opcodes

The above operators (or alphabet) can be used to define opcodes through the definition of the behavior of $*$. An opcode can be thought of as a mapping from a (single) verb to another verb or to the crash $\perp$ state. We denote by $[n]$ the set of integers from 0 to $n - 1$ inclusive and define opcodes with respect to the aforementioned set of noun operations (as part of the alphabet) and the noun data

structure. The process by which these opcodes are updated is specified by the canonical Nock opcode $*$, which can be formally defined as a mapping $*$: $\mathcal{V}^\lambda(\mathbb{F}, \mathcal{X}) \rightarrow (\mathcal{V}^\lambda(\mathbb{F}, \mathcal{X}), \perp)$. Note that this is strictly different to the operators in that it is a mapping between *program states* as opposed to just nouns. We can further generalize the complexity of the specification by defining a verb alphabet $\{*_1, \dots, *_k\}$ of different mappings between program states. However, this is not necessary to fully specify Nock as it only has a single program transition function represented by $*$. Indeed, the opcodes can alternatively be understood as defining $*$: they represent the transformation rules for when $*$ is applied to an arbitrary noun and suffice to fully specify the language.

Definition 4.2 (Nock Transition Function). *The Nock opcodes $\mathcal{OP} = \{\text{OP}_0, \dots, \text{OP}_{10}\}$ are a set of deterministic mappings $\text{OP}_i : \mathcal{D}(\mathcal{V}^\lambda(\mathbb{F}, \mathcal{X})) \rightarrow \mathcal{V} \cup \{\perp\}$ to verbs (in $\mathcal{V}^\lambda(\mathbb{F}, \mathcal{X})$) from the computable opcode domain $\mathcal{D}(\mathcal{V}^\lambda(\mathbb{F}, \mathcal{X})) \subset \mathcal{V}^\lambda(\mathbb{F}, \mathcal{X})$ given by*

$$\mathcal{D}(\mathcal{V}^\lambda(\mathbb{F}, \mathcal{X})) := \{v := (a, n) \in \mathcal{V}^\lambda(\mathbb{F}, \mathcal{X}) \mid n := [n_1, [i, n_2]], (i \in [11]) \cup (\text{type}(i) = \text{cell}), a_1 = \langle * \rangle, \forall j \neq 1, a_j = \langle \perp \rangle\},$$

over the alphabet $\mathcal{X} := \langle =, /, \#, - \rangle$ and field $\mathbb{F}$, each defined for $\text{type}(i) = \text{atom}$ by its value $i \in [11]$ and parametrized by the Nock transition function $*$:

1. $\text{OP}_0 : v := *[n_1, [0, n_2]] \rightarrow /(n_2, n_1)$
2. $\text{OP}_1 : v := *[n_1, [1, n_2]] \rightarrow n_2$
3. $\text{OP}_2 : v := *[n_1, [2, n_2]] \rightarrow *[n_1, \text{head}(n_2)], *[n_1, \text{tail}(n_2)]$
4. $\text{OP}_3 : v := *[n_1, [3, n_2]] \rightarrow -[3, *[n_1, n_2]] = ?(*[n_1, n_2])$
5. $\text{OP}_4 : v := *[n_1, [4, n_2]] \rightarrow -[4, *[n_1, n_2]] = +(*[n_1, n_2])$
6. $\text{OP}_5 : v := *[n_1, [5, n_2]] \rightarrow =(*[n_1, \text{head}(n_2)], *[n_1, \text{tail}(n_2)])$
7. $\text{OP}_6 : v := *[n_1, [6, n_2]] \rightarrow *[n_1, *[\text{tail}(n_2), [0, *[[2, 3], [0, *[n_1, [4, [4, \text{head}(n_2)]]]]]]]$
8. $\text{OP}_7 : v := *[n_1, [7, n_2]] \rightarrow *[n_1, \text{head}(n_2)], \text{tail}(n_2)]$
9. $\text{OP}_8 : v := *[n_1, [8, n_2]] \rightarrow *[[*[n_1, \text{head}(n_2)], n_1], \text{tail}(n_2)]$
10. $\text{OP}_9 : v := *[n_1, [9, n_2]] \rightarrow *[n_1, \text{tail}(n_2)], 2, [[0, 1], 0, \text{head}(n_2)]]$
11. $\text{OP}_{10} : v := *[n_1, [10, n_2]] \rightarrow \#[\text{head}(\text{head}(n_2)), *[n_1, \text{tail}(\text{head}(n_2))], *[n_1, \text{tail}(n_2)]]$.

We also define the concatenation opcode $\text{cons} : \mathcal{D}(\mathcal{V}^\lambda(\mathbb{F}, \mathcal{X})) \rightarrow \mathcal{V}^\lambda(\mathbb{F}, \mathcal{X}) \cup \{\perp\}$ for $\text{type}(i) = \text{cell}$ as:

$$\text{cons} : v := *[n_1, [i, n_2]] \rightarrow *[n_1, i], *[n_1, n_2]].$$

By restricting the domain of the above mappings to $\mathcal{D}(\mathcal{V}^\lambda(\mathbb{F}, \mathcal{X}))$, we are ensuring their computability without requiring any recursive calls to $*$. This makes it extremely easy to formally verify the transition without having to take into account (potentially unbounded) recursion at the opcode level. More specifically, $\mathcal{D}(\mathcal{V}^\lambda(\mathbb{F}, \mathcal{X}))$ can be understood as requiring that, in order for an opcode to be successfully computable as the next update to the transcript, the following should hold:

1. Input verb v of the form $v := *[n_1, [i, n_2]]$
2. $n_1, [i, n_2]$ are in terminal state
3. Noun i is an atom for a valid opcode (i.e. $i \in [11]$) or a cell

In the case that the input does not satisfy the above, the transition function computing it will crash and output the null state $\perp$. We note that the program will initially check whether it needs to run the concatenation operator by looking at the value of i : if $\text{type}(i) = \text{cell}$, it will apply the concatenation opcode mapping. Note that the computable opcode domain is a key restriction, as it can be partitioned into disjoint subsets that each uniquely associates with a given opcode. This happens because each opcode should only be executed if it has been called, and this partitioning allows the interpreter to know which opcode is next without ambiguity.

4.3 Nock Program & Transition Function

We now use the above to define the notion of an Nock program and its successfully verifying transcript. The program will be processing the above opcodes as verbs so that any sequence of them can be executed over arbitrary data stored in the verb's noun. This will be done in a locally updatable manner so that any partially completed transcript can be deterministically extended to its unique complete transcript through iterated local transitions or opcode processing steps.

Definition 4.3 (Nock Program). *A Nock program $F \in \mathcal{V}^\lambda(\mathbb{F}, \mathcal{X})$ of length λ is defined as a verb over nouns of size λ over field $\mathbb{F}$ and operator alphabet $\mathcal{X} := \langle ?, =, /, \#, - \rangle$ iff the following properties hold:*

1. $F = *[\text{subject}, \text{formula}]$, where $\text{subject}, \text{formula} \in \mathcal{N}^\lambda(\mathbb{F})$, and
2. *subject and formula are in terminal state.*

Furthermore, we define the language **NockProgram** as:

$$\text{NockProgram}^\Lambda := \{F \in \mathcal{V}^\lambda(\mathbb{F}, \mathcal{X}) \mid F \text{ is an Nock program of length } \lambda \leq \Lambda\}.$$

Note the above implies that all programs are just verbs in a specific state: namely they are subject-formula pairs with no unresolved recursive dependencies that are provided to the canonical operation for the Nock virtual machine, denoted by $*$. Applying $*$ to any program allows for a deterministic update based on the opcode mappings and will extend the computation by one discrete unit of computation. By specifying a unique order by which the opcodes in a verb have to be executed we arrive at a deterministic transcript for the complete computation. It is clear that this is equivalent to defining a specific interpreter.

In order to do this, we need to specify the aritmetization **map** which, as mentioned previously, is a subtree-index retrieval function that, upon input of a verb and subtree/internal node index $i \in [\lambda - 1]$, returns the verb (noun and associated action array) corresponding to the given internal node/subtree index. Although any ordering can be used, we will order the nouns (and hence the verbs) first by largest depth and secondly by left/right placement respectively, with deepest rightmost coming first. Note that we can equivalently choose deepest leftmost without changing any of the provided formal properties. Thus, for simplicity of exposition we will refer to the aritmetization as **map** := DFS, where the index of each internal node is the order in which a Depth-First Search walk starting from the root first explores it, excluding leaves. Note that this defines an ordering (with respect to a verb F) of all verbs F_1, F_2 existing within F by looking at their labels:

$$F_1 >_F F_2 \Leftrightarrow \text{map}(i_1, F) = F_1, \text{map}(i_2, F) = F_2 \text{ and } i_1 > i_2.$$

If $\forall F \in \mathcal{F}, >_F$ is a strict ordering, we say that **map** enforces a strict ordering on subtrees in the set $\mathcal{F}$.

Furthermore, for a given verb F , we can query its deepest and rightmost subtree's index that has a non-null action pending by looking at the largest non-null action index i . This will correspond to the subtree defined by the i -th zero in the dyck word of the verb's noun and is represented using **getIndex** : $\mathcal{V}^\lambda(\mathbb{F}, \mathcal{X}) \rightarrow [\lambda - 1]$. Note that *any* total ordering can be used to generate the **getIndex** function and we keep this generality in the below specification. This function is generically defined as:

$$\text{getIndex}^\Lambda(F) = \underset{\text{map}(i, F) \in \text{NockProgram}^\Lambda}{\text{argmax}} (\text{map}(i, F)).$$

Definition 4.4 (Nock Evaluation Trace). *The consistent evaluation trace $T = (t_0, \dots, t_n)$ of a verb F is an ordered set of verbs. Trace T is a (the) consistent trace for F of size n iff:*

1. $t_0 = F$
2. $\forall i \in [n], \text{Nock}(t_i, t_{i+1}) = 0$

Furthermore, for a consistent trace T of a verb F (denoted as $T(F)$) we also provide the following distinctions:

1. *If t_n is in terminal state or null, the trace T is complete.*
2. *If T is complete and t_n is not null, the verb F is valid.*

3. If T is complete and t_n is null, the verb F is invalid.
4. If t_n is not in terminal state or null, the trace T is partial up to n steps and incomplete.
5. If $c \in \mathcal{V}$ is an element of $\mathsf{T}(\mathsf{F})$, c is a continuation (of F).

A terminating Nock verb is an element of the NockVM language, i.e. has a finite transcript.

Definition 4.5. A verb F terminates (or is finite) if it is an element of the following language:

$$\text{NockVM}^\Lambda := \{\mathsf{F} \in \mathcal{V}^\lambda(\mathbb{F}, \mathcal{X}) \mid \exists n \in \mathbb{N} \text{ and a complete trace } \mathsf{T} = (t_0, \dots, t_n) \text{ s.t. } t_0 = \mathsf{F}, \text{ and } \lambda \leq \Lambda\},$$

where $\mathcal{X} := \langle ?, =, /, \#, - \rangle$.

Corollary 4.1. All valid and invalid verbs are finite.

Lemma 4.2. Let $c \in \mathcal{V}^\lambda(\mathbb{F}, \mathcal{X})$ be a continuation (of some verb F) that is not followed by a null operator in a consistent evaluation trace. Then $\text{map}(c, \text{getIndex}(c)) \in \mathcal{D}(\mathcal{V}^\lambda(\mathbb{F}, \mathcal{X}))$.

Note, however, that not all elements of NockVM^Λ are actually programs, as the latter is required to be in a specific ‘starting’ state with respect to its actions: namely it should only have one * action on its root (Property 1) and nothing else in its action array (Property 2). All ‘partial states’ of finite verbs are also elements of NockVM^Λ , which is actually the set of all intermediate steps (or *continuations*) of all transcripts of finite verbs. If we take all valid programs $\text{NockValidProgram}^\Lambda := \{\mathsf{F} \in \text{NockProgram}^\Lambda \mid \mathsf{F} \text{ is a valid verb}\}$, we note that $\text{NockValidProgram}^\Lambda \subsetneq \text{NockVM}^\Lambda$. The verbs in this intersection are elements of partial transcripts for which the next subtree to be executed has not been retrieved. It is this subtree which will be in $\mathcal{D}(\mathcal{V}^\lambda(\mathbb{F}, \mathcal{X}))$ and thus executed as the next opcode. We make precise the exact structure of NockVM^Λ by showing its equivalence to the set of all continuations of finite verbs, defined below:

$$\text{FiniteTransitions}^\Lambda := \{\mathsf{F} \in \mathcal{V}^\lambda(\mathbb{F}, \mathcal{X}) \mid \mathsf{F} \text{ is a continuation of a finite verb and } \lambda \leq \Lambda\}.$$

Lemma 4.3.

$$\text{NockVM}^\Lambda \equiv \text{FiniteTransitions}^\Lambda$$

Theorem 4.4. If map enforces a strict ordering on finite verbs, then for every finite verb F and integer n , there exists at most one consistent evaluation trace of size n for F .

Finally, for the purposes of Section 5, we can define NockVMROM^Λ as in Definition 4.5 but without OP_{10} . Note that this is just the Nock opcodes without the ability to write to memory, and will form the basis for the VM that we will be compiling using the framework developed in the following section.

4.4 Noun Polynomial Representation

One last ingredient that is specific to the Nock language and will prove useful in its compilation is the representation of its data objects as polynomials. To this end, we specify a succinct polynomial representation of an n -ary tree, using the definitions in Section 2.3, and derive security bounds on its usage. We have defined a noun as a tuple of (1) a length λ ordered set of binary tree leaf elements, and (2) a length $2(\lambda - 1)$ dyck word, with a universe $\mathcal{N}^\lambda(\mathbb{F}) = \mathcal{L}_{dk}^{2 \cdot (\lambda - 1)} \times \mathcal{DT}^\lambda(\mathbb{F})$ where the superscript denotes restriction to strings of that length. We note that every ordered leaf set of length λ in $\mathcal{DT}^\lambda(\mathbb{F})$ coupled with a dyck word of length $2 \cdot (\lambda - 1)$ over the boolean alphabet (i.e. element of $\mathcal{L}_{dk}^{2 \cdot (\lambda - 1)}$) fully and uniquely specify a binary tree with λ leaves, each storing a field element from $\mathbb{F}$. More generally, we will refer to the space of nouns of all possible lengths smaller or equal to Λ as $\mathcal{N}^{\lambda \leq \Lambda}(\mathbb{F}) := \{(d, w) \mid d \in \mathcal{DT}^{|d|}(\mathbb{F}) \text{ and } w \in \mathcal{L}_{dk}^{2 \cdot (|d| - 1)}, |d| \leq \Lambda\}$, where $\text{len}(n) = |d|$ defines the number of leaves in the binary tree represented by the noun $n = (d, w)$.

Given the above representation, we define nouns (or, equivalently, labeled proper binary trees) as univariate polynomials over $\mathbb{F}$ of degree at most $\lambda \leq \Lambda$. We associate $dt \in \mathcal{DT}^\lambda(\mathbb{F})$ with the univariate polynomial F_d of degree less than λ whose coefficient vector equals $dt = (dt_1, \dots, dt_\lambda) \in \mathbb{F}^\lambda$, or

$$F_d(X) = \sum_{i=1}^{\lambda} dt_i \dot{X}^{i-1} \text{ where } X \text{ indeterminate.}$$

We represent elements of the dyck language (or dyck words) w of length $2(\lambda - 1)$ over the boolean alphabet as $w = w_1, \dots, w_{2(\lambda-1)}$ where $w \in \mathcal{L}_{dk}^{2(\lambda-1)}$ as a polynomial with coefficients in $\mathbb{F}$, where:

$$F_w(X) = \sum_{i=1}^{2(\lambda-1)} w_i \dot{X}^{i-1} \text{ where } X \text{ indeterminate.}$$

The coefficients here are boolean $w_i \in \{0, 1\}$ but generalize to $\mathbb{F}^n$ when working with n -ary trees.

Definition 4.6. *The Noun Polynomial Representation (NPR) of a noun $\text{noun} := (dt, w) \in \mathcal{N}^\lambda(\mathbb{F})$ over field $\mathbb{F}$ is a mapping $\Phi : \mathcal{N}^\lambda(\mathbb{F}) \in \mathbb{F}^\lambda[X] \times \mathbb{F}^\lambda[X] \times \mathbb{F}^{2(\lambda-1)}[X]$ from nouns over $\mathbb{F}$ to a tuple of polynomials with coefficients in $\mathbb{F}$ defined as:*

$$\Phi(dt, w)[X_1, X_2, X_3] = \left(X_1^{|dt|}, \sum_{i=1}^{\lambda} dt_i X_2^{i-1}, \sum_{i=1}^{2(\lambda-1)} w_i X_3^{i-1} \right) \text{ where } X_1, X_2, X_3 \text{ indeterminates.}$$

Definition 4.7. *Let fn denote the fingerprinting function $\text{fn} : (\cup_{\lambda \leq \Lambda} \mathcal{N}^\lambda(\mathbb{F})) \rightarrow \mathbb{K} \times \mathbb{K} \times \mathbb{K}$ for nouns over field $\mathbb{F}$ with length $\lambda \leq \Lambda$ and evaluations in $\mathbb{K}$ as:*

$$\text{fn}(n = (dt, w); r) = \Phi(dt, w)[r, r, r] = (r^{|dt|}, F_d(r), F_w(r)), \quad (1)$$

where $r \in \mathbb{K}$ is uniformly randomly sampled.

Lemma 4.5. *The function fn satisfies the following property:*

$$\Pr_{r \in \mathbb{K}} [\text{fn}(n_1; r) = \text{fn}(n_2; r); n_1 \neq n_2] \leq \frac{2 \cdot (2\Lambda - 1)}{|\mathbb{K}|} := \epsilon(\Lambda, \mathbb{K}), \quad (2)$$

where the probability is taken over the random coins of fn .

Corollary 4.6. *For a list of nouns $n_1, \dots, n_k$ of length at most Λ , the probability that there exists a collision (i.e. indices $i, j \in [k]$ for which $n_i \neq n_j$ but $\text{fn}(n_i) = \text{fn}(n_j)$) satisfies the following property:*

$$\Pr_{r \in \mathbb{K}} [\exists i, j \in [k], n_i \neq n_j, \text{fn}(n_i; r) = \text{fn}(n_j; r)] \leq 1 - (1 - \epsilon(\Lambda, \mathbb{K}))^{k(k-1)/2} < \epsilon(\Lambda, \mathbb{K}) \cdot \frac{k(k-1)}{2}.$$

For clarity, we define the collision soundness error $\epsilon_{\text{col}}^{\Lambda, \mathbb{F}} := \epsilon(\Lambda, \mathbb{K})$ of a single instance in the following sections. Requiring that $\Lambda, k \in \text{negl}(|\mathbb{K}|)$ means that the above scheme is sufficiently secure with respect to noun length Λ and total noun number k (or ‘table size’). Instantiating with $|\mathbb{K}| \approx 2^{192}$, $\Lambda = 2^7$ and $k = 2^{32}$ yields 120 bits of security. If we keep the size of nouns constant at 2^7 , we get that a maximal table size of $k = 2^{52}$ distinct nouns yields around 80 bits of security. In general, the security level is approximately $\log_2 |\mathbb{K}| - 2 \log_2 k - \log_2 (\Lambda) - 1$. In the case where $\Lambda = 2^9$, we get 81 and 121 bits of security for table sizes of 2^{51} and 2^{31} respectively.

4.5 Noun Validation via Recursive Decomposition

A fundamental property of nouns, and thus of the NPR, is their iterative algebraic structure. Since by definition every noun can be decomposed into smaller nouns (which could be atoms), a simple yet sufficient way to check the validity of a given noun involves verifying that its decomposition is done correctly. If we have a valid noun $n \in \mathcal{N}^\lambda$ for which the left and right children are nouns n_L, n_R with λ_L, λ_R leaves respectively, it follows that $n_L \in \mathcal{N}^{\lambda_L}$ and $\mathcal{N}^{\lambda_R}$. We denote this decomposition by $n = \text{cons}(n_L, n_R)$ and note that, by uniqueness, it is well-defined. Thus, if it is known that the children $n_{L,R}$ are valid nouns of sizes $\lambda_{L,R}$, all we have to do is verify that they are the valid decompositions of n , i.e. verify that $n = \text{cons}(n_L, n_R)$. We make this precise in the following lemma:

Lemma 4.7.

$$\exists! \lambda_L, \lambda_R \in \mathbb{N}_+ \text{ s.t. } \lambda = \lambda_L + \lambda_R, n_L \in \mathcal{N}^{\lambda_L}(\mathbb{F}), n_R \in \mathcal{N}^{\lambda_R}(\mathbb{F}) \Leftrightarrow n = \text{cons}(n_L, n_R), n \in \mathcal{N}^\lambda(\mathbb{F}).$$

We can leverage the NPR in order to check a noun’s validity by recursively checking the cons relation. To do this, we use the following lemmas:

Lemma 4.8. Let $\Phi(n)[X] = (\Phi(n)_1[X], \Phi(n)_2[X], \Phi(n)_3[X]) \in \mathbb{F}^3[X]$ be the NPR of a valid noun n . If λ_L, λ_R are the number of leaves of its left and right children respectively, it follows that:

$$\begin{aligned} \Phi(n)_1 &= \Phi(n_L)_1 \cdot \Phi(n_R)_1 \\ \Phi(n)_2 &= \Phi(n_L)_2 + X^{\lambda_L} \cdot \Phi(n_R)_2 \\ \Phi(n)_3 &= \Phi(n_L)_3 + X^{2(\lambda_L-1)} + X^{2\lambda_L-1} \cdot \Phi(n_R)_3 \end{aligned} \Leftrightarrow n = \text{cons}(n_L, n_R).$$

Lemma 4.8 is actually rather powerful, as it means that we can verify the validity of some noun by only looking at its NPR. Indeed, by iteratively decomposing the noun into its left and right children until we reach atoms, it is possible to check that it is valid by only performing 3 algebraic equality checks on its NPR polynomial and those of its children. This is useful as, in the RAP arithmetization, this verification requires only three gates and can be done with a constant number of columns for arbitrary-depth nouns. The process for this can be specified in the following logic by utilizing a multiset object `mset` on which to add/remove nouns. We initialize `mset` by pushing the noun n that we want to verify, then run the following loop until `mset` is empty (and we output `accept`):

1. Remove some n from `mset`.
2. If n is not an atom:
 - (a) Check that $n = \text{cons}(n_L, n_R)$. Reject if it is not.
 - (b) Add n_L, n_R onto `mset`.

Algorithm 1 Noun Validity Verification

```

1: procedure CHECKVALIDNOUN(noun  $n_0$ ; transcript  $T$ )
2:   Initialize mset =  $\{n_0\}$ ,  $i = 0$ 
3:   Parse  $(t_1, \dots, t_K) \leftarrow T$ 
4:   while mset  $\neq \emptyset$  do
5:      $i++$ 
6:     Parse  $(n, n_L, n_R) \leftarrow t_i$ 
7:     If  $n \notin \text{mset}$ , return Reject
8:     mset  $\leftarrow \text{mset} \setminus \{n\}$ 
9:     if  $\text{type}(n) = \text{atom}$  then
10:      If  $n_L, n_R \neq \perp$ , return Reject
11:    else
12:      If  $n \neq \text{cons}(n_L, n_R)$ , return Reject
13:      mset  $\rightarrow \text{mset} \cup \{n_L, n_R\}$ 
14:    end if
15:  end while
16:  return Accept
17: end procedure

```

We can visualize the above process (formalized in Algorithm 1) as performing a walk on n . Starting at the root, we decompose into the left and right children n_L, n_R (by checking $n = \text{cons}(n_L, n_R)$). Then, we pick one of the children and iterate, keeping track of all nodes that still need to be traversed. As long as all nodes are checked, the order in which the traversal happens is not relevant since the only thing that needs to be verified is that either (1) the decomposition of each step is correct, or (2) the node is an atom and has no children. If the node is not an atom, it will result in two more nodes needing to be traversed (as the tree is proper), while if it is an atom no more children will be added to the stack.

Theorem 4.9. $n \in \mathcal{N}^\lambda(\mathbb{F}) \Leftrightarrow \exists T \text{ s.t. } \text{CheckValidNoun}(n; T) = \text{Accept}, |T| = 2\lambda - 1$

Theorem 4.9 demonstrates that `CheckValidNoun` acts as a verifier for whether a given input n_0 is actually a valid noun of a given size λ . This is done through access to a validity transcript T of size $2\lambda - 1$, which is necessary and sufficient to demonstrate that $n_0 \in \mathcal{N}^\lambda$. The main benefit of this verifier is that it utilized two extremely efficient gadgets and can be instantiated really cheaply: it only requires the verification of $\lambda - 1$ `cons` operations (which by Lemma 4.8 reduces to 3 degree 2 gate checks per `cons`) and $2\lambda - 1$ read/write operations to a set-membership gadget. The latter has been extensively studied in prior work and is particularly cheap to implement in the RAP arithmetization.

5 Verifiable VM Compilation: NockVM

Given the formal specification of the Nock programming language in Section 4, alongside an optimally succinct polynomial representation of its fundamental data primitives in Section 2.3, it becomes possible to fully specify a constraint system for verifying the Nock VM using the compilation framework developed in Section 3. In contrast to other languages, in Definition 4.5 we have specified all Nock opcodes as sequences of *local* updates to nouns, making it extremely efficient when compiled using the IO and traversal RAP gadgets. This is a result of three fundamental features of the Nock paradigm:

1. Nock is uniquely execution-memory separable,
2. The computation and memory traces of Nock can be modeled as tree traversals, and
3. All underlying variables can be succinctly represented using the NPR.

Indeed, instead of *imposing* structure on the memory trace through the use of a Merkle tree or other external data structure, Nock seamlessly allows for the verification of its memory graph exactly because it has strong algebraic structure to begin with. In this section, we provide the final ingredients needed to actualize a circuit that verifies the full execution and memory traces of any Nock program. Perhaps one of the most powerful aspects of Nock is its lack of distinct ‘registers’: indeed, by recursively verifying each opcode resolution, we can minimize the total number of constraints needed for an arbitrary noun, regardless of its size. This is intricately related to Nock’s *homoiconicity*: namely that all code and memory is represented in the same way, although we note that this property is not a necessary precondition for compiling other programming languages using this framework.

5.1 Nock Memory Representation

Before looking at the formal results that permit the construction of a minimal VM for Nock, the memory record semantics that the language follows need to be specified. These are a straightforward application of the NPR to the Nock addressing scheme, which is implicitly defined in a recursive manner through the memory lookup ‘slot’ operation $\backslash$, activated by running opcode 0. Indeed, the only opcode that performs memory lookups in any Nock program is OP_0 , doing so through an implicit call to the $\backslash(n_2, n_1)$ slot operator over two nouns n_1, n_2 . At a high level, this operator begins at the root of n_1 , which by definition is a proper binary tree, and parses n_2 as an integer label of the address of the node that needs to be returned *relative to* n_1 . Thus, $\backslash(n_2, n_1)$ recursively traverses n_1 until it finds the node whose relative address label (called the *axis*) equals n_2 .

This addressing scheme is actually natural to binary (and k -ary) trees and can be thought of as an ordering of the nodes as they are observed in the course of a BFS traversal. When we begin at some node n_1 , its address (or ‘axis’) is equal to 1: hence $\backslash(1, n_1) = n_1$. This is the base case and holds for all nouns in both binary and k -ary trees. Now, assume that we have a noun (i.e. a binary tree) n_i whose address is equal to axis_i relative to n_1 . We can recursively compute the addresses of its children relative to n_1 with a simple relation:

$$\text{axis}_{n_1}(n_i) = \text{axis}_i \Leftrightarrow \begin{cases} \text{axis}_{n_1}(\text{left}(n_i)) = 2 \cdot \text{axis}_i \\ \text{axis}_{n_1}(\text{right}(n_i)) = 2 \cdot \text{axis}_i + 1, \end{cases}$$

while for a k -ary tree, the address labeling scheme generalizes to:

$$\text{axis}_{n_1}(n_i) = \text{axis}_i \Leftrightarrow \forall j \in [k], \text{axis}_{n_1}(\text{child}_j(n_i)) = k \cdot \text{axis}_i + (j - 1).$$

The above scheme is useful in our context as it is *locally updatable*: indeed, if each node carries with it its axis, the process of constraining the axes of its children is a straightforward algebraic equality depending on the position of the child. Since we will be operating over memory addresses that are binary, we will restrict the analysis to the case where $k = 2$, which is equivalent to the definition of the Nock slot operator $\backslash$. This provides a natural definition of a Nock memory encoding, based on the NPR representation of nouns and the axis labeling scheme, specified below.

Definition 5.1. *The polynomial Nock Memory Representation $\text{pNMR}(\mathbb{F})$ is a memory encoding family (c.f. Definition 2.3) over $\mathbb{F}$ and $\lambda \in \mathbb{N}_+$, if:*

1. The value domain $\mathcal{VD} := (\text{root_fn}, \text{child_fn}) \in (\mathbb{F}^\lambda[X] \times \mathbb{F}^\lambda[X] \times \mathbb{F}^{2 \cdot (\lambda-1)}[X])^2$ consists of:
 - (a) The root noun fingerprint $\text{root_fn} := \Phi(\text{root})[X] \in \Phi(\mathcal{N}^\lambda(\mathbb{F}))[X]$ of noun root
 - (b) The referenced noun fingerprint $\text{child_fn} := \Phi(\text{child})[X] \in \Phi(\mathcal{N}^\lambda(\mathbb{F}))[X]$ of noun child
2. The address space $\mathcal{A} := \text{axis}_{\text{root}}(\text{child}) \in \mathbb{F}$ consists of the axis label of child with respect to root
3. The timestamp space $\mathcal{T} := X^k \in \mathbb{F}[X]$ encodes $k \in \mathbb{N}$ as a monomial with unit coefficient

Given that Definition 5.1 is fundamentally a polynomial representation of memory, we also provide a version whereby pNMR is evaluated at random extension field elements $\alpha, \beta \in \mathbb{K}$. This will be useful in encoding the memory records using a constant number of extension field elements in the RAPs that will be specified for Nock. We specify the size of this representation formally in Corollary 5.1.

Definition 5.2. The Nock Memory Representation $\text{NMR}^{\alpha, \beta}(\mathbb{F}, \mathbb{K})$ is a memory encoding family (c.f. Definition 2.3) over $\mathbb{F}$, extension field elements $\alpha, \beta \in \mathbb{K}$, where $\mathbb{K}$ an extension of $\mathbb{F}$, and $\lambda \in \mathbb{N}_+$, if:

1. The value domain $\mathcal{VD} := (\text{root_fn}, \text{child_fn}) \in \mathbb{K}^3 \times \mathbb{K}^3$ consists of two NPR evaluations over α :
 - (a) The root noun fingerprint $\text{root_fn} := \text{fn}(\text{root}; \alpha) \in \mathbb{K}^3$ of noun root
 - (b) The referenced noun fingerprint $\text{child_fn} := \text{fn}(\text{child}; \alpha) \in \mathbb{K}^3$ of noun child
2. The address space $\mathcal{A} := \text{axis}_{\text{root}}(\text{child}) \in \mathbb{F}$ consists of the axis label of child with respect to root
3. The timestamp space $\mathcal{T} := \beta^k \in \mathbb{K}$ is an integer k , encoded as a power of $\beta \in \mathbb{K}$

Corollary 5.1. For some field $\mathbb{F}$ and extension field $\mathbb{K}$ of degree e , $\text{NMR}^{\alpha, \beta}(\mathbb{F}, \mathbb{K}) \subseteq \mathbb{F}^{7e+1}$.

5.2 Execution-Memory Separation

We begin by formalizing the link between the structure of Nock, specified in Section 4, and the RAP gadgets designed in Section 3. The first step is to demonstrate that Nock can be verified in a modular fashion, namely through the decoupling of its execution and memory traces. More precisely, we show that the language NockVM (c.f. Definition 4.5) can be verified using two simple (local) languages - $\mathcal{L}_{\text{comp}}$ and $\mathcal{L}_{\text{mem}}$, using the request-response model (c.f. Section 2.2) on $\text{NMR}^{\alpha, \beta}$ memory records.

As an outline of the strategy, we will utilize Definitions 3.6 and 3.8 to prove that Nock is memory-execution separable and that it admits an efficient one-hot RAP arithmetization. This is achieved in the same fashion for both the computation and memory verification and goes as follows:

1. We specify labels $\mathcal{LT}_{\text{comp}}, \mathcal{LT}_{\text{mem}}$ for the computation and memory tables respectively, as per the semantics of Definition 3.6, including state and data label components.
2. $\mathcal{LT}_{\text{comp}}$ and $\mathcal{LT}_{\text{mem}}$ are then used to specify two verifiers $V_{\text{comp}}, V_{\text{mem}}$ that implicitly define the following transition function languages:

$$\mathcal{L}_{\text{comp}} = \{(x^{\text{comp}}, x_{\text{out}}^{\text{comp}}) \in \mathcal{LT}_{\text{comp}} \times (\mathcal{LT}_{\text{comp}} \cup \{\perp\})^3 \mid \exists w \text{ s.t. } V_{\text{comp}}(((x^{\text{comp}}, x_{\text{out}}^{\text{comp}}), w) = 0)\},$$

$$\mathcal{L}_{\text{mem}} = \{(x^{\text{mem}}, x_{\text{out}}^{\text{mem}}) \in \mathcal{LT}_{\text{mem}} \times (\mathcal{LT}_{\text{mem}} \cup \{\perp\})^2 \mid \exists w \text{ s.t. } V_{\text{mem}}(((x^{\text{mem}}, x_{\text{out}}^{\text{mem}}), w) = 0)\}.$$

3. We then demonstrate that NockVM (c.f. Definition 4.5) is $(\mathcal{L}_{\text{comp}}, \mathcal{L}_{\text{mem}})$ -traversable (c.f. Definition 3.7) and is thus memory-execution separable.
4. The languages $\mathcal{L}_{\text{comp}}, \mathcal{L}_{\text{mem}}$ are shown to satisfy Definition 3.8.
5. Given the above, we then utilize Theorems 3.2 and 3.6 to obtain an efficient RAP arithmetization for verifying Nock, including efficiency results pertaining to trace size and total number of gates.

The above process provides an explicit example of the generic compilation framework for traversable languages that was developed in Section 3. Indeed, if we can specify local verifiers for the memory and execution components of a given VM that satisfy Definition 3.7, a sound RAP arithmetization for language verification is then immediate. We note that this process is indeed general, and its efficiency hinges on the size and overhead incurred in producing a representation of the verifiers for the computation and memory traces in a RAP. To this end, we provide explicit constraint numbers, trace architectures and relevant schematics in the following subsections and the provided appendices.

5.2.1 Compute Table Traversal

For some element (verb) v_0 to be in **NockVM**, it has to have a finite set of trace elements $(v_1, \dots, v_N)$, also verbs, which satisfy the transition relation $\text{Nock}(v_i, v_{i+1}) = 0$ for all $i \in \{0\} \cup [N - 1]$. By its definition, (c.f. Algorithm 4), **Nock** performs the following (recursive) steps:

1. Takes in input and output verbs v_i, v_{i+1}
2. Retrieves the subtree index F_i (also a verb) from v_i that needs to be processed next
3. Parses the right child of F_i as $\text{formula}_i = \text{right}(F_i)$, being the next formula to be processed
4. Parses the left child of F_i as $\text{subject}_i = \text{left}(F_i)$, being the next subject to be processed
5. Executes $F_{i+1} := *[\text{subject}_i, \text{formula}_i]$ using the opcode update rules in Definition 4.2
6. Checks that v_{i+1} and v_i only differ in that subtree F_i is replaced by F_{i+1}

We specify a language $\mathcal{L}_{\text{comp}}$ which we show is both state-traversable and state-defined (c.f. Definition 3.8) over labeling universe $\mathcal{LT}_{\text{comp}} := \text{OP} \times \mathcal{IN} \times \mathcal{OUT}$, where $\mathcal{S} := \text{OP}$ and $\mathcal{DT} := \mathcal{IN} \times \mathcal{OUT}$, where $\mathcal{IN} := \mathcal{N}^\lambda(\mathbb{F}) \times \mathcal{N}^\lambda(\mathbb{F})$ is a subject-formula noun pair and $\mathcal{OUT} := \mathcal{N}^\lambda(\mathbb{F})$ is an output noun. For the given node, its state is the opcode **state** being resolved, while the requisite data field contains the input noun **input** := (subject, formula) (or subject-formula pair in the $\mathcal{IN}$ field) and the result of the resolution **output** (in the $\mathcal{OUT}$ field). Together, these form the label (state, (subject, formula), output). Since all of the children also have labels, we denote the label of the i -th child (or subformula) as $(\text{sf_state}_i, (\text{sf_subject}_i, \text{sf_formula}_i), \text{sf_output}_i)$. Indeed, these semantics are enough to fully verify a **Nock** execution trace, as $\mathcal{L}_{\text{comp}}$ will check the following:

1. (subject, formula) $\in \mathcal{IN} \equiv \mathcal{N}^\lambda(\mathbb{F}) \times \mathcal{N}^\lambda(\mathbb{F})$ subject-formula pair is consistent with **OP**, i.e. the formula head $\text{head}(\text{formula})$ is equal to the opcode number (or a cell for **cons**) specified in **OP**,
2. All subformulas that need to be computed have correctly formed inputs $(\text{sf_subject}_i, \text{sf_formula}_i)$ and an (optimistic) evaluation sf_output_i , which will be checked later on in the tree,
3. **output** is correctly computed from the inputs and sf_output_i subformula evaluations.

Satisfying Definition 3.8: At a high level, $\mathcal{L}_{\text{comp}}$ checks that the children each compute the dependencies specified by the given opcode (or state) verifying that the computation of (subject, formula) results in **output**. Since these are fully defined by the opcode, it is immediate that $\mathcal{L}_{\text{comp}}$ is state-defined. Moreover, the only opcodes that do not lead to recursive calls (and thus require no children) are opcodes 0 and 1, thus meaning that the set $\{\text{OP}_0, \text{OP}_1\} \subseteq \text{OP}$ satisfies the property that all valid nodes with opcode 0 and 1 as states (and only these) are leaves. This immediately implies that $\mathcal{L}_{\text{comp}}$ is a solely state traversable language with $\{\text{OP}_0, \text{OP}_1\}$ as its terminal states.

Proving Nock Execution Trace Verification: To build intuition on why this language, when compiled to a tree, suffices to verify the **Nock** execution trace, it is enough to note that every opcode is of the form $*[n_1[i, n_2]]$ and the transition function in **Nock** transforms the above form to one where the operator $*$ is applied to some deterministic computation over n_1, i, n_2 . The total number of subsequent calls to $*$ in the output of the opcode resolution is the **fanout** value of the given opcode, or the total number of children. For example, opcode 2 has three children whose evaluations need to be independently verified: $\text{sf}_1 = *[n_1, \text{head}(n_2)]$, $\text{sf}_2 = *[n_1, \text{tail}(n_2)]$ and $*[\text{sf}_1, \text{sf}_2]$. Each of these is a distinct child and the $\mathcal{L}_{\text{comp}}$ checks that $\text{output} = *[\text{sf}_1, \text{sf}_2]$ in addition to checking that $\text{OP} = \text{OP}_2$.

Moreover, for all inputs, the computation table needs to have access to the subject, formula and potentially (depending on the opcode) the children of n_2 . These will be encoded as accesses from memory, given that they are just lookups at given axis values on n_2 . More specifically, given access to **subject** = n_1 and **formula** = $[i, n_2]$, we thus require lookups to verify that:

1. For all opcodes: two lookups to verify $i = \text{head}(\text{formula}), n_2 = \text{tail}(\text{formula})$
2. For opcodes 2, 5, 7, 8, 9 : two additional lookups to verify $\text{head}(n_2), \text{tail}(n_2)$
3. For opcode 6 : four additional lookups to verify $\text{head}(n_2), \text{tail}(n_2), \text{head}(\text{tail}(n_2)), \text{tail}(\text{tail}(n_2))$.

Since these are all memory address lookups of fixed axis values with the input formula formula as the root, they are added as external memory registers. Thus, in the naive case the maximal number of memory records used are 6 or, in the language of the previous section, we require an arity of $M_{\text{Nock}} = 6$ (c.f. Definition 3.3) when operating over memory encoding NMR for language NockVM. We note, however, that given access to the root and one of the children, we can check the veracity of the second child solely through the use of the Nock concatenation operation `cons`, as `parent = cons(child1, child2)`. Thus, we can actually do 3 lookups instead of 6 at the additional cost of 3 `cons` checks. Given that the latter is extremely cheap and that the former costs are linear in the gate degree, we opt for using this optimization when designing the RAP. We provide the requisite formal statements below, alongside the full specification of the verifier for $\mathcal{L}_{\text{comp}}$ in Algorithm 5. Note that we do not include opcode 10, as this encodes write operations to memory and requires specific treatment. Thus, Algorithm 5 in Appendix B encodes Nock *without* memory writes.

Theorem 5.2. *The language $\mathcal{L}_{\text{comp}}(\mathbb{F}, \mathbf{e})$ specified by the verifier in Algorithm 5 satisfies Definition 3.8. Moreover, N evaluations of the verifier in $\mathcal{L}_{\text{comp}}$ can be checked by using a RAP arithmetization with N rows, $57 \cdot \mathbf{e} + 13$ columns and $20 \cdot \mathbf{e} + 13$ degree 4 transition gates.*

5.2.2 Memory Table Traversal

We now specify the walk that verifies the memory used, as formalized in Appendix B.2 and Algorithm 6. Its specification is much smaller than that for the compute table, as all the memory table needs to do is check that the initial input (the subject-formula pair seeded into the compute table) is a valid noun. Thus, it suffices for the memory table to provide two types of verification:

1. Check that the input (`input_subject`, `input_formula`) is a valid noun
2. All axis lookups are valid, i.e. that in every `memory_record = ((root, child), address, timestamp)` used in the compute table, `axisroot(child) = address`.

Checking Noun Validity The first objective is straightforward in the context of a walk: we proceed as in the previous section, defining a language $\mathcal{L}_{\text{mem}}$ checking that `parent = cons(left_child, right_child)` at every node, while going down the whole noun in a BFS traversal. By traversing the whole noun, this relation is checked between all parents and children, with a final check that the leaves are all atoms. That this suffices to verify that the input is a valid noun is formally proven in Theorem 4.9, whose construction forms the backbone of the $\mathcal{L}_{\text{mem}}$ language. Note that, by application of the NPR, this can be verified extremely cheaply using three algebraic checks as per Lemma 4.8.

We provide the (incomplete) semantics required: $\mathcal{DT}_1 := \mathcal{N}^\lambda(\mathbb{K}) \times (\mathcal{N}^\lambda(\mathbb{K}))^2$, where the data object is parametrized as (`input`, (`left_child`, `right_child`)). These can be easily represented in a RAP using the NPR, exactly as in the compute table, using 3 sets of $\mathbb{K}^3$ elements, for a total of $9 \cdot \mathbf{e}$ columns of $\mathbb{F}$, where $\mathbf{e}$ the order of the extension $\mathbb{K}$ over $\mathbb{F}$. Then all that the language $\mathcal{L}_{\text{mem}}$ does is check the concatenation relation between the parent and the children. It further pushes the children that are not atoms (which would be leaves in the traversal) into the stack for further recursive processing until all nouns have been ‘unrolled’ into atoms. Exactly like the previous example, this also informs the (incomplete set of) states required: $\mathcal{S}_1 = \{\text{decompose}, \text{halt}\}$, where if the state is `decompose`, the node is a cell and has at most two non-null children, while if it’s `halt`, then it’s a leaf and thus a terminal state.

Axis Lookup Verification Ensuring, however, that every memory record in the computation table is well formed, in that it satisfies the condition `axisroot(child) = address`, is somewhat more involved. However, we can leverage the fact that we are performing a walk down the tree in order to compute the axis of each memory record in an incremental fashion. To do this, we define an accumulator column (equivalent to the one used in $\text{RAP}_{\mathcal{L}-10}$, using a Log-Derivative aggregation multiset `intermediate`), and add two extra states $\mathcal{S}_2 = \{\text{read_root}, \text{verify_child}\}$ to the label of $\mathcal{L}_{\text{mem}}$, with each one having a fanout of 1. When in one of these states, no verification of the relationship between the parent and children is performed, but instead the node is deferred for verification at a later stage: namely there is only one child with a non-null label which has the exact same label as the parent. Finally, we will add two more variables, which are polynomials in $\mathbb{F}[X]$ to the data field of each nodes label: $\mathcal{LT}_2 = \{\text{axis_indicator}, \text{axis_counts}\}$. Together, these define the complete label for the memory traversal as $\mathcal{LT}_{\text{mem}} = (\mathcal{S}_1 \cup \mathcal{S}_2) \times (\mathcal{LT}_1 \times \mathcal{LT}_2) =$

$\{\text{decompose}, \text{halt}, \text{read_root}, \text{verify_child}\} \times ((\mathcal{N}^\lambda(\mathbb{F}), (\mathcal{N}^\lambda(\mathbb{F}), \mathcal{N}^\lambda(\mathbb{F}))) \times (\mathbb{F}[X] \times \mathbb{F}[X]))$. Note that in the RAP arithmetization, the polynomials `axis_indicator` and `axis_counts` are represented by their evaluation at a random extension field element, which makes them individual elements in $\mathbb{K}$.

Satisfying Definition 3.8: It is immediate that $\mathcal{L}_{\text{mem}}$ is state-traversable, with $\{\text{halt}\} = \mathcal{S}_{\text{leaf}}$. Moreover, it's also a state-defined dynamic labeling language, as the fanout of `decompose` is always two, while that of `halt` is always zero and the states $\{\text{read_root}, \text{verify_child}\}$ both have fanout of 1. This ensures that $\mathcal{L}_{\text{mem}}$ satisfies Definition 3.8 required to utilize Theorem 3.6.

Batch Proving Axis Consistency: When the state is `read_root`, we will remove a memory record from the IO column, check that its root node is equal to the parent label, and add the memory record to the global accumulator `intermediate`. Crucially, we add its timestamp (contained in the IO column as part of the memory record semantics) to the `axis_counts` and `axis_indicator` columns. Note that this timestamp is unique and represents the line number of the compute table at which the memory record was requested, thus having the closed form $z^i \in \mathbb{K}$, where i is the compute table line number.

Similarly, when the state is `verify_child`, we will remove an element from `intermediate` and check that it is equal to the parent node of the label. We will then remove z^i from the `axis_indicator` column and `address` · z^i from the `axis_counts` column, where `address` is the axis value of the child with respect to the root that is specified in the memory record. Furthermore, we also constrain the updates of the `axis_indicator` and `axis_counts` columns down a traversal: as elements of the label, their updates between parents and children can be constrained. When the parent is in the `decompose` state, the `axis_indicator` value will have to satisfy:

$$\text{axis_indicator}_{\text{parent}} = \text{axis_indicator}_{\text{left_child}} + \text{axis_indicator}_{\text{right_child}},$$

while the `axis_counts` variables will have to satisfy:

$$2 \cdot \text{axis_counts}_{\text{parent}} = \text{axis_counts}_{\text{left_child}} + \text{axis_counts}_{\text{right_child}} - \text{axis_indicator}_{\text{right_child}}.$$

Finally, if the node is in the `halt` state, we check that the parent is an atom, the children are null (foregoing any check of the `cons` operator), and that the `axis_counts`, `axis_indicator` variables equal zero.

This suffices to ensure that all axes are correctly counted: indeed, the `axis_indicator` variable keeps track of all the axis counts that are being checked in the given path, indexing the respective memory records through their unique timestamps. This is because we only add or remove memory records from `axis_indicator` when we hit a parent node that is either a root of some memory lookup (and thus add its timestamp z^i to the `axis_indicator` variable), or we hit a parent node that is a child of some memory lookup and thus remove the respective timestamp from `axis_indicator`. Then, when we are in a `decompose` state, we split `axis_indicator` into `axis_indicator`_{left_child} and `axis_indicator`_{right_child}. That their sum needs to equal `axis_indicator`, alongside the fact that they are verifiably elements of the form $\sum_{j=1}^k z^{i_j}$ means that the prover has to effectively ‘pick’ which timestamps go into which child.

Given the above, the `axis_counts` variable actually does the counting: at every step of the traversal (i.e. nodes that have state `decompose`) this variable is split into two variables `axis_counts`_{left_child} and `axis_counts`_{right_child}. Each of these is the same as the `axis_indicator` variable, only that the coefficients of each of the z^i are equal to the axis count up to that node. This is because the independent powers of `axis_counts`_{parent} are split into left and right children, then (1) either doubled for the left child, or (2) doubled and added to `axis_indicator`_{right_child} for the right child. Note that this implies *batching*: since each memory record is represented as some variable with a unique power, i.e. z^i , by doubling the `axis_counts` when we move left we double the coefficients of *all* timestamps that are in the left subtree, while if we double the `axis_counts` and add the `axis_indicator` value we double and add one to *every* power represented. When we remove a memory record from the `intermediate` aggregator in the `verify_child` state, we are also explicitly removing `axis` · z^i from `axis_counts`, where `axis` is obtained from the aggregator and is the value we need to verify.

Finally, by checking that both `axis_counts` and `axis_indicator` are zero in all `halt` states, we ensure that every memory record has been accounted for, both when initiating and when terminating an axis count, and that no record has been removed in a malicious way. This could happen if the split between child nodes is not honest, and would result in either the `axis_indicator` or `axis_count` variables containing unaccounted-for elements. Note that the uniqueness (per memory lookup) and uniformly random values of each timestamp z^i are key: since z is random, z^i is multiplicatively constrained and its inclusion

in the IO requests verified in the compute table. Alongside the fact that it is requested every time we are in a `read_root` or `verify_child` state, these checks leave no room for a malicious prover to provide a dishonest timestamp or to decompose the `axis_counts` and `axis_indicator` variables maliciously. This is because, at the end, the IO and `intermediate` multisets are also checked to be zero, so every timestamp needs to be removed from the IO multiset, added to `intermediate`, and then removed again. The above approach is thus equivalent to checking that all roots and children of memory records are found and that their axes are all correctly specified.

Arbitrary Subject Axis Lookups: Until now, the memory table performs a walk solely on the noun defined by the initial subject-formula pair. Although the batch axis lookup process outlined above allows for axis verifications over all subjects within the initial noun, it does not provide for lookups over mutated subjects, which are an integral part of the specification and are implemented through opcodes `OP2`, `OP7`, `OP8`, and `OP9`. The intuition behind this is that these new subjects form part of the ‘virtual’ memory, allowing end-users to fully leverage the expressibility of Nock by performing memory lookups on newly defined variables. Thankfully, the process above works in the exact same way here, as we can simply feed all new subjects as a seed to the memory table and leverage Corollary 3.7 to demonstrate that no efficiency overhead is incurred. Of course, the additional subjects will add extra rows for verification, but this does not affect the total number of gates or column number.

Intermediate Accumulator: It should also be noted that the accumulator variable `intermediate` that keeps track of all memory records which have had their roots verified and their timestamp values added to `axis_counts`, `axis_indicator` but have not yet completed their traversals to their child nodes are stored externally to the node labels. This can be thought of as a variable ‘global’ to the traversal. However, it is not necessary to provide for different techniques for the treatment of this process as it is equivalent to the method already formalized in Section 3.1. Indeed, when specifying the verifier in Algorithm 6, the `intermediate` accumulator is considered a witness element and its constraints are implemented in the exact same way as in $\text{RAP}_{\text{mem-IO}}$, $\text{RAP}_{\text{comp-IO}}$. We omit the specific formal details of this for simplicity, as the argument is equivalent to Theorem 3.2, noting simply that in this case the IO and `intermediate` columns function as the two memory record columns that are matched within the same table. To this end, a global terminal constraint check is required to ensure that the final value of `intermediate` is zero, while its initial value is also constrained to be zero as part of the memory table RAP. The formal specification of these details is provided in Appendix B.2.2 for completeness.

Theorem 5.3. *The language $\mathcal{L}_{\text{mem}}(\mathbb{F}, \mathbf{e})$ specified by the verifier in Algorithm 6 satisfies Definition 3.8. Moreover, N evaluations of the verifier in $\mathcal{L}_{\text{mem}}$ can be checked by using a RAP arithmetization with N rows, $54 \cdot \mathbf{e} + 5$ columns and $20 \cdot \mathbf{e} + 13$ degree 4 transition gates.*

5.2.3 A Minimal VM For Nock

Given all the above ingredients, we are finally able to put everything together and perform a compilation of the Nock programming language into a RAP that formally verifies it. The first immediate result of our design is that NockVM (c.f. Definition 4.5) is uniquely memory-execution separable (c.f. Definition 3.1) over the memory encoding $\text{NMR}^{\alpha, \beta}$. Furthermore, by inspection of the constraint system semantics for both the compute and memory tables in Appendices B.1.2 and B.2.2, it is immediate that the RAPs they define satisfy the properties required by Definition 3.2 with respect to Algorithms 5 and 6. This implies that NockVM is considered to have a canonical RAP representation.

Moreover, given that we are using the $\text{NPR}^{\alpha, \beta}$ memory encoding and that every memory request from the compute table is unique, it is immediate that our design is also unique request (c.f. Definition 3.5). This permits the usage of the compilation technique inherent in Theorem 3.2 with the RAPs defined in Appendices B.1.2 and B.2.2 to arrive at two RAPs which provide soundness guarantees for the IO matching between the computation and memory tables. Finally, since we have demonstrated in the previous section that the RAPs verifying Algorithms 5 and 6 satisfy Definition 3.8, we are able to use Theorem 3.6 (for the compute table) and Corollary 3.7 (for the memory table) to build two RAP_{walk} gadgets that, along with the IO columns, provide a full specification of NockVMROM. NockVMROM is defined as NockVM but without the ability to write to memory, or equivalently prohibition of the `#` operation. We formally define it as the language recognized by a conjunction on the verifiers for Algorithms 5 and 6 alongside consistency between inputs and outputs provided in the `mem_record`

elements (denoted as W_*^{IO} below), which also forms a part of each witness:

$$V_{\text{NockVMROM}}(X, Y; W_{\text{comp}}^{\text{int}}, W_{\text{comp}}^{\text{IO}}, W_{\text{mem}}^{\text{int}}, W_{\text{mem}}^{\text{IO}}) = 1 \Leftrightarrow \begin{aligned} &V_{\text{comp}}(X, (W_{\text{comp}}^{\text{int}}, W_{\text{comp}}^{\text{IO}})) = 1 \\ &V_{\text{mem}}(Y, (W_{\text{mem}}^{\text{int}}, W_{\text{mem}}^{\text{IO}})) = 1 \\ &(W_{\text{comp}}^{\text{IO}}, W_{\text{mem}}^{\text{IO}}) \in \mathcal{L}_{\text{mlO}}^{\text{NMR}^{\alpha, \beta}}. \end{aligned}$$

Theorem 5.4. *The language NockVMROM is uniquely memory-execution separable over memory encoding $\text{NMR}^{\alpha, \beta}$ with a canonical RAP arithmetization that is opcode and memory one-hot of size 10 and arity 3. Moreover, NockVMROM is traversible with respect to Algorithms 5 and 6 and can be verified by two RAPs $\text{RAP}_{\text{NockComp-IO}}, \text{RAP}_{\text{NockMem-IO}}$. The verification of N opcodes and L memory records has perfect completeness and negligible soundness error if $N, L \in \text{negl}(\mathbb{K})$. Moreover, the following efficiency requirements apply:*

1. $\text{RAP}_{\text{NockComp-IO}}$ computing N opcodes and L memory records

(a) Table Building

- One commitment to at most $N + (9 \cdot \mathbf{e} + 1) \cdot L$ elements in $\mathbb{F}$
- Two rounds of interaction, providing three random elements in $\mathbb{K}$
- $62 \cdot \mathbf{e} + 13$ columns

(b) Transition Constraints

- $15 + 24 \cdot \mathbf{e}$ gates of degree at most 4

(c) Initial & Terminal Constraints

- $2 \cdot \mathbf{e}$ gates of degree 1
- $\mathbf{e}$ gates of degree 3
- Two equality checks

(d) N total number of rows

2. $\text{RAP}_{\text{NockMem-IO}}$ verifying a forest with M nodes and L memory records

(a) Table Building

- One commitment to at most M elements in $\mathbb{F}$
- One round of interaction, providing a random element in $\mathbb{K}$
- $61 \cdot \mathbf{e} + 13$ columns in $\mathbb{F}$

(b) Transition Constraints

- $24 \cdot \mathbf{e} + 16$ gates of degree at most 4

(c) Initial & Terminal Constraints

- $2 \cdot \mathbf{e}$ gates of degree 1
- $2 \cdot \mathbf{e}$ gates of degree 3
- Four equality checks

(d) $M + 2L$ total number of rows

We note that given the arguments presented in the previous section, it can further be shown that NockVMROM models a NockVM instance as presented in Definition 4.5, just without the $\#$ operation enabled. We leave the formal equivalence demonstration to the extended version of this transcript but note that it does not affect the generic results provided, as these are agnostic to the verifier used. Indeed, by presenting any ISA's specification in the format of two distinct verifiers as above, the generic design and soundness analysis here suffices to construct a zkVM implementation. Thus, it can be productive to look at how ISAs can be represented with two tables that each (1) satisfy Definitions 3.2, 3.3 and 3.5 (and thus Theorem 3.2), and (2) verify traversable languages as in Definitions 3.6, 3.7. For any ISA satisfying these traits, a zkVM design is immediate by application of Theorems 3.6 and 3.2 to each table.

References

- [AST24] Arasu Arun, Srinath Setty, and Justin Thaler. Jolt: Snarks for virtual machines via lookups. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–33. Springer, 2024.
- [Axi25] Axiom. Openvm. 2025. <https://openvm.dev/whitepaper.pdf>.
- [Azt24] Aztec. Rap. 2024. <https://hackmd.io/@aztec-network/plonk-arithmetization-air>.
- [Azt25] Aztec. Noir. 2025. <https://noir-lang.org/docs/>.
- [BEG⁺94] Manuel Blum, Will Evans, Peter Gemmell, Sampath Kannan, and Moni Naor. Checking the correctness of memories. *Algorithmica*, 12:225–244, 1994.
- [BFL91] László Babai, Lance Fortnow, and Carsten Lund. Non-deterministic exponential time has two-prover interactive protocols. *Computational complexity*, 1:3–40, 1991.
- [BMIMT⁺22] Marta Bellés-Muñoz, Miguel Isabel, Jose Luis Muñoz-Tapia, Albert Rubio, and Jordi Baylina. Circom: A circuit description language for building zero-knowledge applications. *IEEE Transactions on Dependable and Secure Computing*, 20(6):4733–4751, 2022.
- [BSBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *Cryptology ePrint Archive*, 2018.
- [BSCG⁺13] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. Snarks for c: Verifying program executions succinctly and in zero knowledge. In *Annual cryptography conference*, pages 90–108. Springer, 2013.
- [BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *Theory of Cryptography: 14th International Conference, TCC 2016-B, Beijing, China, October 31–November 3, 2016, Proceedings, Part II 14*, pages 31–60. Springer, 2016.
- [BSCTV17] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. Scalable zero knowledge via cycles of elliptic curves. *Algorithmica*, 79:1102–1160, 2017.
- [CHM⁺20] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. Marlin: Preprocessing zkSnarks with universal and updatable srs. In *Advances in Cryptology–EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I 39*, pages 738–768. Springer, 2020.
- [CWC⁺21] Collin Chin, Howard Wu, Raymond Chu, Alessandro Coglio, Eric McCarthy, and Eric Smith. Leo: A programming language for formally verified, zero-knowledge applications. *Cryptology ePrint Archive*, Paper 2021/651, 2021.
- [EH24] Liam Eagen and Ulrich Haböck. Bypassing the characteristic bound in logUp. *Cryptology ePrint Archive*, Paper 2024/2067, 2024.
- [GPR21] Lior Goldberg, Shahar Papini, and Michael Riabzev. Cairo—a turing-complete stark-friendly cpu architecture. *Cryptology ePrint Archive*, 2021.
- [Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology–EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8–12, 2016, Proceedings, Part II 35*, pages 305–326. Springer, 2016.
- [GWC19] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive*, 2019.

- [Hab22] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Paper 2022/1530, 2022.
- [Lit25] Lita. Valida. 2025. <https://lita.gitbook.io/lita-documentation/quick-start/valida-zk-vm>.
- [LZZ⁺24] Tianyi Liu, Zhenfei Zhang, Yuncong Zhang, Wenqing Hu, and Ye Zhang. Ceno: Non-uniform, segment and parallel zero-knowledge virtual machine. Cryptology ePrint Archive, Paper 2024/387, 2024.
- [Suc25] Succinct. Sp1. 2025. <https://github.com/succinctlabs/sp1>.
- [Tea25] Risc Zero Team. Risc zero zkvm: Scalable, transparent arguments of risc-v integrity. 2025. <https://dev.risczero.com/proof-system-in-detail.pdf>.
- [YMDP16] Curtis Yarvin, Philip Monk, Anton Dyudin, and Raymond Pasco. Urbit: A solid-state interpreter. *White Paper*, May, 26, 2016.

A Nock Specification

A.1 Nock Operator Specification

The full specification of all existing Nock operators, with domain $\mathcal{N}^\lambda(\mathbb{F})$ and range $\mathcal{N}^\lambda(\mathbb{F}) \cup \{\perp\}$, are provided in Algorithm 2. The new ordering operator is separately specified in Algorithm 3.

A.2 Nock Transition Function Specification

The transition function (verifier) `Nock` for the Nock programming language, with statements from the universe $\mathcal{V}^\lambda(\mathbb{F}) \times \mathcal{V}^\lambda(\mathbb{F})$, is formally specified in Algorithm 4.

B NockVM Formula Specifications

B.1 Compute Table

B.1.1 Algorithmic Specification

The language $\mathcal{L}_{\text{comp}} := \{(x_{\text{comp}}^{\text{comp}}, x_{\text{out}}^{\text{comp}}) \mid \exists W_{\text{comp}} \text{ s.t. } V_{\text{comp}}((x_{\text{comp}}^{\text{comp}}, x_{\text{out}}^{\text{comp}}), W_{\text{comp}}) = 0\} \cup \{\perp\}^4$, where $x_{\text{comp}} \in \mathcal{LT}_{\text{comp}}, x_{\text{out}}^{\text{comp}} \in (\mathcal{LT}_{\text{comp}} \cup \{\perp\})^3$ and $(x_{\text{comp}}^{\text{comp}}, x_{\text{out}}^{\text{comp}}) \in \mathcal{LT}_{\text{comp}} \times (\mathcal{LT}_{\text{comp}} \cup \{\perp\})^3$ where:

$$x_{\text{comp}}^{\text{comp}} := (\text{state}, (\text{input_subject}, \text{input_formula}), \text{evaluation})$$

$$x_{\text{out}}^{\text{comp}} := (\text{subformula}_1, \text{subformula}_2, \text{subformula}_3)$$

$$\text{subformula}_1 = (\text{sf_state}_1, \text{child_one_input} := (\text{child_one_subject}, \text{child_one_formula}), \text{sf_evaluation}_1)$$

$$\text{subformula}_2 = (\text{sf_state}_2, \text{child_two_input} := (\text{child_two_subject}, \text{child_two_formula}), \text{sf_evaluation}_2)$$

$$\text{subformula}_3 = (\text{sf_state}_3, \text{child_three_input} := (\text{child_three_subject}, \text{child_three_formula}), \text{sf_evaluation}_3)$$

$$\mathcal{LT}_{\text{comp}} := \{\text{OP}_i \cup \text{cons}\}_{i \in [9]} \times ((\mathcal{N}^\lambda(\mathbb{F}), \mathcal{N}^\lambda(\mathbb{F})), \mathcal{N}^\lambda(\mathbb{F})),$$

$$W_{\text{comp}} := (\text{ln_prev}, \text{ln}, \text{mem_record}_1, \text{mem_record}_2, \text{mem_record}_3) \in (\mathbb{F}[X])^2 \times (\text{Rec}^{\text{NMR}^{\alpha, \beta}})^3.$$

The full algorithmic specification is provided in Algorithm 5.

Lemma B.1. *Fix $\text{rap_inputs} := (\alpha, z, a, b, c) \in \mathbb{K}^5$ as initial inputs to the RAP RAP_{comp} defined in Section B.1.2. RAP_{comp} admits a satisfying assignment for $(x_{\text{comp}}^{\text{comp}}, x_{\text{out}}^{\text{comp}}, W_{\text{comp}})$ if and only if $V_{\text{comp}}(x_{\text{comp}}^{\text{comp}}, x_{\text{out}}^{\text{comp}}; W_{\text{comp}}) = \text{Accept}$ as defined in Algorithm 5.*

Proof. (Sketch) Lines 2-17 of Algorithm 5 by inspection correspond directly to the Memory Record Accesses subsection of Section B.1.2.

Consider the ‘if’, ‘else if’ statements on Lines 18, 22, ..., 59 of Algorithm 5. These conditions on `formula_head` – that it is either a cell or otherwise has a value in the set $\{0, \dots, 9\}$ – are mutually exclusive; because of the ‘else’ statement on Line 64 they are also exhaustive. The immediate subsequent ‘then check..’ ties these conditions to particular states specified by opcodes. We can see how the constraints in the State Variables (SV) subsection of Section B.1.2 exactly mirror this.

For suppose the SV constraints hold. Then exactly one op_i equals 1 and all others equal 0 by SV constraints 2 and 3. By SV constraint 5, either $\text{op}_{11} = 1$ and $\text{f_h.dyck} \neq 0$ or $\text{op}_{11} = 0$ and $\text{f_h.dyck} = 0$. In the former case, the other SV constraints are satisfied and $\text{f_h.dyck} \neq 0$, $\text{op}_{11} = 1$ correspond exactly to `f_h` being a cell and `state = cons`. In the latter case, $\text{op}_j = 1$ for exactly one j and $\text{f_h.size} = z$, $\text{f_h.dyck} = 0$, $\text{f_h.leaf} = j$ by SV constraints 4-6. These are exactly the various conditions in the ‘if’ and ‘else if’ statements discussed in the previous paragraph, as claimed.

It’s easy to see by inspection for opcodes 0, 1, 2, 4, 7, and 9 that the code blocks gated by these if / else-if conditions are in correspondence with the constraints in Table 1 of the Evaluation Constraints (ECs) subsection of Section B.1.2. For the remaining opcodes the table contains the constraints corresponding to simple equalities between different variables and can be verified by inspection. We will now examine the other constraints for these opcodes and see the role they play. We will use the notation convention EC.a.b to denote the constraint numbered b under the heading for opcode a in the evaluation constraints subsection of Section B.1.2.

For opcode 3, the final output evaluation is 0 or 1, for true or false, which is the role of EC.3.1. (Notice that the evaluation has atomic type by the constraints in Table 1.) This value (0 or 1) must be correlated with the type (resp. cell or atom) of the first subformula evaluation. This correlation is achieved by EC.3.4 because constraints EC.3.2-3 guarantee that `inv_gen` is the generalized inverse of $z - \text{sf1.e.size}$.

For opcode 5, again the final output evaluation is 0 or 1, which is the role of EC.5.1. These values have to be correlated, respectively, to whether the two subformula evaluations are equal or not. Constraints EC.5.2-4 ensure that when the evaluation is 0, the two subformula products are equal. In the contrary case when the evaluation is 1, the random linear combination in EC.5.5 over the random field elements a, b, c must be invertible, so the two subformula products cannot be equal. This ensures that the final output evaluation is correlated with the equality of the subformula products.

For opcode 6 the only additional constraint is that the second subformula value must be 0 or 1.

For opcodes 8 and 11, the constraints are capturing something subtle in Algorithm 5. This is that performing cons should implicitly check that the sizes involved are nonzero, since the sizes are exponentiated values of z . This is precisely what the constraints EC.8.1 and EC.11.1 do. $\square$

B.1.2 Constraint Specification

Inputs: The table takes as initial inputs the following variables:

1. $(\alpha, \alpha^2, \alpha^3) : \mathbb{K}^3$ uniformly random extension field element powers used to keep track of the memory access timestamps
2. $(a, b, c) : \mathbb{K}^3$ uniformly random extension field elements
3. $z : \mathbb{K}$ uniformly random extension field element used to seed the NPR

Label Variables The following define the 3-ary tree labels, where $\mathcal{LT} = \text{OP} \times \mathcal{IN} \times \mathcal{OUT}$, for $\text{input} := (\text{input_subject}, \text{input_formula}) \in \mathcal{IN}$, $\text{evaluation} \in \mathcal{OUT}$ and the respective labels of the three children $(\text{child_one_input}, \text{sf_evaluation}_1)$, $(\text{child_two_input}, \text{sf_evaluation}_2)$, $(\text{child_three_input}, \text{sf_evaluation}_3)$.

1. $s : \mathbb{K}^3$ is a fingerprint triple $(s.\text{size}, s.\text{dyck}, s.\text{leaf})$ denoting input_subject
2. $f : \mathbb{K}^3$ is a fingerprint triple $(f.\text{size}, f.\text{dyck}, f.\text{leaf})$ denoting input_formula
3. $e : \mathbb{K}^3$ is the evaluation of f on s denoting evaluation
4. $\text{sf1_s} : \mathbb{K}^3$ is a fingerprint triple $(\text{sf1_s.size}, \text{sf1_s.dyck}, \text{sf1_s.leaf})$ denoting $\text{child_one_input}(1)$
5. $\text{sf1_f} : \mathbb{K}^3$ is a fingerprint triple $(\text{sf1_f.size}, \text{sf1_f.dyck}, \text{sf1_f.leaf})$ denoting $\text{child_one_input}(2)$
6. $\text{sf1_e} : \mathbb{K}^3$ is the evaluation of sf1_f on sf1_s denoting sf_evaluation_1
7. $\text{sf2_s} : \mathbb{K}^3$ is a fingerprint triple $(\text{sf2_s.size}, \text{sf2_s.dyck}, \text{sf2_s.leaf})$ denoting $\text{child_two_input}(1)$
8. $\text{sf2_f} : \mathbb{K}^3$ is a fingerprint triple $(\text{sf2_f.size}, \text{sf2_f.dyck}, \text{sf2_f.leaf})$ denoting $\text{child_two_input}(2)$
9. $\text{sf2_e} : \mathbb{K}^3$ is the evaluation of sf2_f on sf2_s denoting sf_evaluation_2
10. $\text{sf3_s} : \mathbb{K}^3$ is a fingerprint triple $(\text{sf3_s.size}, \text{sf3_s.dyck}, \text{sf3_s.leaf})$ of the first subformula subject, denoting $\text{child_three_input}(1)$
11. $\text{sf3_f} : \mathbb{K}^3$ is a fingerprint triple $(\text{sf3_f.size}, \text{sf3_f.dyck}, \text{sf3_f.leaf})$ of the first subformula formula, denoting $\text{child_three_input}(2)$
12. $\text{sf3_e} : \mathbb{K}^3$ is the evaluation of sf3_f on sf3_s , denoting sf_evaluation_3

Memory Record Accesses Formula decomposition is done with the memory record access columns:

1. $f_h : \mathbb{K}^3$ is the head of input_formula , denoted by formula_head
2. $f_t : \mathbb{K}^3$ is the tail of input_formula , denoted by formula_tail
3. $f_th : \mathbb{K}^3$ is the head of f_t , denoted by formula_tail_head
4. $f_tt : \mathbb{K}^3$ is the tail of f_t , denoted by formula_tail_tail
5. $f_tth : \mathbb{K}^3$ is the head of f_tt , denoted by $\text{formula_tail_tail_head}$
6. $f_ttt : \mathbb{K}^3$ is the tail of f_tt , denoted by $\text{formula_tail_tail_tail}$

Note that the following constraint is always operative

- $f = \text{cons}(f_h, f_t)$

. As per the specification, there also exist 3 memory records of which the first always verifies the opening of a formula at the given opcode. More specifically:

1. $\text{mem_record}_1 = ((\text{input_formula}, \text{formula_tail}), 3, \text{timestamp}_1) \in (\mathbb{K}^3)^2 \times \mathbb{F} \times \mathbb{K}$, constraining:
 - $\ln(i-1) \cdot \alpha = \text{timestamp}_1$
2. $\text{mem_record}_2 = ((\text{mem_root}_2, \text{mem_child}_2), \text{mem_axis}_2, \text{timestamp}_2) \in (\mathbb{K}^3)^2 \times \mathbb{F} \times \mathbb{K}$, constraining:
 - If $\sum_{i=2,5,6,7,8,9} \text{op}_i = 1$:
 - $\text{mem_root}_2 = \text{f_t}$, $\text{mem_child}_2 = \text{f_th}$, $\text{mem_axis}_2 = 2$
 - $\text{cons}(\text{f_th}, \text{f_tt}) = \text{f_t}$
 - $\ln(i-1) \cdot \alpha^2 = \text{timestamp}_2$
 - If $\text{op}_0 = 1$:
 - $\text{mem_root}_2 = \text{input_subject}$, $\text{mem_child}_2 = \text{evaluation}$, $\text{mem_axis}_2 = \text{f_t.leaf}$
 - $\ln(i-1) \cdot \alpha^2 = \text{timestamp}_2$
3. $\text{mem_record}_3 = ((\text{mem_root}_3, \text{mem_child}_3), \text{mem_axis}_3, \text{timestamp}_3) \in (\mathbb{K}^3)^2 \times \mathbb{F} \times \mathbb{K}$, constraining:
 - All are set to zero, unless $\text{op}_6 = 1$ when:
 - $\text{mem_root}_3 = \text{f_tt}$, $\text{mem_child}_3 = \text{f_tth}$, $\text{mem_axis}_3 = 2$
 - $\text{cons}(\text{f_tth}, \text{f_ttt}) = \text{f_tt}$
 - $\ln(i-1) \cdot \alpha^3 = \text{timestamp}_3$
 - Or $\text{op}_9 = 1$ when
 - $\text{mem_root}_3 = \text{sf1_e}$, $\text{mem_child}_3 = \text{sf2_f}$, $\text{mem_axis}_3 = \text{f_th.leaf}$, $\text{timestamp}_3 = \ln(i-1) \cdot \alpha^3$
4. The $\ln$ column, which keeps track of the total number of memory accesses requested until that row (inclusive), also needs to be constrained:

$$\ln(i-1) \cdot \left(\alpha \cdot \left(\sum_{j \in \{1,3,4\}} \text{op}_j(i) \right) + \alpha^2 \cdot \left(\sum_{j \in \{0,2,5,7,8\}} \text{op}_j(i) \right) + \alpha^3 \cdot \left(\sum_{j \in \{6,9\}} \text{op}_j(i) \right) \right) = \ln(i)$$

State Variables: For each row, we define the following set of variables as columns, which are one-hot selectors for the state OP:

1. $\text{op}_j : \mathbb{F}^{11}$ selectors for whether a certain opcode is activated, with $j \in \{0, \dots, 9\} \cup \{11\}$ with 11 representing **cons**, noting that opcode 10 as in the definition is not supported here
2. $\forall j \in \{0, \dots, 9\} \cup \{11\}, \text{op}_j(i) \cdot (1 - \text{op}_j(i)) = 0$
3. $\text{op}_{11}(i) + \sum_{j=0}^9 \text{op}_j(i) = 1$
4. $(1 - \text{op}_{11}(i)) \cdot \text{f_h.size}(i) = z \cdot \sum_{j=0}^9 \text{op}_j(i)$
5. $(1 - \text{op}_{11}(i)) \cdot \text{f_h.dyck}(i) + \text{op}_{11}(i) \cdot (1 - \text{f_h.dyck}(i) \cdot \text{cons.inv}(i)) = 0$
6. $(1 - \text{op}_{11}(i)) \cdot \text{f_h.leaf}(i) = \sum_{j=0}^9 j \cdot \text{op}_j(i)$

See the Concatenation Constraint subsection below for the full context around the variable **cons.inv**.

Evaluation Constraints Each opcode needs to have a specific constraint set that is activated based on the relevant selector. We break things down per opcode that is being run for clarity, then put these together in the evaluation gates in the end. In addition, we abuse notation when stating, for example, that $s = f$ to mean that $s.\text{size} = f.\text{size}$, $s.\text{dyck} = f.\text{dyck}$, $s.\text{leaf} = f.\text{leaf}$. This is made explicit in the final specification constraints in Table 1, where $\perp$ means no constraints are specified. In addition to the equality constraints (plus the **cons** constraints) of Table 1, we provide a set of non-linear constraints for specific opcodes below, which require an additional 4 gates.

We introduce an extra column **inv_gen** which plays a general role as an inverse quantity with particular semantics dictated per opcode.

	OP ₀	OP ₁	OP ₂	OP ₃	OP ₄	OP ₅	OP ₆
sf1_s	0	0	s	s	s	s	s
sf1_f	0	0	f_th	f_t	f_t	f_th	$(1 - \text{sf2.e.leaf}) \cdot \text{f_tth} + \text{sf2.e.leaf} \cdot \text{f_ttt}$
sf1_e	0	0	$\perp$	$\perp$	$(z, 0, \perp)$	$\perp$	e
sf2_s	0	0	s	0	0	s	s
sf2_f	0	0	f_tt	0	0	f_tt	f_th
sf2_e	0	0	$\perp$	0	0	$\perp$	$(z, 0, \perp)$
sf3_s	0	0	sf1_e	0	0	0	0
sf3_f	0	0	sf2_e	0	0	0	0
sf3_e	0	0	e	0	0	0	0
f_t	$(z, 0, \perp)$	e	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
f_th	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
e	$\perp$	$\perp$	$\perp$	$(z, 0, \perp)$	$1 + \text{sf1.e}$	$(z, 0, \perp)$	$\perp$

	OP ₇	OP ₈	OP ₉	OP ₁₁
sf1_s	sf2_e	cons(sf2_e, s)	s	s
sf1_f	f_tt	f_tt	f_tt	f_h
sf1_e	e	e	$\perp$	$\perp$
sf2_s	s	s	sf1_e	s
sf2_f	f_th	f_th	$\perp$	f_t
sf2_e	$\perp$	$\perp$	$\perp$	$\perp$
sf3_s	0	0	0	0
sf3_f	0	0	0	0
sf3_e	0	0	0	0
f_t	$\perp$	$\perp$	$\perp$	$\perp$
f_th	$\perp$	$\perp$	$\perp$	$\perp$
e	$\perp$	$\perp$	sf2_e	cons(sf1_e, sf2_e)

Table 1: Equality constraints for constructing V_{comp} .

• **Opcode 3:**

1. $\text{e.leaf}(i) \cdot (1 - \text{e.leaf}(i)) = 0$
2. $\text{inv_gen}(i) \cdot (\text{inv_gen}(i) \cdot (z - \text{sf1.e.size}(i)) - 1) = 0$
3. $(z - \text{sf1.e.size}(i)) \cdot (\text{inv_gen}(i) \cdot (z - \text{sf1.e.size}(i)) - 1) = 0$
4. $\text{inv_gen} \cdot (z - \text{sf1.e.size}) + \text{e.leaf} = 1$

• **Opcode 5:**

1. $\text{e.leaf}(i) \cdot (1 - \text{e.leaf}(i)) = 0$
2. $(1 - \text{e.leaf}(i)) \cdot (\text{sf1.e.size}(i) - \text{sf2.e.size}(i)) = 0$
3. $(1 - \text{e.leaf}(i)) \cdot (\text{sf1.e.dyck}(i) - \text{sf2.e.dyck}(i)) = 0$
4. $(1 - \text{e.leaf}(i)) \cdot (\text{sf1.e.leaf}(i) - \text{sf2.e.leaf}(i)) = 0$
5.
$$\text{inv_gen}(i) \cdot [a \cdot (\text{sf1.e.size}(i) - \text{sf2.e.size}(i)) + b \cdot (\text{sf1.e.dyck}(i) - \text{sf2.e.dyck}(i)) + c \cdot (\text{sf1.e.leaf}(i) - \text{sf2.e.leaf}(i)) + (1 - \text{e.leaf}(i))] = 1$$

• **Opcode 6:**

1. $\text{sf2.e.leaf}(i) \cdot (1 - \text{sf2.e.leaf}(i)) = 0$

• **Opcode 8:**

1. $\text{sf2.e.size}(i) \cdot \text{input_subject.size}(i) \cdot \text{inv_gen}(i) = 1$

- **Opcode 11:**

$$1. \text{sf1_e.size}(i) \cdot \text{sf2_e.size}(i) \cdot \text{inv_gen}(i) = 1$$

One can reduce the total number of gates by combining some of these constraints into one. E.g.:

$$\begin{aligned} & \text{op}_3(i) \cdot (\text{inv_gen} \cdot (z - \text{sf1_e.size}) + \text{e.leaf}) \\ & + \text{op}_5(i) \cdot \text{inv_gen}(i) \cdot [a \cdot (\text{sf1_e.size}(i) - \text{sf2_e.size}(i)) + b \cdot (\text{sf1_e.dyck}(i) - \text{sf2_e.dyck}(i)) \\ & + c \cdot (\text{sf1_e.leaf}(i) - \text{sf2_e.leaf}(i)) + (1 - \text{e.leaf}(i))] \\ & \text{op}_8(i) \cdot \text{sf2_e.size}(i) \cdot \text{inv_gen}(i) + \\ & \text{op}_{11}(i) \cdot \text{sf1_e.size}(i) \cdot \text{sf2_e.size}(i) \cdot \text{inv_gen}(i) = 1 \end{aligned}$$

B.2 Memory Table

B.2.1 Algorithmic Specification

The language $\mathcal{L}_{\text{mem}} := \{(x_{\text{mem}}^{\text{mem}}, x_{\text{out}}^{\text{mem}}) | \exists W_{\text{mem}} \text{ s.t. } V_{\text{mem}}((x_{\text{mem}}^{\text{mem}}, x_{\text{out}}^{\text{mem}}), W_{\text{mem}}) = 0\} \cup \{\perp\}^3$, where $x_{\text{mem}} \in \mathcal{L}\mathcal{T}_{\text{mem}}, x_{\text{out}}^{\text{mem}} \in (\mathcal{L}\mathcal{T}_{\text{mem}} \cup \{\perp\})^2$ and $(x_{\text{mem}}^{\text{mem}}, x_{\text{out}}^{\text{mem}}) \in \mathcal{L}\mathcal{T}_{\text{mem}} \times (\mathcal{L}\mathcal{T}_{\text{mem}} \cup \{\perp\})^2$ where:

$$x_{\text{mem}}^{\text{mem}} := (\text{state}, (\text{parent}, (\text{axis_counts}_P, \text{axis_indicator}_P)))$$

$$x_{\text{out}}^{\text{mem}} := ((\text{left_state}, (\text{left_child}, (\text{axis_counts}_L, \text{axis_indicator}_L))), (\text{right_state}, (\text{right_child}, (\text{axis_counts}_R, \text{axis_indicator}_R))))$$

$$\mathcal{L}\mathcal{T}_{\text{mem}} := \{\text{decompose}, \text{halt}, \text{read_root}, \text{verify_child}\} \times (\mathcal{N}^\lambda(\mathbb{F}), (\mathcal{N}^\lambda(\mathbb{F}), \mathcal{N}^\lambda(\mathbb{F}))) \times (\mathbb{F}[\mathbb{K}] \times \mathbb{F}[\mathbb{K}]),$$

$$W_{\text{mem}} := (\text{intermediate}_i, \text{intermediate}_{i+1}, \text{mem_record}) \in \mathbb{F}(\mathbb{K}) \times \mathbb{F}(\mathbb{K}) \times \text{Rec}^{\text{NMR}^{\alpha, \beta}}.$$

The full algorithmic specification is provided in Algorithm 6.

Lemma B.2. *The RAP given by RAP_{mem} as defined in Section B.2 admits a satisfying assignment for $(x_{\text{mem}}^{\text{mem}}, x_{\text{out}}^{\text{mem}}, W_{\text{mem}})$ if and only if $V_{\text{mem}}(x_{\text{mem}}^{\text{mem}}, x_{\text{out}}^{\text{mem}}, W_{\text{mem}}) = \text{Accept}$ as defined in Algorithm 6.*

Proof. (Sketch): The constraints of Table 2 correspond straightforwardly to Algorithm 6 by identifying the columns of the table with the ‘if’ and ‘else-if’ blocks of the algorithm. Following the approach of the proof of Lemma B.1 in the previous subsection yields the desired result. $\square$

B.2.2 Constraint Specification

Label Variables The following define the 2-ary tree labels, where $\mathcal{L}\mathcal{T} = \mathcal{S} \times \mathcal{N}^\lambda(\mathbb{F}) \times \mathcal{C}$, for $\text{parent} \in \mathcal{N}^\lambda(\mathbb{F}), (\text{axis_indicator}_P, \text{axis_counts}_P) \in \mathcal{C}$ and the respective labels of the two children $(\text{left_state}, \text{left_child}, \text{axis_indicator}_L, \text{axis_counts}_L), (\text{right_state}, \text{right_child}, \text{axis_indicator}_R, \text{axis_counts}_R)$. Here $\mathcal{S} := \{\text{decompose}, \text{halt}, \text{read_root}, \text{verify_child}\}$ denotes the four states of the system.

1. $\text{pt} : \mathbb{K}^5$ consists of:

- (a) A fingerprint triple $\text{pt.noun} := (\text{pt.size}, \text{pt.dyck}, \text{pt.leaf})$ denoting the noun parent
- (b) An element pt.indicator , denoting axis_indicator_P
- (c) An element pt.counts , denoting axis_counts_P

2. $\text{lc} : \mathbb{K}^5$ consists of:

- (a) A fingerprint triple $\text{lc.noun} := (\text{lc.size}, \text{lc.dyck}, \text{lc.leaf})$ denoting the noun left_child
- (b) An element lc.indicator , denoting axis_indicator_L
- (c) An element lc.counts , denoting axis_counts_L

3. $\text{rc} : \mathbb{K}^5$ consists of:

- (a) A fingerprint triple $\text{rc.noun} := (\text{rc.size}, \text{rc.dyck}, \text{rc.leaf})$ denoting the noun right_child
- (b) An element rc.indicator , denoting axis_indicator_R

	decompose	halt	read_root	check_child
pt.noun	cons(lc.noun, rc.noun)	$(z, 0, \perp)$	lc.noun	lc.noun
pt.indicator	lc.indicator + rc.indicator	0	lc.indicator – timestamp	lc.indicator + timestamp
pt.counts	$(1/2) \cdot (\text{lc.counts} + \text{rc.counts} - \text{rc.indicator})$	0	lc.counts – timestamp	lc.counts + axis · timestamp
lc.noun	$\perp$	$(0, 0, 0)$	$\perp$	$\perp$
rc.noun	$\perp$	$(0, 0, 0)$	$\perp$	$\perp$
mem.root	$(0, 0, 0)$	$(0, 0, 0)$	pt.noun	$\perp$
mem.child	$(0, 0, 0)$	$(0, 0, 0)$	$\perp$	pt.noun

Table 2: Equality constraints for constructing V_{mem} .

(c) An element **rc.counts**, denoting axis.counts_R

Memory Record Accesses As per the specification, there exists 1 memory record that is either written to or from. More specifically:

$$\text{mem_record} = ((\text{mem_root}, \text{mem_child}), \text{axis}, \text{timestamp}) \in (\mathbb{K}^3)^2 \times \mathbb{F} \times \mathbb{K}$$

Intermediate Aggregator As per the specification, an intermediate aggregator $\text{intermediate} \in \mathbb{K}$. This has the following requirements:

$$(\text{RLC}^\gamma(\text{mem_record}(i)) - \beta) \cdot (\text{intermediate}(i) - \text{intermediate}(i - 1)) = \text{op}_3(i) - \text{op}_2(i)$$

State Variables: For each row, we define the following set of variables as columns, which are one-hot selectors for the state values in $\mathcal{S}$:

1. $\text{op}_j : \mathbb{F}^4$ selectors for whether a certain state from $\mathcal{S} := \{\text{decompose}, \text{halt}, \text{read_root}, \text{verify_child}\}$ is respectively activated, with $j \in \{0, 1, 2, 3\}$
2. $\forall j \in \{0, \dots, 3\}, \text{op}_j(i) \cdot (1 - \text{op}_j(i)) = 0$
3. $\sum_{j=0}^3 \text{op}_j(i) = 1$

Equality Constraints The equality constraints are specified per state in Table 2. As before, $\perp$ means no constraint is imposed.

C Theorem & Lemma Proofs

C.1 Section 2.2: Offline Memory Verification

Proof of Lemma 2.1. For any instance in $\mathcal{L}_{\text{IO}}^{\text{MEM},K}$ one can define $k_2 = k$ and $\text{unique_requests} = \text{requests}$. Then for any $i \in [k]$, define $j = i \in [k_2]$ to satisfy the third condition of the definition of $\mathcal{L}_{\text{mlO}}^{\text{MEM},K}$. This establishes that $\mathcal{L}_{\text{IO}}^{\text{MEM},K} \subseteq \mathcal{L}_{\text{mlO}}^{\text{MEM},K}$ and the strict inclusion is trivial to see by constructing a sequence of requests that are not unique. $\square$

Proof of Lemma 2.2. This is Lemma 3 in [Hab22] with F replaced by $\mathbb{F}(\mathbf{Y})$. $\square$

Proof of Lemma 2.3. This is immediate from Lemma 2.2. $\square$

Proof of Lemma 2.4. It's obvious that

$$\mathcal{L}_{\text{uSE}}^{Y,K}(\mathbb{F}) \subsetneq \mathcal{L}_{\text{SE}}^{Y,K}(\mathbb{F})$$

so the lemma is established by (2).

The mapping which establishes (2) is the identity map. The only point that needs clarifying is that the condition

$$\exists \sigma \in \text{Sym}_k \text{ s.t. } \sigma(\text{responses}) = \text{requests}$$

in the definition of $\mathcal{L}_{\text{IO}}^{\text{MEM},K}$ is equivalent to

$$\prod_{i=1}^k (\text{RLC}^Y(\text{requests}(i)) + X) = \prod_{i=1}^k (\text{RLC}^Y(\text{responses}(i)) + X)$$

in the definition of $\mathcal{L}_{\text{MSE}}^{Y,K}(\mathbb{F})$.

If the responses are a permutation of the requests, then the product equality follows by the commutativity of polynomial multiplication. The reverse implication follows from the fact that $\mathbb{F}[X, \mathbf{Y}]$, interpreted as polynomials in X , is a unique factorization domain: the two products are two factorizations of the same polynomial, so the factors are equal up to a permutation, and thus the set of $\text{RLC}^Y(\text{requests}(i))$ is a permutation of the set of $\text{RLC}^Y(\text{responses}(i))$. Since these polynomials in $\mathbf{Y}$ are formally equivalent to the underlying lists, the conclusion follows. $\square$

Proof of Lemma 2.5. We begin computing

$$\sum_{i=1}^{n_1} \frac{1}{X + a_i(\mathbf{Y})}$$

by collecting terms with equal denominators:

$$\sum_{i=1}^{n_1} \frac{1}{X + a_i(\mathbf{Y})} = \sum_{s \in S} \frac{|a^{-1}(s)|}{X + s} = \sum_{t \in T} \frac{|a^{-1}(t)|}{X + t}.$$

Note that $|a^{-1}(t)| = 0$ for any $t \notin S$.

We can note that this transformation of the sum is analogous to the passage from the Riemann to the Lebesgue integral, so we now make the inverse passage to proceed:

$$\sum_{t \in T} \frac{|a^{-1}(t)|}{X + t} = \sum_{t \in T} \frac{|a^{-1}(t)|}{|b^{-1}(t)|} \cdot \frac{|b^{-1}(t)|}{X + t} = \sum_{i=1}^{n_2} \frac{|a^{-1}(b_i(\mathbf{Y}))|}{|b^{-1}(b_i(\mathbf{Y}))|} \cdot \frac{1}{X + b_i(\mathbf{Y})}.$$

This reveals that we can set $m_i = \frac{|a^{-1}(b_i(\mathbf{Y}))|}{|b^{-1}(b_i(\mathbf{Y}))|}$; but note this choice is not necessarily unique. Additionally, note that if $S = T$, then a is surjective onto T so $a^{-1}(b_i(\mathbf{Y}))$ is a non-empty set, and therefore m_i can be chosen such that $m_i \neq 0$.

Conversely, suppose

$$\sum_{i=1}^{n_1} \frac{1}{X + a_i(\mathbf{Y})} = \sum_{i=1}^{n_2} \frac{m_i}{X + b_i(\mathbf{Y})}$$

for some field elements m_i . Again, collect terms with the same denominators:

$$\sum_{s \in S} \frac{|a^{-1}(s)|}{X+s} = \sum_{t \in T} \frac{\mu(t)}{X+t}$$

where

$$\mu(t) = \sum_{i \in b^{-1}(t)} m_i$$

is the sum of the m_i where the sequence b has the common value t . By the uniqueness of fractional decompositions, each term $\frac{|a^{-1}(s)|}{X+s}$ on the left-hand side must equal a unique corresponding term $\frac{\mu(t)}{X+t}$ on the right-hand side; in particular, each $s \in S$ equals some unique $t \in T$, or $S \subseteq T$.

Finally, if in addition all m_i satisfy $m_i \neq 0$ and the b_i are all unique, then we see $\mu(t) = m_i \neq 0$ for some $i = i(t)$ uniquely associated to each t . Thus every $\frac{\mu(t)}{X+t}$ term on the right-hand side must have a matching $\frac{|a^{-1}(s)|}{X+s}$ term on the left, so $T \subseteq S$ yielding $S = T$. $\square$

Proof of Lemma 2.6. The equivalence map is the identity. For an instance $(\text{requests}, \text{responses}) \in \mathcal{L}_{\text{mIO}}^{\text{MEM}, K}$, we are provided by definition with k such that $\text{requests} \in \text{Seq}_k^{\text{MEM}}$ and k_2 such that $\text{responses} \in \text{Seq}_{k_2}^{\text{MEM}}$. Define $k_1 = k$, $k_2 = k_2$; M is as in $\text{MEM} := \mathbb{F}^M$ to satisfy the first condition of $\mathcal{L}_{\text{umLDSE}}^{Y, K}(\mathbb{F})$. Also by definition, there is $\sigma \in \text{Sym}_{k_2}$ s.t. $\text{responses} = \sigma(\text{unique_requests})$ so the underlying sets of responses and requests are equal. Thus we apply Lemma 2.5 (particularly for the special case of set equality) to produce $m \in (\mathbb{F}_{\text{char}(\mathbb{F})} \setminus \{0\})^{k_2}$ satisfying the rest of the conditions of $\mathcal{L}_{\text{umLDSE}}^{Y, K}(\mathbb{F})$.

For an instance $(\text{requests}, \text{responses}) \in \mathcal{L}_{\text{umLDSE}}^{Y, K}(\mathbb{F})$, set $k = k_1$, $k_2 = k_2$ so $\text{requests} \in \text{Seq}_k^{\text{MEM}}$. By the definition of $\mathcal{L}_{\text{umLDSE}}^{Y, K}(\mathbb{F})$, the elements of the responses sequence are unique and the conditions of the definition allow us to apply Lemma 2.5 in the case where the $m_i \neq 0$ and the b_i are unique to conclude the underlying sets of the sequences requests and responses are equal. This implies the existence of a $\text{unique_requests} \in \text{Seq}_{k_2}^{\text{MEM}}$ with the properties as in the definition of $\mathcal{L}_{\text{mIO}}^{\text{MEM}, K}$. $\square$

C.2 Section 3: Verifiable VM Compilation

Proof of Theorem 3.2. Suppose $\mathcal{L}_{\text{Lang}}$ is a unique request language. Per the definition, this language is uniquely memory-execution separable $(x, T) \rightarrow (x_{\text{prog}}, x_{\text{mem}}, T_{\text{prog}}, T_{\text{mem}}, \text{IO}_{\text{prog}}, \text{IO}_{\text{mem}})$ and has a RAP arithmetization such that

$$x \in \mathcal{L}_{\text{Lang}} \Leftrightarrow \begin{array}{l} (x_{\text{prog}}, T_{\text{prog}}, \text{IO}_{\text{prog}}) \text{ satisfies } \text{RAP}_{\text{prog}} \\ (x_{\text{mem}}, T_{\text{mem}}, \text{IO}_{\text{mem}}) \text{ satisfies } \text{RAP}_{\text{mem}} \\ (\text{IO}_{\text{prog}}, \text{IO}_{\text{mem}}) \in \mathcal{L}_{\text{mIO}}^{\text{MEM}}. \end{array}$$

Since $\mathcal{L}_{\text{Lang}}$ is unique request and therefore – by virtue of Corollary 3.1 – unique response, we in fact have $(\text{IO}_{\text{prog}}, \text{IO}_{\text{mem}}) \in \mathcal{L}_{\text{IO}}^{\text{MEM}}$ *a priori* so

$$x \in \mathcal{L}_{\text{Lang}} \Leftrightarrow \begin{array}{l} (x_{\text{prog}}, T_{\text{prog}}, \text{IO}_{\text{prog}}) \text{ satisfies } \text{RAP}_{\text{prog}} \\ (x_{\text{mem}}, T_{\text{mem}}, \text{IO}_{\text{mem}}) \text{ satisfies } \text{RAP}_{\text{mem}} \\ (\text{IO}_{\text{prog}}, \text{IO}_{\text{mem}}) \in \mathcal{L}_{\text{IO}}^{\text{MEM}}. \end{array}$$

Define $\text{RAP}_{\text{prog-IO}}$, $\text{RAP}_{\text{mem-IO}}$ as the extensions of RAP_{prog} , RAP_{mem} detailed in the immediately preceding subsections.

We can see that the two conditions

$$\begin{array}{l} (x_{\text{prog}}, T_{\text{prog}}, \text{IO}_{\text{prog}}, \text{acc}_{\text{prog}}) \text{ satisfies } \text{RAP}_{\text{prog-IO}} \\ (x_{\text{mem}}, T_{\text{mem}}, \text{ver}, \text{acc}_{\text{mem}}) \text{ satisfies } \text{RAP}_{\text{mem-IO}} \end{array}$$

are equivalent to the conditions

$$\begin{array}{l} (x_{\text{prog}}, T_{\text{prog}}, \text{IO}_{\text{prog}}) \text{ satisfies } \text{RAP}_{\text{prog}} \\ (x_{\text{mem}}, T_{\text{mem}}, \text{IO}_{\text{mem}}) \text{ satisfies } \text{RAP}_{\text{mem}} \end{array}$$

together with the multiset aggregation constraint between x_{prog} , IO_{prog} , and acc_{prog} , the terminal constraint for acc_{prog} , the initial and constraints between ver , x_{mem} , and acc_{mem} , and the terminal constraint

for acc_{mem} . Thus it's apparent that what needs to be proved is that $(\text{IO}_{\text{prog}}, \text{IO}_{\text{mem}}) \in \mathcal{L}_{\text{IO}}^{\text{MEM}}$ is equivalent to these aggregation and initial and terminal constraints.

From Lemma 2.6, we see that $(\text{IO}_{\text{prog}}, \text{IO}_{\text{mem}}) \in \mathcal{L}_{\text{IO}}^{\text{MEM}}$ is equivalent to $(\text{IO}_{\text{prog}}, \text{IO}_{\text{mem}})$ being in the subspace of $\mathcal{L}_{\text{umLDSE}}^{Y,K}(\mathbb{F})$ where **requests** is unique. The logarithmic derivative aggregation and initial and terminal constraints are in fact exactly checking that $(\text{IO}_{\text{prog}}, \text{IO}_{\text{mem}})$ is in $\mathcal{L}_{\text{umLDSE}}^{Y,K}(\mathbb{F})$ by Lemma 2.5: the pair of **ver**, x_{mem} are equivalent to IO_{mem} , the initial and aggregation constraints create the logarithmic derivative sums, and the terminal constraints check the equality of the two sums with high probability. $\square$

Proof of Theorem 3.6. Checking that a stateful labeled tree is recursively defined by the function F is equivalent to exhaustively visiting each node of the tree and checking that the children of each node are determined by the function F . Let RAP_{walk} be as described above with $(\text{seed}, \text{initial_count}) = (\text{RLC}^\gamma(\text{slt}) \cdot \alpha, \alpha)$ and $\text{out}_{\text{walk}} = 0$. We will argue that this constraint system is precisely encoding and enforcing that $\{\text{current_node}\}_{i=1}^h$ is a Breadth-First Traversal (BFT) of $\text{slt} \in \mathcal{L}_{\text{SLT}}^n$ which at each node checks that the children of each node are consistent with the function F .

The F constraints are equivalent to the consistency check of the children, so it suffices for the rest of the proof to argue that the remaining constraints are equivalent to enforcing a BFT. As a piece of convenient notation throughout the proof, we'll write $j(i)$ for the unique j in row i such that $\text{st}(i, j) = 1$.

We take the following as self-evident. Suppose we are given a list of tree nodes whose first element is the root of the tree. Initialize a second list – a queue – with the root of the tree. Iterate through the first list and at each tree node add the children of that node in left-to-right order to the end of the queue. Then the given list is equal to the queue if and only if the given list is a full breadth-first traversal of the tree.

In our arithmetized model, we can give an equivalent formulation using polynomials. The given list is equivalently

$$\text{node_list}(X) = \sum_{i=1}^h \text{RLC}^\gamma(\text{current_node}(i, \cdot)) \cdot X^i.$$

We can also define

$$\text{node_list}_k(X) = \sum_{i=1}^k \text{RLC}^\gamma(\text{current_node}(i, \cdot)) \cdot X^i$$

to be the list of nodes in node_list that have been read after k steps of the queuing algorithm described above. We define $\text{node_queue}_k(X)$ to be the derived queue of nodes after k steps of the algorithm, so we have the initialized value

$$\text{node_queue}_0(X) = \text{RLC}^\gamma(\text{current_node}(1, \cdot)) \cdot X$$

and the recurrence relation

$$\text{node_queue}_{k+1}(X) = \text{node_queue}_k(X) + X^{\deg(\text{node_queue}_k(X))} \cdot \sum_{l=1}^{\text{fanout}(j(k+1))} \text{RLC}^\gamma(\text{children}(k+1, l, \cdot)) \cdot X^l.$$

In order that this be correct and sensible in all cases, we employ a convention that a sum with a lower index larger than its upper index equals 0, which occurs here exactly when $\text{fanout}(j(k+1)) = 0$ or equivalently when there are no children of node $k+1$.

This yields the degree recurrence relation

$$\deg(\text{node_queue}_{k+1}(X)) = \deg(\text{node_queue}_k(X)) + \text{fanout}(j(k+1))$$

which can be solved with a telescoping sum to yield $\deg(\text{node_queue}_k(X)) = 1 + \sum_{i=1}^k \text{fanout}(j(i))$. Plugging this into the recurrence relation for node_queue yields, from another telescoping sum, for $k \geq 1$,

$$\text{node_queue}_k(X) = \text{RLC}^\gamma(\text{current_node}(1, \cdot)) \cdot X + \sum_{i=1}^k \sum_{l=1}^{\text{fanout}(j(i))} \text{RLC}^\gamma(\text{children}(i, l, \cdot)) \cdot X^{1+l+\sum_{m=1}^{i-1} \text{fanout}(j(m))}.$$

We find from this that $\{\text{current_node}\}_{i=1}^h$ is a full BFT traversal if and only if $\text{node_list}(X) = \text{node_queue}_h(X)$, which holds with soundness error $\frac{h}{|\mathbf{K}|}$ if and only if $-\text{node_list}(\alpha) + \text{node_queue}_h(\alpha) = 0$ for randomly drawn $\alpha \in \mathbf{K}$.

Now, from the initial line constraint $\ln(1) = \alpha$ and the line number increment constraint $\ln(i) = \alpha \cdot \ln(i-1)$ it holds that

$$\ln(k) = \alpha^k$$

while from $\text{initial_count} = \alpha$ and $\text{count}(i) = \text{count}(i-1) \cdot \alpha^{\text{fanout}(i(j))}$ we find

$$\text{count}(k) = \alpha^{1 + \sum_{i=1}^k \text{fanout}(j(i))}.$$

Summing the Line-Indexed Multiset Argument Aggregation constraint and substituting the initial constraint involving $\text{acc_walk}(1)$ yields

$$\begin{aligned} \text{acc_walk}(k) = & - \sum_{i=1}^k \text{RLC}^\gamma(\text{current_node}(i, \cdot)) \cdot \alpha^i \\ & + \text{seed} + \sum_{i=1}^k \sum_{l=1}^{\text{fanout}(j(i))} \text{RLC}^\gamma(\text{children}(i, l, \cdot)) \cdot \alpha^{1+l+\sum_{m=1}^{i-1} \text{fanout}(j(m))} \end{aligned}$$

Note that $\text{seed} = \text{RLC}^\gamma(\text{slt}) \cdot \alpha = \text{RLC}^\gamma(\text{current_node}(1, \cdot)) \cdot \alpha$, so

$$\text{acc_walk}(k) = -\text{node_list}_k(\alpha) + \text{node_queue}_k(\alpha)$$

($\Rightarrow$) Suppose $\text{slt} \in \mathcal{L}_{\text{RVT}}^{F,n}$. Assigning $\{\text{current_node}\}_{i=1}^h$ according to a full BFT of the nodes of slt , and allocating $\ln$, count , acc_walk to satisfy the constraints of RAP_{walk} , then because the constraints are satisfied we let $k = h$ in $\text{acc_walk}(k) = -\text{node_list}_k(\alpha) + \text{node_queue}_k(\alpha)$ and use the definition of out_walk to achieve

$$\text{out_walk} = \text{acc_walk}(h) = -\text{node_list}_h(\alpha) + \text{node_queue}_h(\alpha).$$

But because the traversal is a full BFT, $\text{acc_walk}(h) = 0$ and so $\text{out_walk} = 0$ holds, as required.

($\Leftarrow$) Supposing the constraints of RAP_{walk} are satisfied and $\text{out_walk} = 0$ we again get

$$0 = -\text{node_list}_h(\alpha) + \text{node_queue}_h(\alpha)$$

which implies with soundness error $\frac{h}{|\mathbf{K}|}$ that $\{\text{current_node}\}_{i=1}^h$ is a full BFT traversal of a tree. $\square$

C.3 Section 4: Formalizing Nock

Proof of Lemma 4.2. Fix c a continuation not adjacent to a $\perp$. Let the next element of the trace be $t_+ \neq \perp$. By definition of the Nock transition function, since t_+ is non-null and the trace is consistent, we have:

1. The tail of $\text{map}(c, \text{getIndex}(c))$ is a cell.
2. If $\text{head}(\text{tail}(\text{map}(c, \text{getIndex}(c))))$ is an atom, then $\text{head}(\text{tail}(\text{map}(c, \text{getIndex}(c)))) \in [11]$.

By definition of the getIndex function, $\text{getIndex}(c)$ is a Nock program. Since (1) both the head and tail are in terminal state (by Property 2) while its root node is labeled with $*$ (by Property 1), it satisfies the requirements that its action array a has $a_1 = *, j \neq 1 : a_j = \perp$. $\square$

Proof of Lemma 4.3. Fix $F \in \text{NockVM}$. By definition there exists a complete trace that begins with F and so, trivially, F is a continuation of itself. Fix $F \in \text{FiniteTransitions}$. By definition, F is a continuation of some element $F_0 \in \text{NockVM}$. This is equivalent to the statement that there exists some j such that F is equal to the j -th index t_j of a complete trace $T = (F_0, t_1, \dots, t_{j-1}, F, t_{j+1}, \dots, t_n)$ for F_0 . However, $T' = (F, t_{j+1}, \dots, t_n)$ is also complete and thus $F \in \text{NockVM}$. $\square$

Proof of Theorem 4.4. Fix F a finite verb and integer n , meaning there exists a complete trace T of size n_0 .

We begin with the case where $n = n_0$. By Lemma 4.3, without loss of generality we can take F to be a continuation of some finite verb. If the next element t_+ in the complete trace is null, then by definition of the Nock transition function there is no other trace in which the next element is non-null, which suffices for uniqueness. If t_+ is non-null, by the Nock transition function, t_+ only differs at index $\text{getIndex}(F)$. Moreover, by Lemma 4.2 it follows that $\text{map}(F, \text{getIndex}(F)) \in \mathcal{D}(\mathcal{V})$ and thus t_+ will only differ at index $\text{getIndex}(F)$ by the deterministic update rule defined by the executed opcode. Note that this is unique for each verb $\text{map}(F, \text{getIndex}(F)) \in \mathcal{D}(\mathcal{V})$. Remains to show that $\text{getIndex}(F)$ always leads to a unique value for each F . However, this is immediate from the fact that map is a strict ordering, as it chooses the largest value by definition. If there was more than one largest value, the ordering would not be strict.

In the case where $n > n_0$, no such transcript exists as the final element in the trace is either null or in terminal state and so no next element exists that keeps the trace consistent. If $n < n_0$, the uniqueness of T means that the only consistent trace of size n is the trace T' that includes the first n elements of T . $\square$

Proof of 4.5. Given that the fingerprints are equal, the conditional $n_1 \neq n_2$ thus holds only if (1) $F_{d_1} \neq F_{d_2}$, (2) $F_{w_1} \neq F_{w_2}$, or (3) $X^{|dt_1|} \neq X^{|dt_2|}$ where $n_1 = (dt_1, w_1), n_2 = (dt_2, w_2)$.

In the case $F_{d_1} \neq F_{d_2}$, we know that $F_{d_1}(r_1) = F_{d_2}(r_1)$ where $r_1 \in \mathbb{K}$ uniformly random field element. Since $F_{d_1} - F_{d_2}$ is a non-zero polynomial of degree at most Λ , Schwartz-Zippel implies that this happens with probability at most $\Lambda/|\mathbb{K}|$. In the second case, the same argument yields an upper bound of $2 \cdot (\Lambda - 1)/|\mathbb{K}|$. Finally, if the noun lengths are not the same $X^{|dt_1|} - X^{|dt_2|}$ is non-zero giving an upper bound of $\Lambda/|\mathbb{K}|$. Since either one of the three of $(dt_1 \neq dt_2) \cap (|dt_1| = |dt_2|), |dt_1| \neq |dt_2|, w_1 \neq w_2$ holds with respective probabilities as above, we get by applying the union bound that:

$$\begin{aligned}
\Pr[\text{fn}(n_1) = \text{fn}(n_2) \cap n_1 \neq n_2] &= \Pr[\text{fn}(n_1) = \text{fn}(n_2) \cap (dt_1 \neq dt_2 \cup w_1 \neq w_2)] \\
&= \Pr[(\text{fn}(n_1) = \text{fn}(n_2) \cap dt_1 \neq dt_2) \cup (\text{fn}(n_1) = \text{fn}(n_2) \cap w_1 \neq w_2)] \\
&\leq \Pr[\text{fn}(n_1) = \text{fn}(n_2) \cap (dt_1 \neq dt_2 \cap |dt_1| = |dt_2|)] + \Pr[\text{fn}(n_1) = \text{fn}(n_2) \cap (dt_1 \neq dt_2 \cap |dt_1| \neq |dt_2|)] \\
&\quad + \Pr[\text{fn}(n_1) = \text{fn}(n_2) \cap w_1 \neq w_2] \\
&\leq \Pr[F_{d_1}(r) = F_{d_2}(r); dt_1 \neq dt_2, |dt_1| = |dt_2|] + \Pr[F_{w_1}(r) = F_{w_2}(r); w_1 \neq w_2] + \\
&\quad + \Pr[r^{|dt_1|} = r^{|dt_2|}; |dt_1| \neq |dt_2|] \\
&= \frac{2(\Lambda - 1)}{|\mathbb{K}|} + 2 \cdot \frac{\Lambda}{|\mathbb{K}|} = \epsilon(\Lambda, \mathbb{K}).
\end{aligned}$$

$\square$

Proof of Corollary 4.6. Let $p \leq \epsilon(\Lambda, \mathbb{K})$ be the probability of a collision between a pair of elements. Without loss of generality, we assume all nouns are different. The total number of distinct pairs of nouns is equal to $k \cdot (k - 1)/2$ so the probability of no table collisions is $(1 - p)^{k(k-1)/2}$. Negating this and bounding above yields the result. A non-optimal but easier to show bound is obtained by noting that the probability of at least one collision in $k(k - 1)/2$ pairs can be bounded by a union bound over the probability of a collision between distinct pairs, yielding an upper bound of $p \cdot k(k - 1)/2$ for the chance of at least one collision. $\square$

Proof of Lemma 4.7. $\Rightarrow$ Since n_R, n_R are valid nouns, each with λ_L, λ_R leaves respectively, the noun n for which $\text{left}(n) = n_L, \text{right}(n) = n_R$ (or $n = \text{cons}(n_L, n_R)$) has $\lambda = \lambda_L + \lambda_R$ leaves. This is also a proper binary tree by definition and thus $n \in \mathcal{N}^\lambda(\mathbb{F})$.

$\Leftarrow$ Since n is a valid noun with λ leaves, there exist n_L, n_R left and right children which are also valid nouns. Setting their number of leaves to λ_L, λ_R completes the proof, as n is proper. $\square$

Proof of Theorem 4.9. $\Rightarrow$ Given n is a valid noun with λ leaves, by Lemma 2.7 it contains $2\lambda - 1$ nodes. We construct $T = (t_1, \dots, t_{2\lambda-1})$ by performing a walk on n , starting at the root. For the i -th node n_i in the traversal, we define $t_i = (n_i, \text{left}(n_i), \text{right}(n_i))$ if n_i is a cell and $t_i = (n_i, \perp, \perp)$ if it is an atom. We then add its children in an `mset` multiset of unexplored nodes and pick one of these for the next iteration of the walk. Suffices to show T always accepts. If it rejects on Line 7, it means that t_i contains a node that was not previously added to `mset`, but this is impossible as each t_i is constructed by walking the tree hence all nodes added would have previously been seen and only traversed nodes are removed. Lines 10 and 12 also always verify by definition of t_i for atoms and cells. Since n is a proper binary tree of finite length, the while loop terminates and thus `CheckValidNoun` accepts.

$\Leftarrow$ If some T with $2\lambda - 1$ elements is an accepting transcript for `CheckValidNoun`, it will contain k atoms and $2\lambda - 1 - k$ cells. Since each atom adds no new elements to `mset`, each cell adds 2 new elements to `mset` and both remove the current element, for a transcript of size $2\lambda - 1$ to verify (starting with one element and ending with none), it means that $1 + (2 - 1) \cdot (2\lambda - 1 - k) + (-1) \cdot k = 0$. This implies that $k = \lambda$, or that there are λ leaves and $\lambda - 1$ internal nodes. Since every node in T has been added at some point (as Line 7 never rejects), then all valid children of internal nodes are checked since Line 12 never rejects. Further, for all atoms the second and third elements of their transcript indices are null (as Line 10 never rejects). Thus n has λ leaves and is a valid noun, hence $n \in \mathcal{N}^\lambda(\mathbb{R})$. $\square$

C.4 Section 5: Verifiable VM Compilation

Proof of 5.2. We demonstrate the claim by construction. Define $\mathcal{L}_{\text{comp}}$ as a recursively verifiable stateful proper n -ary tree (c.f. Definition 3.6) with labeling space $\mathcal{LT}_{\text{comp}}$ and verifier V_{comp} . Fix $S_{\text{leaf}} := \{\text{OP}_0, \text{OP}_1\}$. Key observation is that, according to V_{comp} , only opcodes 0 and 1 do not have any children and thus form leaves. All other opcodes do, and they are deterministic based on the opcode. Thus, we can define the fanout (solely a function of state) to equal the number of children (or invocations of the operator in the result). These observations suffice to satisfy Definition 3.8. Subsequently, Theorem 3.6 applied to $\mathcal{L}_{\text{comp}}$ yields the result. $\square$

Proof of 5.3. We demonstrate the claim by construction. Define $\mathcal{L}_{\text{mem}}$ as a recursively verifiable stateful proper n -ary tree (c.f. Definition 3.6) with labeling space $\mathcal{LT}_{\text{mem}}$ and verifier V_{mem} . Fix $S_{\text{leaf}} := \{\text{halt}\}$. Key observation is that, according to V_{mem} , only `halt` does not have any children and thus forms all leaves. All other states do: `decompose` has a fanout of 2 and `read_root/verify_child` have fanout of 1. These observations suffice to satisfy Definition 3.8. Subsequently, Corollary 3.7 applied to $\mathcal{L}_{\text{mem}}$ yields the result. $\square$

Algorithm 2 Nock: Noun Operators

```
1: procedure ?(noun  $n$ )
2:   if  $\text{type}(n) = \text{cell}$  then return 0
3:   end if
4:   return 1
5: end procedure
6: procedure =(noun  $n_1$ , noun  $n_2$ )
7:   if  $n_1 = n_2$  then return 0
8:   end if
9:   return 1
10: end procedure
11: procedure +(noun  $n$ )
12:   if  $?(n) = 1$  then return  $(1 + n) \bmod p$ 
13:   end if
14:   return  $\perp$ 
15: end procedure
16: procedure /(noun  $\text{axis}$ , noun  $n$ )
17:   if  $?(\text{axis}) = 1$  then
18:     if  $= (\text{axis}, 0)$  then return  $\perp$ 
19:     else if  $= (\text{axis}, 1)$  then return  $n$ 
20:     else if  $= (\text{axis}, 2)$  then
21:       if  $?(n) = 0$  then return  $\text{head}(n)$ 
22:       else
23:         return  $\perp$ 
24:       end if
25:     else if  $= (\text{axis}, 3)$  then
26:       if  $?(n) = 0$  then return  $\text{tail}(n)$ 
27:       else
28:         return  $\perp$ 
29:       end if
30:     else
31:        $b \leftarrow \text{axis} \bmod 2$ 
32:       return  $/(2 + b, /((\text{axis} - b)/2, n))$ 
33:     end if
34:   end if
35:   return  $\perp$ 
36: end procedure
37: procedure #(noun  $\text{axis}$ , noun  $\text{new}$ , noun  $\text{old}$ )
38:   if  $?(\text{axis}) = 1$  then
39:     if  $= (\text{axis}, 0)$  then return  $\perp$ 
40:     else if  $= (\text{axis}, 1)$  then
41:        $\text{new} \rightarrow \text{old}$ 
42:       return  $\text{old}$ 
43:     else
44:        $b \leftarrow \text{axis} \bmod 2$ 
45:        $a \leftarrow (\text{axis} - b)/2$ 
46:        $\text{parent} \leftarrow \backslash(a, \text{old})$ 
47:       if  $= (b, 0)$  then
48:          $\text{new} \rightarrow \text{head}(\text{parent})$ 
49:         return  $\#(a, \text{parent}, \text{old})$ 
50:       else
51:          $\text{new} \rightarrow \text{tail}(\text{parent})$ 
52:         return  $\#(a, \text{parent}, \text{old})$ 
53:       end if
54:     end if
55:   end if
56:   return  $\perp$ 
57: end procedure
```

Algorithm 3 Nock: Ordering Operator

```
1: procedure -(noun  $i$ , noun  $n$ )
2:   if type( $i$ ) = atom then
3:     if  $i = 3$  then return ?( $n$ )
4:     else if  $i = 4$  then return +( $n$ )
5:   end if
6: end if
7:   return  $\perp$ 
8: end procedure
```

Algorithm 4 Nock Transition Function

```
1: procedure NOCK(verb  $T_i$ , verb  $T_{i+1}$ )
2:   Parse (actions, noun) $_i \leftarrow T_i$ , index  $\leftarrow$  getIndex( $T_i$ )
3:    $F_i \leftarrow$  map( $T_i$ , index),  $F_{i+1} \leftarrow$  map( $T_{i+1}$ , index)
4:   Check that  $T_i \setminus F_i = T_{i+1} \setminus F_{i+1}$ , returning 1 if false
5:   if type(tail( $F_i$ ))  $\neq$  cell then
6:     if  $F_{i+1} = \perp$  then return 0
7:   end if
8:   return 1
9: end if
10: if type(head(tail( $F_i$ ))) = atom then
11:   if head(tail( $F_i$ ))  $\notin$  [11], then
12:     if  $F_{i+1} = \perp$  then return 0
13:   end if
14:   else
15:     if  $F_{i+1} = \text{OP}_{\text{head}(\text{tail}(F_i))} : F_i$  then return 0
16:   end if
17:   end if
18: else
19:   if type(head(tail( $F_i$ ))) = cell then
20:     if  $F_{i+1} = \text{cons} : F_i$  then return 0
21:   end if
22:   end if
23: end if
24:   return 1
25: end procedure
```

Algorithm 5 NockCompute Verifier V_{comp}

```
1: procedure  $V_{\text{comp}}(x_{\text{comp}}^{\text{comp}}, x_{\text{out}}^{\text{comp}}; W_{\text{comp}})$ 
2:   Check that  $((\text{input\_formula}, \text{formula\_tail}), 3, \text{ln\_prev} \cdot z) = \text{mem\_record}_1$ , reject otherwise
3:   Check that  $\text{cons}(\text{formula\_head}, \text{formula\_tail}) = \text{input\_formula}$ , reject otherwise
4:   if  $\text{formula\_head} \in \{2, 5, 6, 7, 8, 9\}$  then
5:     Check that  $((\text{formula\_tail}, \text{formula\_tail\_head}), 2, \text{ln\_prev} \cdot z^2) = \text{mem\_record}_2$ , reject otherwise
6:     Check that  $\text{cons}(\text{formula\_tail\_head}, \text{formula\_tail\_tail}) = \text{formula\_tail}$ , reject otherwise
7:     if  $\text{formula\_head} \in \{6\}$  then
8:       Check  $\text{ln} = \text{ln\_prev} \cdot z^3$ ,  $((\text{formula\_tail\_tail}, \text{formula\_tail\_tail\_head}), 2, \text{ln}) = \text{mem\_record}_3$ , reject o/w
9:       Check that  $\text{cons}(\text{formula\_tail\_tail\_head}, \text{formula\_tail\_tail\_tail}) = \text{formula\_tail\_tail}$ , reject o/w
10:    else if  $\text{formula\_head} \in \{9\}$  then
11:      Check  $\text{ln} = \text{ln\_prev} \cdot z^3$ , reject o/w
12:      Check  $\text{mem\_root}_3 = \text{sf\_evaluation}_1$ ,  $\text{mem\_axis}_3 = \text{formula\_tail\_head}$ ,  $\text{timestamp}_3 = \text{ln}$ , reject o/w
13:    else Check that  $\text{ln} = \text{ln\_prev} \cdot z^2$ ,  $\text{mem\_record}_3 = (0, 0, 0, 0)$ , reject otherwise
14:    end if
15:  else if  $\text{state} \neq \text{OP}_0$  then check that  $\text{ln} = \text{ln\_prev} \cdot z$ ,  $\text{mem\_record}_{2,3} = (0, 0, 0, 0)$ , reject otherwise
16:  else Check  $\text{ln} = \text{ln\_prev} \cdot z^2$ ,  $\text{ln} = \text{timestamp}_2$ ,  $\text{mem\_record}_3 = (0, 0, 0, 0)$ , reject otherwise
17:  end if
18:  if  $\text{type}(\text{formula\_head}) = \text{cell}$  then check that  $\text{state} = \text{cons}$ , reject otherwise
19:     $\text{child\_one\_input} \leftarrow (\text{input\_subject}, \text{formula\_head})$ ,  $\text{child\_two\_input} \leftarrow (\text{input\_subject}, \text{formula\_tail})$ 
20:    Check that  $\text{evaluation} = \text{cons}(\text{sf\_evaluation}_1, \text{sf\_evaluation}_2)$ , reject otherwise
21:    Define  $\text{child\_three} \leftarrow (\perp, \perp)$ , and check  $\text{sf\_evaluation}_3 = \perp$ , reject otherwise
22:  else if  $\text{formula\_head} = 0$  then check that  $\text{state} = \text{OP}_0$ , reject otherwise
23:    Check that  $\text{evaluation} = \text{mem\_child}_2$ ,  $\text{input\_subject} = \text{mem\_root}_2$ ,  $\text{mem\_axis}_2 = \text{formula\_tail}$  reject o/w
24:    Define  $\text{child\_one}, \text{child\_two}, \text{child\_three} \leftarrow (\perp, \perp)$ , and check  $\text{sf\_evaluation}_{1,2,3} = \perp$ , reject o/w
25:  else if  $\text{formula\_head} = 1$  then check that  $\text{state} = \text{OP}_1$ , reject otherwise
26:    Check that  $\text{evaluation} = \text{formula\_tail}$ , reject otherwise
27:    Define  $\text{child\_one}, \text{child\_two}, \text{child\_three} \leftarrow (\perp, \perp)$ , and check  $\text{sf\_evaluation}_{1,2,3} = \perp$ , reject otherwise
28:  else if  $\text{formula\_head} = 2$  then check that  $\text{state} = \text{OP}_2$ , reject otherwise
29:     $\text{child\_one\_input} \leftarrow (\text{input\_subject}, \text{formula\_tail\_head})$ ,  $\text{child\_two\_input} \leftarrow (\text{input\_subject}, \text{formula\_tail\_tail})$ 
30:     $\text{child\_three\_input} \leftarrow (\text{sf\_evaluation}_1, \text{sf\_evaluation}_2)$ 
31:    Check that  $\text{evaluation} = \text{sf\_evaluation}_3$ , reject otherwise
32:  else if  $\text{formula\_head} = 3$  then check that  $\text{state} = \text{OP}_3$ , reject otherwise
33:     $\text{child\_one\_input} \leftarrow (\text{input\_subject}, \text{formula\_tail})$ , check that  $\text{evaluation} = ?(\text{sf\_evaluation}_1)$ , reject o/w
34:    Define  $\text{child\_two}, \text{child\_three} \leftarrow (\perp, \perp)$ , and check  $\text{sf\_evaluation}_{2,3} = \perp$ , reject otherwise
35:  else if  $\text{formula\_head} = 4$  then check that  $\text{state} = \text{OP}_4$ , reject otherwise
36:     $\text{child\_one\_input} \leftarrow (\text{input\_subject}, \text{formula\_tail})$ , check that  $\text{evaluation} = +(\text{sf\_evaluation}_1)$ , reject o/w
37:    Define  $\text{child\_two}, \text{child\_three} \leftarrow (\perp, \perp)$ , and check  $\text{sf\_evaluation}_{2,3} = \perp$ , reject otherwise
38:  else if  $\text{formula\_head} = 5$  then check that  $\text{state} = \text{OP}_5$ , reject otherwise
39:     $\text{child\_one\_input} \leftarrow (\text{input\_subject}, \text{formula\_tail\_head})$ ,  $\text{child\_two\_input} \leftarrow (\text{input\_subject}, \text{formula\_tail\_tail})$ 
40:    Check that  $\text{evaluation} = ?(\text{sf\_evaluation}_1, \text{sf\_evaluation}_2)$ , reject otherwise
41:    Define  $\text{child\_three} \leftarrow (\perp, \perp)$ , and check  $\text{sf\_evaluation}_3 = \perp$ , reject otherwise
42:  else if  $\text{formula\_head} = 6$  then, check that  $\text{state} = \text{OP}_6$ , reject otherwise
43:    Check that  $\text{type}(\text{sf\_evaluation}_2) = \text{atom}$  and that  $\text{sf\_evaluation}_2 \in \{0, 1\}$ , reject otherwise
44:    if  $\text{sf\_evaluation}_2 = 0$  then  $\text{child\_one\_input} \leftarrow (\text{input\_subject}, \text{formula\_tail\_tail\_head})$ 
45:    else  $\text{child\_one\_input} \leftarrow (\text{input\_subject}, \text{formula\_tail\_tail\_tail})$ 
46:    end if
47:     $\text{child\_two\_input} \leftarrow (\text{input\_subject}, \text{formula\_tail\_head})$ , check that  $\text{evaluation} = \text{sf\_evaluation}_1$ , reject o/w
48:    Define  $\text{child\_three} \leftarrow (\perp, \perp)$ , and check  $\text{sf\_evaluation}_3 = \perp$ , reject otherwise
49:  else if  $\text{formula\_head} = 7$  then check that  $\text{state} = \text{OP}_7$ , reject otherwise
50:     $\text{child\_one\_input} \leftarrow (\text{sf\_evaluation}_1, \text{formula\_tail\_tail})$ 
51:     $\text{child\_two\_input} \leftarrow (\text{input\_subject}, \text{formula\_tail\_head})$ 
52:    Check that  $\text{evaluation} = \text{sf\_evaluation}_1$ , reject otherwise
53:    Define  $\text{child\_three} \leftarrow (\perp, \perp)$ , and check  $\text{sf\_evaluation}_3 = \perp$ , reject otherwise
54:  else if  $\text{formula\_head} = 8$  then check that  $\text{state} = \text{OP}_8$ , reject otherwise
55:     $\text{child\_one\_input} \leftarrow (\text{cons}(\text{sf\_evaluation}_2, \text{input\_subject}), \text{formula\_tail\_tail})$ 
56:     $\text{child\_two\_input} \leftarrow (\text{input\_subject}, \text{formula\_tail\_head})$ 
57:    Check that  $\text{evaluation} = \text{sf\_evaluation}_1$ , reject otherwise
58:    Define  $\text{child\_three} \leftarrow (\perp, \perp)$ , and check  $\text{sf\_evaluation}_3 = \perp$ , reject otherwise
59:  else if  $\text{formula\_head} = 9$  then, check that  $\text{state} = \text{OP}_9$ , reject otherwise
60:     $\text{child\_one\_input} \leftarrow (\text{input\_subject}, \text{formula\_tail\_tail})$ 
61:     $\text{child\_two\_input} \leftarrow (\text{sf\_evaluation}_1, \text{mem\_child}_3)$ 
62:    Check that  $\text{evaluation} = \text{sf\_evaluation}_2$ , reject otherwise
63:    Define  $\text{child\_three} \leftarrow (\perp, \perp)$ , and check  $\text{sf\_evaluation}_3 = \perp$ , reject otherwise
64:  else Reject
65:  end if
66:  return Accept
```

Algorithm 6 NockMemory Verifier V_{mem}

```
1: procedure  $V_{\text{mem}}(x_{\text{mem}}^{\text{mem}}, x_{\text{out}}^{\text{mem}}; W_{\text{mem}})$ 
2:   if state = decompose then
3:     Check that mem_record = (0, 0, 0, 0), reject otherwise
4:     Check that cons(left_child, right_child) = parent, reject otherwise
5:     Check that  $2 \cdot \text{axis\_counts}_P = \text{axis\_counts}_L + \text{axis\_counts}_R - \text{axis\_indicator}_R$ , reject otherwise
6:     Check that  $\text{axis\_indicator}_P = \text{axis\_indicator}_L + \text{axis\_indicator}_R$ , reject otherwise
7:   else if state = halt then
8:     Check that mem_record = (0, 0, 0, 0), reject otherwise
9:     Check that  $\text{axis\_counts}_P, \text{axis\_indicator}_P = 0$ , reject otherwise
10:    Check that  $x_{\text{out}}^{\text{mem}} = \perp$ , reject otherwise
11:   else if state = read_root then
12:     Parse ((mem_root, mem_child), axis, timestamp)  $\leftarrow$  mem_record
13:     Check that mem_root = parent, reject otherwise
14:     Check that left_child = parent, left_state = state, reject otherwise
15:     Check that  $\text{axis\_counts}_P = \text{axis\_counts}_L - \text{timestamp}$ , reject otherwise
16:     Check that  $\text{axis\_indicator}_P = \text{axis\_indicator}_L - \text{timestamp}$ , reject otherwise
17:     Check that right_child, right_state, axis_indicator_R, axis_counts_R =  $\perp$ , reject otherwise
18:     Check that  $\text{intermediate}_{i+1} - \text{intermediate}_i = (\text{RLC}(\text{mem\_record}) + X)^{-1}$ , reject otherwise
19:   else if state = verify_child then
20:     Parse ((mem_root, mem_child), axis, timestamp)  $\leftarrow$  mem_record
21:     Check that mem_child = parent, reject otherwise
22:     Check that left_child = parent, left_state = state, reject otherwise
23:     Check that  $\text{axis\_counts}_P = \text{axis\_counts}_L + \text{axis} \cdot \text{timestamp}$ , reject otherwise
24:     Check that  $\text{axis\_indicator}_P = \text{axis\_indicator}_L + \text{timestamp}$ , reject otherwise
25:     Check that right_child, right_state, axis_indicator_R, axis_counts_R =  $\perp$ , reject otherwise
26:     Check that  $\text{intermediate}_i - \text{intermediate}_{i+1} = (\text{RLC}(\text{mem\_record}) + X)^{-1}$ , reject otherwise
27:   else Reject
28:   end if
29:   return Accept
30: end procedure
```
